

OSGi Working Group
OSGi Specification repo for
`org.osgi.service.dmtRelease`

2026 SNAPSHOT

September 2026



LICENSE

Eclipse Foundation Specification License – v1.0

By using and/or copying this document, or the Eclipse Foundation document from which this statement is linked, you (the licensee) agree that you have read, understood, and will comply with the following terms and conditions:

Permission to copy, and distribute the contents of this document, or the Eclipse Foundation document from which this statement is linked, in any medium for any purpose and without fee or royalty is hereby granted, provided that you include the following on ALL copies of the document, or portions thereof, that you use:

- link or URL to the original Eclipse Foundation document.
- All existing copyright notices, or if one does not exist, a notice (hypertext is preferred, but a textual representation is permitted) of the form: "Copyright © [date-of-document] Eclipse Foundation, Inc. <<url to this license>>".

Inclusion of the full text of this NOTICE must be provided. We request that authorship attribution be provided in any software, documents, or other items or products that you create pursuant to the implementation of the contents of this document, or any portion thereof.

No right to create modifications or derivatives of Eclipse Foundation documents is granted pursuant to this license, except anyone may prepare and distribute derivative works and portions of this document in software that implements the specification, in supporting materials accompanying such software, and in documentation of such software, PROVIDED that all such works include the notice below. HOWEVER, the publication of derivative works of this document for use as a technical specification is expressly prohibited.

The notice is:

"Copyright © [date-of-document] Eclipse Foundation. This software or document includes material copied from or derived from [title and URI of the Eclipse Foundation specification document]."

Disclaimers

THIS DOCUMENT IS PROVIDED "AS IS," AND THE COPYRIGHT HOLDERS AND THE ECLIPSE FOUNDATION MAKE NO REPRESENTATIONS OR WARRANTIES, EXPRESS OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, NON-INFRINGEMENT, OR TITLE; THAT THE CONTENTS OF THE DOCUMENT ARE SUITABLE FOR ANY PURPOSE; NOR THAT THE IMPLEMENTATION OF SUCH CONTENTS WILL NOT INFRINGE ANY THIRD PARTY PATENTS, COPYRIGHTS, TRADEMARKS OR OTHER RIGHTS.

THE COPYRIGHT HOLDERS AND THE ECLIPSE FOUNDATION WILL NOT BE LIABLE FOR ANY DIRECT, INDIRECT, SPECIAL OR CONSEQUENTIAL DAMAGES ARISING OUT OF ANY USE OF THE DOCUMENT OR THE PERFORMANCE OR IMPLEMENTATION OF THE CONTENTS THEREOF.

The name and trademarks of the copyright holders or the Eclipse Foundation may NOT be used in advertising or publicity pertaining to this document or its contents without specific, written prior permission. Title to copyright in this document will at all times remain with copyright holders.

Preface

Implementation Requirements

An implementation of a Specification: (i) must fully implement the Specification including all its required interfaces and functionality; (ii) must not modify, subset, superset or otherwise extend the OSGi Name Space, or include any public or protected packages, classes, Java interfaces, fields or methods within the OSGi Name Space other than those required and authorized by the Specification. An implementation that does not satisfy limitations (i) (ii) is not considered an implementation of the Specification and must not be described as an implementation of the Specification. "OSGi Name Space" shall mean the public class or interface declarations whose names begin with "org.osgi" or any recognized successors or replacements thereof. An implementation of a Specification must not claim to be a compatible implementation of the Specification unless it passes the Technology Compatibility Kit ("TCK") for the Specification.

Feedback

This specification can be downloaded from the OSGi Documentation web site:

<https://docs.osgi.org/specification/>

Comments about this specification can be raised at:

<https://github.com/osgi/osgi/issues>

Table of Contents

1	Introduction	5
1.1	Reader Level	5
1.2	Version Information	5
1.3	References	5
117	Dmt Admin Service Specification	7
117.1	Introduction	7
117.2	The Device Management Model	10
117.3	The DMT Admin Service	13
117.4	Manipulating the DMT	13
117.5	Meta Data	21
117.6	Plugins	24
117.7	Sharing the DMT	30
117.8	Access Control Lists	37
117.9	Notifications	41
117.10	Exceptions	43
117.11	Events	43
117.12	OSGi Object Modeling	49
117.13	Security	57
117.14	org.osgi.service.dmt	61
117.15	org.osgi.service.dmt.spi	110
117.16	org.osgi.service.dmt.notification	125
117.17	org.osgi.service.dmt.notification.spi	128
117.18	org.osgi.service.dmt.security	129
117.19	References	134

Licensed under the Eclipse Foundation Specification License – v1.0. Copyright © Contributors to the Eclipse Foundation

DRAFT

1 Introduction

This document contains a specification developed by the OSGi working group.

1.1 Reader Level

This specification is written for the following audiences:

- Application developers
- Framework and system service developers (system developers)
- Architects

This specification assumes that the reader has at least one year of practical experience in writing Java programs. Experience with embedded systems and server-environments is a plus. Application developers must be aware that the OSGi environment is significantly more dynamic than traditional desktop or server environments.

System developers require a very deep understanding of Java. At least three years of Java coding experience in a system environment is recommended. A Framework implementation will use areas of Java that are not normally encountered in traditional applications. Detailed understanding is required of class loaders, garbage collection, Java 2 security, and Java native library loading.

Architects should focus on the introduction of each subject. This introduction contains a general overview of the subject, the requirements that influenced its design, and a short description of its operation as well as the entities that are used. The introductory sections require knowledge of Java concepts like classes and interfaces, but should not require coding experience.

Most of these specifications are equally applicable to application developers and system developers.

1.2 Version Information

Each OSGi specification has an independent version, and packages defined by those specifications have their own versions as well. Specification and package versions follow semantic versioning rules.

1.3 References

- [1] *OSGi Specifications*
<https://docs.osgi.org/specification/>

DRAFT

Licensed under the Eclipse Foundation Specification License – v1.0. Copyright © Contributors to the Eclipse Foundation

117 Dmt Admin Service Specification

Version 2.0

117.1 Introduction

There are a large number of Device Management standards available today. Starting with the ITU X.700 series in the seventies, SNMP in the eighties and then an explosion of different protocols when the use of the Internet expanded in the nineties. Many device management standards have flourished, and some subsequently withered, over the last decades. Some examples:

- X.700 CMIP
- IETF SNMP
- IETF LDAP
- OMA DM
- Broadband Forum TR-069
- UPnP Forum's Device Management
- IETF NETCONF
- OASIS WS Distributed Management

This heterogeneity of the remote management for OSGi Framework based devices is a problem for device manufacturers. Since there is often no dominant protocol these manufacturers have to develop multiple solutions for different remote management protocols. It is also problematic for device operators since they have to choose a specific protocol but by that choice could exclude a class of devices that do not support that protocol. There is therefore a need to allow the use of multiple protocols at minimal costs.

Almost all management standards are based on hierarchical object models and provide *primitives* like:

- Get and replace values
- Add/Remove instances
- Discovery of value names and instance ids
- Provide notifications

A Device Management standard consists of a *protocol stack* and a number of *object models*. The protocol stack is generic and shared for all object types; the object model describes a specific device's properties and methods. For example, the protocol stack can consist of a set of SOAP message formats and an object model is a Deployment Unit. An object model consists of a data model and sometimes a set of functions.

The core problem is that the generic Device Management Tree must be mapped to device specific functions. This specification therefore defines an API for managing a device using general device management concepts but providing an effective plugin model to link the generic tree to the specific device functions.

The API is decomposed in the following packages/functionality:

- `org.osgi.service.dmt` - Main package that provides access to the local Device Management Tree. Access is session based.

- `org.osgi.service.dmt.notification` - The notification package provides the capability to send alerts to a management server.
- `org.osgi.service.dmt.spi` - Provides the capability to register subtree handlers in the Device Management Tree.
- `org.osgi.service.dmt.notification.spi` - The API to provide the possibility to extend the notification system.
- `org.osgi.service.dmt.security` - Permission classes.

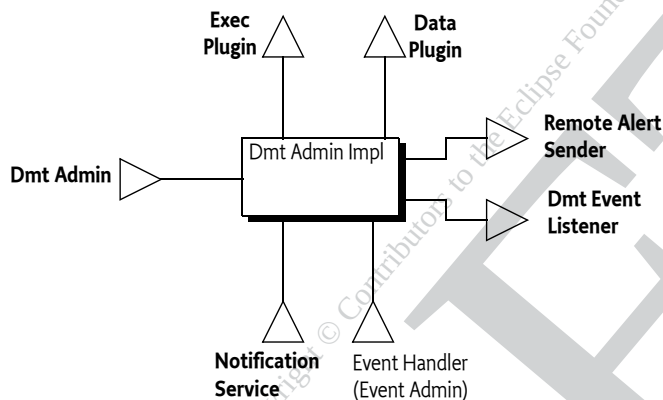
117.1.1

Entities

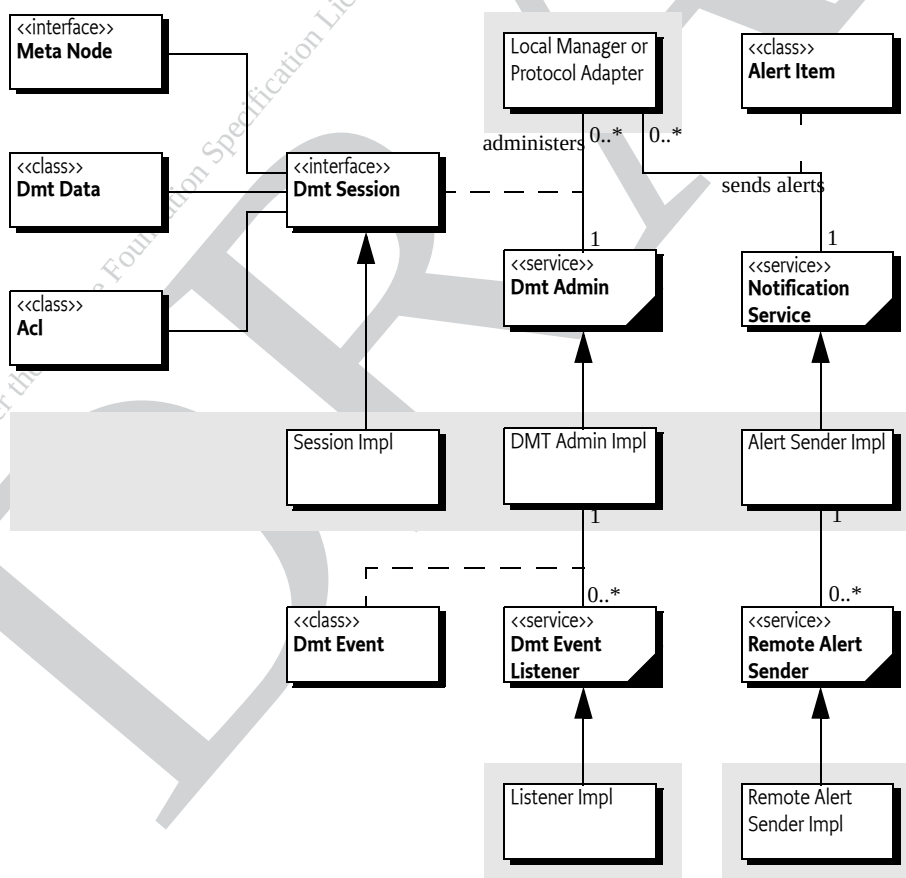
- *Device Management Tree* - The Device Management Tree (DMT) is the logical view of manageable aspects of an OSGi Environment, implemented by plugins and structured in a tree with named nodes.
- *Dmt Admin* - A service through which the DMT can be manipulated. It is used by *Local Managers* or by *Protocol Adapters* that initiate DMT operations. The Dmt Admin service forwards selected DMT operations to Data Plugins and execute operations to Exec Plugins; in certain cases the Dmt Admin service handles the operations itself. The Dmt Admin service is a singleton.
- *Dmt Session* - A session groups a set of operations on a sub-tree with optional transactionality and locking. Dmt Session objects are created by the Dmt Admin service and are given to a plugin when they first join the session.
- *Local Manager* - A bundle which uses the Dmt Admin service directly to read or manipulate the DMT. Local Managers usually do not have a principal associated with the session.
- *Protocol Adapter* - A bundle that communicates with a management server external to the device and uses the Dmt Admin service to operate on the DMT. Protocol Adapters usually have a principal associated with their sessions.
- *Meta Node* - Information provided by the node implementer about a node for the purpose of performing validation and providing assistance to users when these values are edited.
- *Multi nodes* - Interior nodes that have a homogeneous set of children. All these children share the same meta node.
- *Plugin* - Services which take the responsibility over a given sub-tree of the DMT: Data Plugin services and Exec Plugin services.
- *Data Plugin* - A Plugin that can create a Readable Data Session, Read Write Data Session, or Transactional Data Session for data operations on a sub-tree for a Dmt Session.
- *Exec Plugin* - A Plugin that can handle execute operations.
- *Readable Data Session* - A plugin session that can only read.
- *Read Write Data Session* - A plugin session that can read and write.
- *Transactional Data Session* - A plugin session that is transactional.
- *Principal* - Represents the optional identity of an initiator of a Dmt Session. When a session has a principal, the Dmt Admin must enforce ACLs and must ignore Dmt Permissions.
- *ACL* - An Access Control List is a set of principals that is associated with permitted operations.
- *Dmt Event* - Information about a modification of the DMT.
- *Dmt Event Listener* - Listeners to Dmt Events. These listeners are services according to the white board pattern.
- *Mount Point* - A point in the DMT where a Plugin or the Dmt Admin service allows other Plugins to have their root.

The overall service interaction diagram is depicted in Figure 117.1 .

Figure 117.1 Overall Service Diagram

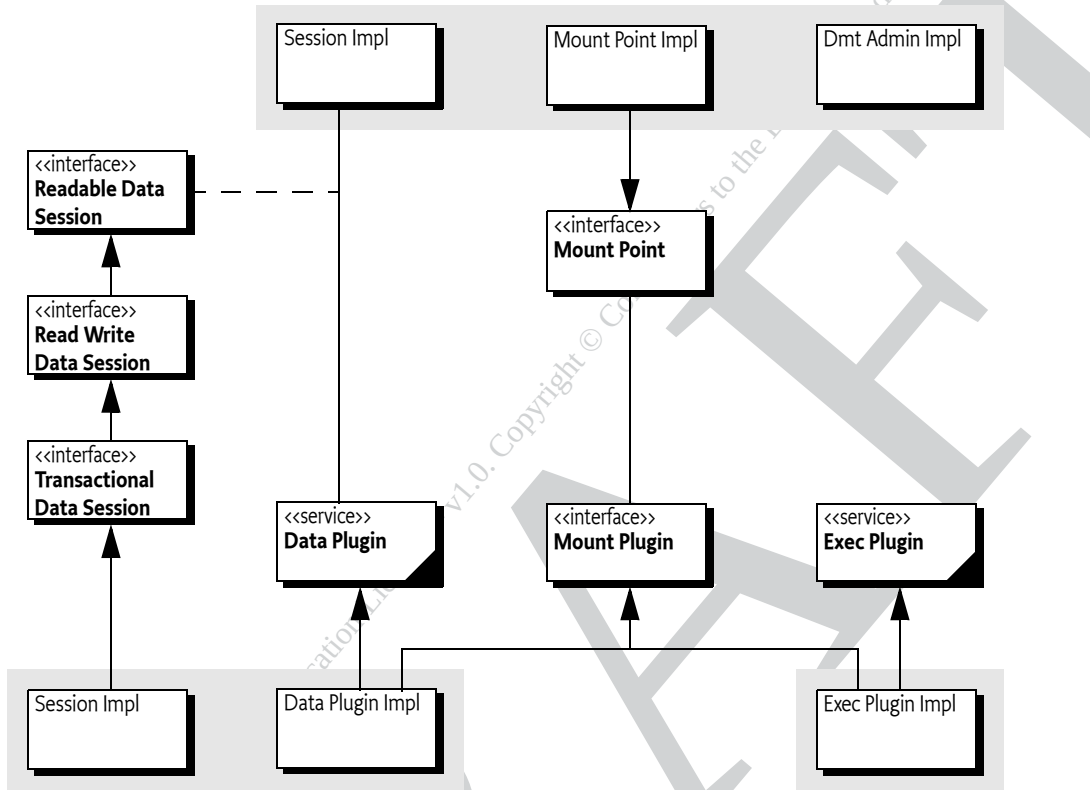


The entities used in the Dmt Admin operations and notifications are depicted in Figure 117.2 .

Figure 117.2 Using Dmt Admin service, *org.osgi.service.dmt* and *org.osgi.service.dmt.notification.** packages

Extending the Dmt Admin service with Plugins is depicted in Figure 117.3 .

Figure 117.3

Extending the Dmt Admin service, *org.osgi.service.dmt.spi* package

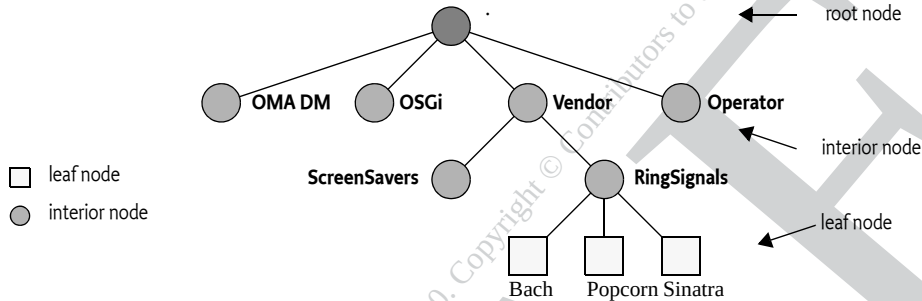
117.2 The Device Management Model

The standard-based features of the DMT model are:

- The Device Management Tree consists of *interior* nodes and *leaf* nodes. Interior nodes can have children and leaf nodes have primitive values.
- All nodes have a set of properties: Name, Title, Format, ACL, Version, Size, Type, Value, and TimeStamp.
- The storage of the nodes is undefined. Nodes typically map to peripheral registers, settings, configuration, databases, etc.
- A node's name must be unique among its siblings.
- Nodes can have Access Control Lists (ACLs), associating operations allowed on those nodes with a particular principal.
- Nodes can have Meta Nodes that describe actual nodes and their siblings.
- Base value types (called *formats* in the standard) are
 - integer
 - long
 - string
 - boolean
 - binary data (multiple types)
 - datetime
 - time

- float
- XML fragments
- Leaf nodes in the tree can have default values specified in the meta node.
- Meta Nodes define allowed access operations (Get, Add, Replace, Delete and Exec)

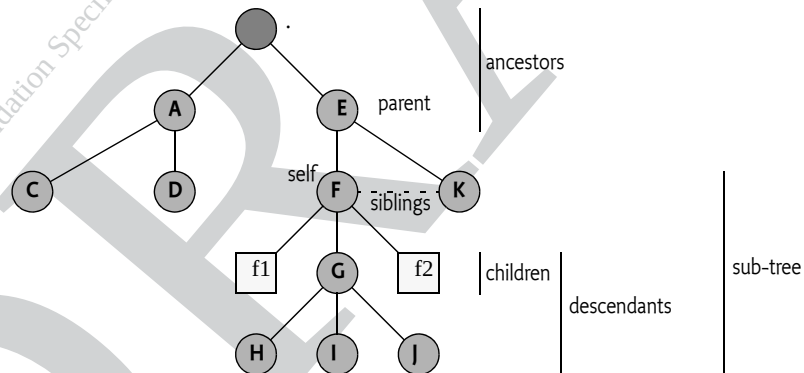
Figure 117.4 Device Management Tree example



117.2.1 Tree Terminology

In the following sections, the DMT is discussed frequently. Thus, well-defined terms for all the concepts that the DMT introduces are needed. The different terms are shown in Figure 117.5.

Figure 117.5 DMT naming, relative to node F



All terms are defined relative to node F. For this node, the terminology is as follows:

- *URI* - The path consisting of node names that uniquely defines a node, see *The DMT Addressing URI* on page 13 .
- *ancestors* - All nodes that are above the given node ordered in proximity. The closest node must be first in the list. In the example, this list is `[./E, ./]`
- *parent* - The first ancestor, in this example this is `./E`.
- *children* - A list of nodes that are directly beneath the given node without any preferred ordering. For node F this list is `{ ./E/F/f1, ./E/F/f2, ./E/F/G }`.
- *siblings* - An unordered list of nodes that have the same parent. All siblings must have different names. For F, this is `{ ./E/K }`
- *descendants* - A list of all nodes below the given node. For F this is `{ ./E/F/f1, ./E/F/G, ./E/F/f2, ./E/F/G/H, ./E/F/G/I, ./E/F/G/J }`
- *sub-tree* - The given node plus the list of all descendants. For node F this is `{ ./E/F, ./E/F/f1, ./E/F/G, ./E/F/f2, ./E/F/G/H, ./E/F/G/I, ./E/F/G/J }`

- *overlap* - Two given URIs overlap if they share any node in their sub-trees. In the example, the sub-tree `./E/F` and `./E/F/G` overlap.
- *data root URI* - A URI which represents the root of a Data Plugin.
- *exec root URI* - A URI which represents the root of an Exec Plugin.
- *Parent Plugin* - A Plugin A is a Parent Plugin of Plugin B if B's root is in A's sub-tree, this requires a Parent Plugin to at least have one mount point.
- *Child Plugin* - A Plugin A is a Child Plugin of Plugin B if A's root is in B's sub-tree.
- *Scaffold Node* - An ancestor node of a Plugin that is managed by the Dmt Admin service to ensure that all nodes are discoverable by traversing from the root.

117.2.2 Actors

There are two typical users of the Dmt Admin service:

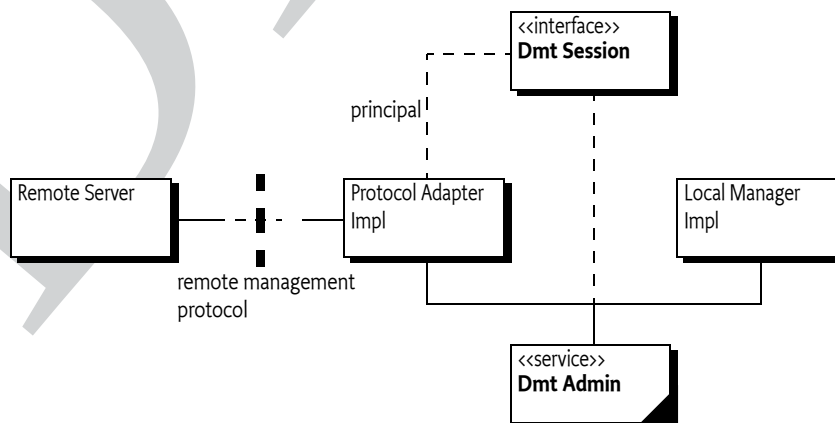
- *Remote manager* - The typical client of the Dmt Admin service is a *Protocol Adapter*. A management server external to the device can issue DMT operations over some management protocol. The protocol to be used is not specified by this specification. For example, OMA DM, TR-069, or others could be used. The protocol operations reach the Framework through the Protocol Adapter, which forwards the calls to the Dmt Admin service in a session. Protocol Adapters should authenticate the remote manager and set the principal in the session. This association will make the Dmt Admin service enforce the ACLs. This requires that the principal is equal to the server name.

The Dmt Admin service provides a facility to send notifications to the remote manager with the Notification Service.

- *Local Manager* - A bundle which uses the Dmt Admin service to operate on the DMT; for example, a GUI application that allows the end user to change settings through the DMT.

Although it is possible to manage some aspects of the system through the DMT, it can be easier for such applications to directly use the services that underlie the DMT; many of the management features available through the DMT are also available as services. These services shield the callers from the underlying details of the abstract, and sometimes hard to use DMT structure. As an example, it is more straightforward to use the Monitor Admin service than to operate upon the monitoring sub-tree. The local management application might listen to Dmt Events if it is interested in updates in the tree made by other entities, however, these events do not necessarily reflect the accurate state of the underlying services.

Figure 117.6 Actors



117.3 The DMT Admin Service

The Dmt Admin service operates on the Device Management Tree of an OSGi-based device. The Dmt Admin API is loosely modeled after the OMA DM protocol: the operations for Get, Replace, Add, Delete and Exec are directly available. The Dmt Admin is a singleton service.

Access to the DMT is session-based to allow for locking and transactionality. The sessions are, in principle, concurrent, but implementations that queue sessions can be compliant. The client indicates to the Dmt Admin service what kind of session is needed:

- *Exclusive Update Session* - Two or more updating sessions cannot access the same part of the tree simultaneously. An updating session must acquire an exclusive lock on the sub-tree which blocks the creation of other sessions that want to operate on an overlapping sub-tree.
- *Multiple Readers Session* - Any number of read-only sessions can run concurrently, but ongoing read-only sessions must block the creation of an updating session on an overlapping sub-tree.
- *Atomic Session* - An atomic session is the same as an exclusive update session, except that the session can be rolled back at any moment, undoing all changes made so far in the session. The participants must accept the outcome: rollback or commit. There is no prepare phase. The lack of full two phase commit can lead to error situations which are described later in this document; see *Plugins and Transactions* on page 26 .

Although the DMT represents a persistent data store with transactional access and without size limitations, the notion of the DMT should not be confused with a general purpose database. The intended purpose of the DMT is to provide a *dynamic view* of the management state of the device; the DMT model and the Dmt Admin service are designed for this purpose.

117.4 Manipulating the DMT

117.4.1 The DMT Addressing URI

The OMA DM limits URIs to the definition of a URI in [8] *RFC 2396 Uniform Resource Identifiers (URI): Generic Syntax* . The [Uri](#) utility classes handles nearly all escaping issues with a number of static methods. All URIs in any of the API methods can use the full Unicode character set. For example, the following URIs as used in Java code are valid URIs for the Dmt Admin service.

```
"/ACME © 2000/A/x"
"/ACME/Address/Street/9C, Avenue St. Drézéry"
```

This strategy has a number of consequences.

- A solidus ('/' \u002F) collides with the use of the solidus as separator of the node names. Solidi must therefore be escaped using a reverse solidus ('\ ' \u005C). The reverse solidus must be escaped with a double reverse solidus sequence. The Dmt Admin service must ignore a reverse solidus when it is not followed by a solidus or reverse solidus. The solidus and reverse solidus must not be escaped using the %oo like escaping defined for URIs. For example, a node that has the name of a MIME type could look like:

```
./OSGi/mime/application\png
```

In Java, a reverse solidus must be escaped as well, therefore requiring double reverse solidi:

```
String a = "./OSGi/mime/application\\png";
```

A literal reverse solidus would therefore require 4 reverse solidi in a Java string.

- The length of a node name is defined to be the length of the byte array that results from UTF-8 encoding a string.

The `Uri` class provides an `encode(String)` method to escape a string and a `decode(String)` method to unescape a string. Though in general the Dmt Admin service implementations should not impose unnecessary constraints on the node name length, it is possible that an implementation runs out of space. In that case it must throw a `DmtException` `URI_TOO_LONG`.

Nodes are addressed by presenting a *relative* or *absolute* URI for the requested node. The URI is defined with the following grammar:

```
uri           ::= relative-uri | absolute-uri
absolute-uri  ::= './' relative-uri
relative-uri  ::= segment ( '/' segment ) *
segment       ::= ( - [ '/' ] ) *
```

The `Uri isAbsoluteUri(String)` method makes it simple to find out if a URI is relative or absolute. Relative URIs require a base URI that is for example provided by the session, see *Locking and Sessions* on page 14 .

Each node name is appended to the previous ones using a solidus ('/' \u002F) as the separating character. The first node of an absolute URI must be the full stop ('.' \u002E). For example, to access the Bach leaf node in the RingTones interior node from Figure 117.4 on page 11 , the URI must be:

```
./Vendor/RingSignals/Bach
```

The URI must be given with the root of the management tree as the starting point. URIs used in the DMT must be treated and interpreted as *case-sensitive*. I.e. `./Vendor` and `./vendor` designate two different nodes. The following mandatory restrictions on URI syntax are intended to simplify the parsing of URIs.

The full stop has no special meaning in a node name. That is, sequences like `..` do not imply parent node. The `isValidUri(String)` method verifies that a URI fulfills all its obligations and is valid.

117.4.2 Locking and Sessions

The Dmt Admin service is the main entry point into the DMT, its usage is to create sessions. A simple example is getting a session on a specific sub-tree. Such a session can be created with the `getSession(String)` method. This method creates an updating session with an exclusive lock on the given sub-tree. The given sub-tree can be a single leaf node, if so desired.

Each session has an ID associated with it which is unique to the machine and is never reused. This id is always greater than 0. The value -1 is reserved as place holder to indicate a situation has no session associated with it, for example an event generated from an underlying service. The URI argument addresses the sub-tree root. If null, it addresses the root of the DMT. All nodes can be reached from the root, so specifying a session root node is not strictly necessary but it permits certain optimizations in the implementations.

If the default exclusive locking mode of a session is not adequate, it is possible to specify the locking mode with the `getSession(String,int)` and `getSession(String,String,int)` method. These methods supports the following locking modes:

- `LOCK_TYPE_SHARED` - Creates a *shared session*. It is limited to read-only access to the given sub-tree, which means that multiple sessions are allowed to read the given sub-tree at the same time.
- `LOCK_TYPE_EXCLUSIVE` - Creates an *exclusive session*. The lock guarantees full read-write access to the tree. Such sessions, however, cannot share their sub-tree with any other session. This type of lock requires that the underlying implementation supports Read Write Data Sessions.
- `LOCK_TYPE_ATOMIC` - Creates an *atomic session* with an exclusive lock on the sub-tree, but with added transactionality. Operations on such a session must either succeed together or fail together.

er. This type of lock requires that the underlying implementation supports Transactional Data Sessions. If the Dmt Admin service does not support transactions, then it must throw a Dmt Exception with the `FEATURE_NOT_SUPPORTED` code. If the session accesses data plugins that are not transactional in write mode, then the Dmt Admin service must throw a Dmt Exception with the `TRANSACTION_ERROR` code. That is, data plugins can participate in atomic sessions as long as they only perform read operations.

The Dmt Admin service must lock the sub-tree in the requested mode before any operations are performed. If the requested sub-tree is not accessible, the `getSession(String,int)`, `getSession(String,String,int)`, or `getSession(String)` method must block until the sub-tree becomes available. The implementation can decide after an implementation-dependent period to throw a Dmt Exception with the `SESSION_CREATION_TIMEOUT` code.

As a simplification, the Dmt Admin service is allowed to lock the entire tree irrespective of the given sub-tree. For performance reasons, implementations should provide more fine-grained locking when possible.

Persisting the changes of a session works differently for exclusive and atomic sessions. Changes to the sub-tree in an atomic session are not persisted until the `commit()` or `close()` method of the session is called. Changes since the last transaction point can be rolled back with the `rollback()` method.

The `commit()` and `rollback()` methods can be called multiple times in a session; they do not close the session. The open, `commit()`, and `rollback()` methods all establish a *transaction point*. The rollback operation cannot roll back further than the last transaction point.

Once a fatal error is encountered (as defined by the DmtException `isFatal()` method), all successful changes must be rolled back automatically to the last transaction point. Non-fatal errors do not rollback the session. Any error/exception in the commit or rollback methods invalidates and closes the session. This can happen if, for example, the mapping state of a plugin changes that has its plugin root inside the session's sub-tree.

Changes in an exclusive session are persisted immediately after each separate operation. Errors do not roll back any changes made in such a session.

Due to locking and transactional behavior, a session of any type must be closed once it is no longer used. Locks must always be released, even if the `close()` method throws an exception.

Once a session is closed no further operations are allowed and manipulation methods must throw a Dmt Illegal State Exception when called. Certain information methods like for example `getState()` and `getRootUri()` can still be called for logging or diagnostic purposes. This is documented with the Dmt Session methods.

The `close()` or `commit()` method can be expected to fail even if all or some of the individual operations were successful. This failure can occur due to multi-node constraints defined by a specific implementation. The details of how an implementation specifies such constraints is outside the scope of this specification.

Events in an atomic session must only be sent at commit time.

117.4.3 Associating a Principal

Protocol Adapters must use the `getSession(String,String,int)` method which features the principal as the first parameter. The principal identifies the external entity on whose behalf the session is created. This server identification string is determined during the authentication process in a way specific to the management protocol.

For example, the identity of the OMA DM server can be established during the handshake between the OMA DM agent and the server. In the simpler case of OMA CP protocol, which is a one-way protocol based on WAP Push, the identity of the principal can be a fixed value.

117.4.4 Relative Addressing

All DMT operation methods are found on the session object. Most of these methods accept a *relative* or *absolute* URI as their first parameter: for example, the method `isLeafNode(String)`. This URI is absolute or relative to the sub-tree with which the session is associated. For example, if the session is opened on:

```
./Vendor
```

then the following URIs address the Bach ring tone:

```
RingTones/Bach
./Vendor/RingTones/Bach
```

Opening the session with a null URI is identical to opening the session at the root. But the absolute URI can be used to address the Bach ring tone as well as a relative URI.

```
./Vendor/RingTones/Bach
Vendor/RingTones/Bach
```

If the URI specified does not correspond to a legitimate node in the tree, a Dmt Exception must be thrown. The only exception to this rule is the `isNodeUri(String)` method that can verify if a node is actually valid. The `getMetaNode(String)` method must accept URIs to non-existing nodes if an applicable meta node is available; otherwise it must also throw a Dmt Exception.

117.4.5 Creating Nodes

The methods that create interior nodes are:

- `createInteriorNode(String)` - Create a new interior node using the default meta data. If the principal does not have Replace access rights on the parent of the new node then the session must automatically set the ACL of the new node so that the creating server has Add, Delete and Replace rights on the new node.
- `createInteriorNode(String,String)` - Create a new interior node. The meta data for this new node is identified by the second argument, which is a URI *identifying* an OMA DM Device Description Framework (DDF) file, this does not have to be a valid location. It uses a format like `org.osgi/1.0/LogManagementObject`. This meta node must be consistent with any meta information from the parent node.
- `createLeafNode(String)` - Create a new leaf node with a default value.
- `createLeafNode(String,DmtData)` - Create a leaf node and assign a value to the leaf-node.
- `createLeafNode(String,DmtData,String)` - Create a leaf node and assign a value for the node. The last argument is the MIME type, which can be null.

For a node to be created, the following conditions must be fulfilled:

- The URI of the new node has to be a valid URI.
- The principal of the Dmt Session, if present, must have ACL Add permission to add the node to the parent. Otherwise, the caller must have the necessary permission.
- All constraints of the meta node must be verified, including value constraints, name constraints, type constraints, and MIME type constraints. If any of the constraints fail, a Dmt Exception must be thrown with an appropriate code.

117.4.6 Node Properties

A DMT node has a number of runtime properties that can be set through the session object. These properties are:

- *Title* - (String) A human readable title for the object. The title is distinct from the node name. The title can be set with `setNodeTitle(String,String)` and read with `getNodeTitle(String)`. This

specification does not define how this information is localized. This property is optional depending on the implementation that handles the node.

- *Type* - (String) The MIME type, as defined in [9] *MIME Media Types*, of the node's value when it is a leaf node. The type of an interior node is a string identifying a DDF type. These types can be set with `setNodeType(String,String)` and read with `getNodeType(String)`.
- *Version* - (int) Version number, which must start at 0, incremented after every modification (for both a leaf and an interior node) modulo 0x10000. Changes to the value or any of the properties (including ACLs), or adding/deleting nodes, are considered changes. The `getNodeVersion(String)` method returns this version; the value is read-only. In certain cases, the underlying data structure does not support change notifications or makes it difficult to support versions. This property is optional depending on the node's implementation.
- *Size* - (int) The size measured in bytes is read-only and can be read with `getNodeSize(String)`. Not all nodes can accurately provide this information.
- *Time Stamp* - (Date) Time of the last change in version. The `getNodeTimestamp(String)` returns the time stamp. The value is read only. This property is optional depending on the node's implementation.
- *ACL* - The Access Control List for this and descendant nodes. The property can be set with `setNodeAcl(String,Acl)` and obtained with `getNodeAcl(String)`.

If a plugin that does not implement an optional property is accessed, a Dmt Exception with the code `FEATURE_NOT_SUPPORTED` must be thrown.

117.4.7 Setting and Getting Data

Values are represented as DmtData objects, which are immutable. They are acquired with the `getNodeValue(String)` method and set with the `setNodeValue(String,DmtData)` method.

DmtData objects are dynamically typed by an integer enumeration. In OMA DM, this integer is called the *format* of the data value. The format of the DmtData class is similar to the type of a variable in a programming language, but the word *format* is used here. The available data formats are listed in the following table.

Table 117.1

Data Formats

Format Type	Java Type	Format Name	Constructor	Get	Description
FORMAT_BASE64	byte[]	base64	DmtData(byte[],boolean)	getBase64()	Binary type that must be encoded with base 64, see [10] <i>RFC 3548 The Base16, Base32, and Base64 Data Encodings</i> .
FORMAT_BINARY	byte[]	binary	DmtData(byte[]) DmtData(byte[],boolean)	Dmt- getBinary()	A byte array. The DmtData object is created with the constructor. The byte array can only be acquired with the method.
FORMAT_BOOLEAN	boolean	boolean	DmtData(boolean)	getBoolean()	Boolean. There are two constants for this type: • FALSE_VALUE • TRUE_VALUE
FORMAT_DATE	String	date	DmtData(String,int)	getString() getDate()	A Date (no time). Syntax defined in [13] <i>XML Schema Part 2: Datatypes Second Edition</i> as the date type.

Format Type	Java Type	Format Name	Constructor	Get	Description
FORMAT_DATE_TIME	String	date-Time	DmtData(Date)	getDateTime()	A Date object representing a point in time.
FORMAT_FLOAT	float	float	DmtData(float)	getFloat()	Float
FORMAT_INTEGER	int	integer	DmtData(int)	getInt()	Integer
FORMAT_LONG	long	long	DmtData(long)	getLong()	Long
FORMAT_NODE	Object	NODE	DmtData(Object)	getNode()	A DmtData object can have a format of FORMAT_NODE . This value is returned from a MetaNode getFormat() method if the node is an interior node or for a data value when the Plugin supports complex values.
FORMAT_NULL					No valid data is available. DmtData objects with this format cannot be constructed; the only instance is the DmtData NULL_VALUE constant.
FORMAT_RAW_BINARY	byte[]	<custom>	DmtData(String,byte[])	getRawBinary()	A raw binary format is always created with a format name. This format name allows the creator to define a proprietary format. The format name is available from the getFormatName() method, which has predefined values for the standard formats.
FORMAT_RAW_STRING	String	<custom>	DmtData(String,String)	getRawString()	A raw string format is always created with a format name. This format name allows the creator to define a proprietary format. The format name is available from the getFormatName() method, which has predefined values for the standard formats.
FORMAT_STRING	String	string	DmtData(String)	getString()	String
FORMAT_TIME	String	time	DmtData(String,int)	getString()	Time of Day. Syntax defined in [13] <i>XML Schema Part 2: Datatypes Second Edition</i> as the time type.

Format Type	Java Type	Format Name	Constructor	Get	Description
FORMAT_XML	String	xml	DmtData(String,int)	getXml()	A string containing an XML fragment. It can be obtained with. The validity of the XML must not be verified by the Dmt Admin service.

117.4.8

Complex Values

The OMA DM model prescribes that only leaf nodes have primitive values. This model maps very well to remote managers. However, when a manager is written in Java and uses the Dmt Admin API to access the tree, there are often unnecessary conversions from a complex object, to leaf nodes, and back to a complex object. For example, an interior node could hold the current GPS position as an OSGi Position object, which consists of a longitude, latitude, altitude, speed, and direction. All these objects are Measurement objects which consist of value, error, and unit. Reading such a Position object through its leaf nodes only to make a new Position object is wasting resources. It is therefore that the Dmt Admin service also supports *complex values* as a supplementary facility.

If a complex value is used then the leaves must also be accessible and represent the same semantics as the complex value. A manager unaware of complex values must work correctly by only using the leaf nodes. Setting or getting the complex value of an interior node must be identical to setting or getting the leaf nodes.

Accessing a complex value requires Get access to the node and all its descendants. Setting a complex value requires Replace access to the interior node. Replacing a complex value must only generate a single Replace event.

Trying to set or get a complex value on an interior node that does not support complex values must throw a Dmt Exception with the code `FEATURE_NOT_SUPPORTED`.

117.4.9

Nodes and Types

The node's type can be set with the `setNodeType(String,String)` method and acquired with `getNodeType(String)`. The namespaces for the types differ for interior and leaf nodes. A leaf node is typed with a MIME type and an interior node is typed with a DDF Document URI. However, in both cases the Dmt Admin service must not verify the syntax of the type name.

The `createLeafNode(String,DmtData,String)` method takes a MIME type as last argument that will type the leaf node. The MIME type reflects how the data of the node should be *interpreted*. For example, it is possible to store a GIF and a JPEG image in a DmtData object with a `FORMAT_BINARY` format. Both the GIF and the JPEG object share the same *format*, but will have MIME types of `image/jpg` and `image/gif` respectively. The Meta Node provides a list of possible MIME types.

The `createInteriorNode(String,String)` method takes a DDF Document URI as the last argument that will type the interior node. This specification defines the DDF Document URIs listed in the following table for interior nodes that have a particular meaning in this specification.

Table 117.2 *Standard Interior Node Types*

Interior Node Type	Description
DDF_SCAFFOLD	Scaffold nodes are automatically generated nodes by the Dmt Admin service to provide the children node names so that Plugins are reachable from the root. See <i>Scaffold Nodes</i> on page 27 .
DDF_MAP	MAP nodes define a key -> value mapping construct using the node name (key) and the node value (value). See <i>MAP Nodes</i> on page 55 .

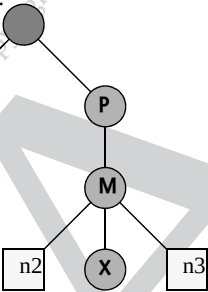
Interior Node Type	Description
DDF_LIST	LIST nodes use the node name to maintain an index in a list. See <i>LIST Nodes</i> on page 53 .

117.4.10 Deleting Nodes

The `deleteNode(String)` method on the session represents the Delete operation. It deletes the sub-tree of that node. This method is applicable to both leaf and interior nodes. Nodes can be deleted by the Dmt Admin service in any order. The root node of the session cannot be deleted.

For example, given Figure 117.7 , deleting node P must delete the nodes `./P`, `./P/M`, `./P/M/X`, `./P/M/n2` and `./P/M/n3` in any order.

Figure 117.7 DMT node and deletion



117.4.11 Copying Nodes

The `copy(String,String,boolean)` method on the `DmtSession` object represents the Copy operation. A node is completely copied to a new URI. It can be specified with a boolean if the whole sub-tree (true) or just the indicated node is copied.

The ACLs must not be copied; the new access rights must be the same as if the caller had created the new nodes individually. This restriction means that the copied nodes inherit the access rights from the parent of the destination node, unless the calling principal does not have Replace rights for the parent. See *Creating Nodes* on page 16 for details.

117.4.12 Renaming Nodes

The `renameNode(String,String)` method on the `DmtSession` object represents the Rename operation, which replaces the node name. It requires permission for the Replace operation. The root node for the current session can not be renamed.

117.4.13 Execute

The `execute(String,String)` and `execute(String,String,String)` methods can *execute* a node. Executing a node is intended to be used when a problem is hard to model as a set of leaf nodes. This can be related to synchronization issues or data manipulation. The execute methods can provide a correlator for a notification and an opaque string that is forwarded to the implementer of the node.

Execute operations can not take place in a read only session because simultaneous execution could make conflicting changes to the tree.

117.4.14 Closing

When all the changes have been made, the session must be closed by calling the `close()` method on the session. The Dmt Admin service must then finalize, clean up, and release any locks. For atomic sessions, the Dmt Admin service must automatically commit any changes that were made since the last transaction point.

A session times out and is invalidated after an extended period of inactivity. The exact length of this period is not specified, but is recommended to be at least 1 minute and at most 24 hours. All methods of an invalidated session must throw an Dmt Illegal State Exception after the session is invalidated.

A session's state is one of the following: `STATE_CLOSED`, `STATE_INVALID` or `STATE_OPEN`, as can be queried by the `getState()` call. The invalid state is reached either after a fatal error case is encountered or after the session is timed out. When an atomic session is invalidated, it is automatically rolled back to the last transaction point of the session.

117.5 Meta Data

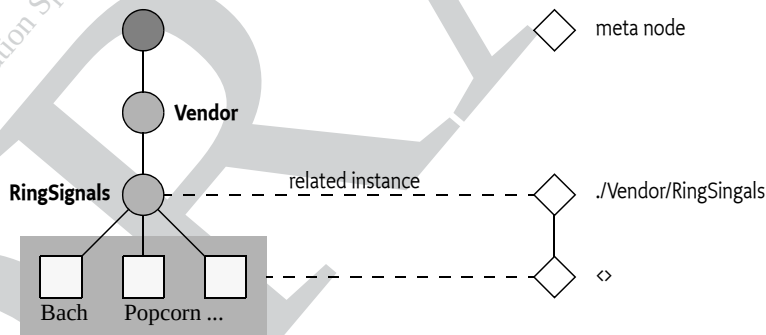
The `getMetaNode(String)` method returns a `MetaNode` object for a given URI. This node is called the *meta node*. A meta node provides information about nodes.

Any node can optionally have a meta node associated with it. The one or more nodes that are described by the meta nodes are called the meta node's *related instances*. A meta node can describe a singleton-related instance, or it can describe all the children of a given parent if it is a *multi-node*. That is to say, meta nodes can exist without an actual instance being present. In order to retrieve the meta node of a multi-node any name can be used.

For example, if a new ring tone, Grieg, was created in Figure 117.8 it would be possible to get the Meta Node for `./Vendor/RingSignals/Grieg` before the node was created. This is usually the case for multi nodes. The model is depicted in Figure 117.8.

Figure 117.8

Nodes and meta nodes



A URI is generally associated with the same Meta Node. The `getMetaNode(String)` should return the same meta node for the same URI except in the case of *Scaffold Nodes* on page 27. As the ownership of scaffold nodes can change from the Dmt Admin service to the Parent Plugin service, or from a Parent Plugin to a Child Plugin, the Meta Node can change as well.

The last segment of the URI to get a Meta Node can be any valid node name, for example, instead of Grieg it would have been possible to retrieve the same Meta Node with the name `./Vendor/RingSignals/o`, `./Vendor/RingSignals/anyName`, `./Vendor/RingSignals/<>`, etc.

The actual meta data can come from two sources:

- *Dmt Admin* - Each Dmt Admin service likely has a private meta data repository. This meta data is placed in the device in a proprietary way.
- *Plugins* - Plugins can carry meta nodes and provide these to the Dmt Admin service by implementing the `getMetaNode(String[])` method. If a plugin returns a non-null value, the Dmt Admin service must use that value, possibly complemented by its own metadata for elements not provided by the plugin.

The MetaNode interface supports methods to retrieve read-only meta data. The following sections describes this meta-data in more detail.

117.5.1 Operations

The `can(int)` method provide information as to whether the associated node can perform the given operation. This information is only about the capability; it can still be restricted in runtime by ACLs and permissions.

For example, if the `can(MetaNode.CMD_EXECUTE)` method returns true, the target object supports the Execute operation. That is, calling the `execute(String,String)` method with the target URI is possible.

The `can(int)` method can take the following constants as parameters:

- `CMD_ADD`
- `CMD_DELETE`
- `CMD_EXECUTE`
- `CMD_GET`
- `CMD_REPLACE`

For example:

```
void foo( DmtSession session, String nodeUri) {  
    MetaNode meta = session.getMetaNode(nodeUri);  
    if ( meta !=null && meta.can(MetaNode.CMD_EXECUTE) )  
        session.execute(nodeUri, "foo" );  
}
```

117.5.2 Scope

The scope is part of the meta information of a node. It provides information about what the life cycle role is of the node. The `getScope()` method on the Meta Node provides this information. The value of the scope can be one of the following:

- **DYNAMIC** - Dynamic nodes are intended to be created and deleted by a management system or an other controlling source. This does not imply that it actually is possible to add new nodes and delete nodes, the actions can still allow or deny this. However, in principle nodes that can be added or deleted have the **DYNAMIC** scope. The LIST and MAP nodes, see *OSGi Object Modeling* on page 49 , always have **DYNAMIC** scope.
- **PERMANENT** - Permanent nodes represent an entity in the system. This can be a network interface, a device description, etc. Permanent nodes in general map to an object in an object oriented language. Despite their name, **PERMANENT** nodes can appear and disappear, for example the plugging in of a USB device might create a new **PERMANENT** node. Generally, the Plugin roots map to **PERMANENT** nodes.
- **AUTOMATIC** - Automatic nodes map in general to nodes that are closely tied to the parent. They are similar to fields of an object in an object oriented language. They cannot be deleted or added.

For example, a node representing the Battery can never be deleted because it is an intrinsic part of the device; it will therefore be **PERMANENT** . The Level and number of ChargeCycle nodes will be **AUTOMATIC** . A new ring tone is dynamically created by a manager and is therefore **DYNAMIC** .

117.5.3 Description and Default

- `getDescription()` - (String) A description of the node. Descriptions can be used in dialogs with end users: for example, a GUI application that allows the user to set the value of a node. Localization of these values is not defined.

- [getDefault\(\)](#) - (DmtData) A default data value.

117.5.4 Validation

The validation information allows the runtime system to verify constraints on the values; it also allows user interfaces to provide guidance.

A node does not have to exist in the DMT in order to have meta data associated with it. Nodes may exist that have only partial meta data, or no metadata, associated with them. For each type of meta-data, the default value to assume when it is omitted is described in [MetaNode](#).

117.5.5 Data Types

A leaf node can be constrained to a certain format and one of a set of MIME types.

- [getFormat\(\)](#) - (int) The required type. This type is a logical OR of the supported formats.
- [getRawFormatNames\(\)](#) - Return an array of possible raw format names. This is only applicable when the [getFormat\(\)](#) returns the [FORMAT_RAW_BINARY](#) or [FORMAT_RAW_STRING](#) formats. The method must return null otherwise.
- [getMimeTypeTypes\(\)](#) - (String[]) A list of MIME types for leaf nodes or DDF types for interior nodes. The Dmt Admin service must verify that the actual type of the node is part of this set.

117.5.6 Cardinality

A meta node can constrain the number of *siblings* (i.e., not the number of children) of an interior or leaf node. This constraint can be used to verify that a node must not be deleted, because there should be at least one node left on that level ([isZeroOccurrenceAllowed\(\)](#)), or to verify that a node cannot be created, because there are already too many siblings ([getMaxOccurrence\(\)](#)).

If the cardinality of a meta node is more than one, all siblings must share the same meta node to prevent an invalid situation. For example, if a node has two children that are described by different meta nodes, and any of the meta nodes has a cardinality >1, that situation is invalid.

For example, the `./Vendor/RingSignals/<>` meta node (where <> stands for any name) could specify that there should be between 0 and 12 ring signals.

- [getMaxOccurrence\(\)](#) - (int) A value greater than 0 that specifies the maximum number of instances for this node.
- [isZeroOccurrenceAllowed\(\)](#) - (boolean) Returns true if zero instances are allowed. If not, the last instance must not be deleted.

117.5.7 Matching

The following methods provide validation capabilities for leaf nodes.

- [isValidValue\(DmtData\)](#) - (DmtData) Verify that the given value is valid for this meta node.
- [getValidValues\(\)](#) - (DmtData[]) A set of possible values for a node, or null otherwise. This can for example be used to give a user a set of options to choose from.

117.5.8 Numeric Ranges

Numeric leaf nodes (format must be [FORMAT_INTEGER](#), [FORMAT_LONG](#), or [FORMAT_FLOAT](#)) can be checked for a minimum and maximum value.

Minimum and maximum values are inclusive. That is, the range is [[getMin\(\)](#),[getMax\(\)](#)]. For example, if the maximum value is 5 and the minimum value is -5, then the range is [-5,5]. This means that valid values are -5, -4, -3, -2... 4, 5.

- [getMax\(\)](#) - (double) The value of the node must be less than or equal to this maximum value.

- `getMin()` - (double) The value of the node must be greater than or equal to this minimum value.

If no meta data is provided for the minimum and maximum values, the meta node must return the `Double.MIN_VALUE`, and `Double.MAX_VALUE` respectively.

117.5.9 Name Validation

The meta node provides the following name validation facilities for both leaf and interior nodes:

- `isValidName(String)` - (String) Verifies that the given name matches the rules for this meta node.
- `getValidNames()` - (String[]) An array of possible names. A valid name for this node must appear in this list.

117.5.10 User Extensions

The Meta Node provides an extension mechanism; each meta node can be associated with a number of properties. These properties are then interpreted in a proprietary way. The following methods are used for user extensions:

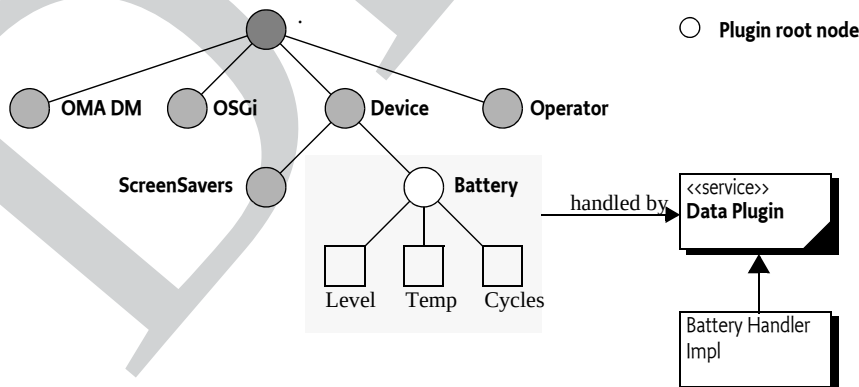
- `getExtensionPropertyKeys()` - Returns an array of key names that can be provided by this meta node.
- `getExtensionProperty(String)` - Returns the value of an extension property.

For example, a manufacturer could use a regular expression to validate the node names with the `isValidName(String)` method. In a web based user interface it is interesting to provide validity checking in the browser, however, in such a case the regular expression string is required. This string could then be provided as a user extension under the key `x-acme-regex-javascript`.

117.6 Plugins

The Plugins take the responsibility of handling DMT operations within certain sub-trees of the DMT. It is the responsibility of the Dmt Admin service to forward the operation requests to the appropriate plugin. The only exceptions are the ACL manipulation commands. ACLs must be enforced by the Dmt Admin service and never by the plugin. The model is depicted in Figure 117.9.

Figure 117.9 Device Management Tree example



Plugins are OSGi services. The Dmt Admin service must dynamically map and unmap the plugins, acting as node handler, as they are registered and unregistered. Service properties are used to specify the sub-tree that the plugin can manage as well as mount points that it provides to *Child Plugins*; plugins that manage part of the Plugin's sub-tree.

For example, a plugin related to Configuration Admin handles the sub-tree which stores configuration data. This sub-tree could start at `./OSGi/Configuration`. When the client wants to add a new configuration object to the DMT, it must issue an Add operation to the `./OSGi/Configuration` node. The Dmt Admin service then forwards this operation to the configuration plugin. The plugin maps the request to one or more method calls on the Configuration Admin service. Such a plugin can be a simple proxy to the Configuration Admin service, so it can provide a DMT view of the configuration data store.

There are two types of Dmt plugins: *data plugins* and *exec plugins*. A data plugin is responsible for handling the sub-tree retrieval, addition and deletion operations, and handling of meta data, while an exec plugin handles a node execution operation.

117.6.1

Data Sessions

Data Plugins must participate in the Dmt Admin service sessions. A Data Plugin provider must therefore register a Data Plugin service. Such a service can create a session for the Dmt Admin service when the given sub-tree is accessed by a Dmt Session. If the associated Dmt Session is later closed, the Data Session will also be closed. Three types of sessions provide different capabilities. Data Plugins do not have to implement all session types; if they choose not to implement a session type they can return null.

- *Readable Data Session* - Must always be supported. It provides the basic read-only access to the nodes and the `close()` method. The Dmt Admin service uses this session type when the lock mode is `LOCK_TYPE_SHARED` for the Dmt Session. Such a session is created with the plugin's `openReadOnlySession(String[],DmtSession)` method which returns a `ReadableDataSession` object.
- *Read Write Data Session* - Extends the Readable Data Session with capabilities to modify the DMT. This is used for Dmt Sessions that are opened with `LOCK_TYPE_EXCLUSIVE`. Such a session is created with the plugin's `openReadWriteSession(String[],DmtSession)` method, which returns a `ReadWriteDataSession` object.
- *Transactional Data Session* - Extends the Read Write Data Session with commit and rollback methods so that this session can be used with transactions. It is used when the Dmt Session is opened with lock mode `LOCK_TYPE_ATOMIC`. Such a session is created with the plugin's `openAtomicSession(String[],DmtSession)` method, which returns a `TransactionalDataSession` object.

117.6.2

URIs and Plugins

The plugin Data Sessions do not use a simple string to identify a node as the Dmt Session does. Instead the URI parameter is a `String[]`. The members of this `String[]` are the different segments. The first node after the root is the second segment and the node name is the last segment. The different segments require escaping of the solidus (`'/' \u002F`) and reverse solidus (`'\' \u005C`).

The reason to use `String[]` objects instead of the original string is to reduce the number times that the URI is parsed. The entry `String` objects, however, are still escaped. For example, the URI `./A/B/image\jpg` gives the following `String[]`:

```
{ ".", "A", "B", "image\\jpg" }
```

A plugin can assume that the path is validated and can be used directly.

117.6.3

Associating a sub-tree

Each plugin is associated with one or more DMT sub-trees. The top node of a sub-tree is called the *plugin root*. The plugin root is defined by a service registration property. This property is different for exec plugins and data plugins:

- `DATA_ROOT_URI` - (String+) A sequence of *data URI*, defining a plugin root for data plugins.

- [EXEC_ROOT_URI](#) - (String+) A sequence of *exec URI*, defining a plugin root for exec plugins.

If the Plugin modifies these service properties then the Dmt Admin service must reflect these changes as soon as possible. The reason for the different properties is to allow a single service to register both as a Data Plugin service as well as an Exec Plugin service.

Data and Exec Plugins live in independent trees and can fully overlap. However, an Exec Plugin can only execute a node when there exists a valid node at the corresponding node in the Data tree. That is, to be able to execute a node it is necessary that [isNodeUri\(String\)](#) would return true.

For example, a data plugin can register itself in its activator to handle the sub-tree `./Dev/Battery`:

```
public void start(BundleContext context) {
    Hashtable ht = new Hashtable();
    ht.put(Constants.SERVICE_PID, "com.acme.data.plugin");
    ht.put(DataPlugin.DATA_ROOT_URI, ".*./Dev/Battery");
    context.registerService(
        DataPlugin.class.getName(),
        new BatteryHandler(context);
    ht );
}
```

If this activator was executed, an access to `./Dev/Battery` must be forwarded by the Dmt Admin service to this plugin via one of the data session.

117.6.4 Synchronization with Dmt Admin Service

The Dmt Admin service can, in certain cases, detect that a node was changed without the plugin knowing about this change. For example, if the ACL is changed, the version and timestamp must be updated; these properties are maintained by the plugin. In these cases, the Dmt Admin service must open a `ReadableDataSession` and call [nodeChanged\(String\[\]\)](#) method with the changed URI.

117.6.5 Plugin Meta Data

Plugins can provide meta data; meta data from the Plugin must take precedence over the meta data of the Dmt Admin service. If a plugin provides meta information, the Dmt Admin service must verify that an operation is compatible with the meta data of the given node.

For example if the plugin reports in its meta data that the `./A` leaf node can only have the text/plain MIME type, the [createLeafNode\(String\)](#) calls must not be forwarded to the Plugin if the third argument specifies any other MIME type. If this contract between the Dmt Admin service and the plugin is violated, the plugin should throw a Dmt Exception [METADATA_MISMATCH](#).

117.6.6 Plugins and Transactions

For the Dmt Admin service to be transactional, transactions must be supported by the data plugins. This support is not mandatory in this specification, and therefore the Dmt Admin service has no transactional guarantees for atomicity, consistency, isolation or durability. The DmtAdmin interface and the DataPlugin (or more specifically the data session) interfaces, however, are designed to support Data Plugin services that are transactional. Exec plugins need not be transaction-aware because the `execute` method does not provide transactional semantics, although it can be executed in an atomic transaction.

Data Plugins do not have to support atomic sessions. When the Dmt Admin service creates a Transactional Data Session by calling [openAtomicSession\(String\[\], DmtSession\)](#) the Data Plugin is allowed to return null. In that case, the plugin does not support atomic sessions. The caller receives a Dmt Exception.

Plugins must persist any changes immediately for Read Write Data Sessions. Transactional Data Sessions must delay changes until the [commit\(\)](#) method is called, which can happen multiple times

during a session. The opening of an atomic session and the `commit()` and `rollback()` methods all establish a *transaction point*. Rollback can never go further back than the last transaction point.

- `commit()` - Commit any changes that were made to the DMT but not yet persisted. This method should not throw an Exception because other Plugins already could have persisted their data and can no longer roll it back. The commit method can be called multiple times in an open session, and if so, the commit must make persistent the changes since the last transaction point.
- `rollback()` - Undo any changes made to the sub-tree since the last transaction point.
- `close()` - Clean up and release any locks. The Dmt Admin service must call the commit methods before the close method is called. A Plugin must not perform any persistency operations in the close method.

The `commit()`, `rollback()`, and `close()` plugin data session methods must all be called in reverse order of that in which Plugins joined the session.

If a Plugin throws a fatal exception during an operation, the Dmt Session must be rolled back immediately, automatically rolling back all data plugins, as well as the plugins that threw the fatal Dmt Exception. The fatality of an Exception can be checked with the Dmt Exception `isFatal()` method.

If a plugin throws a non-fatal exception in any method accessing the DMT, the current operation fails, but the session remains open for further commands. All errors due to invalid parameters (e.g. non-existing nodes, unrecognized values), all temporary errors, etc. should fall into this category.

A rollback of the transaction can take place due to any irregularity during the session. For example:

- A necessary Plugin is unregistered or unmapped
- A fatal exception is thrown while calling a plugin
- Critical data is not available
- An attempt is made to breach the security

Any Exception thrown during the course of a `commit()` or `rollback()` method call is considered fatal, because the session can be in a half-committed state and is not safe for further use. The operation in progress should be continued with the remaining Plugins to achieve a *best-effort* solution in this limited transactional model. Once all plugins have been committed or rolled back, the Dmt Admin service must throw an exception, specifying the cause exception(s) thrown by the plugin(s), and should log an error.

117.6.7 Side Effects

Changing a node's value will have a side effect of changing the system. A plugin can also, however, cause state changes with a get operation. Sometimes the pattern to use a get operation to perform a state changing action can be quite convenient. The get operation, however, is defined to have no side effects. This definition is reflected in the session model, which allows the DMT to be shared among readers. Therefore, plugins should refrain from causing side effects for read-only operations.

117.6.8 Copying

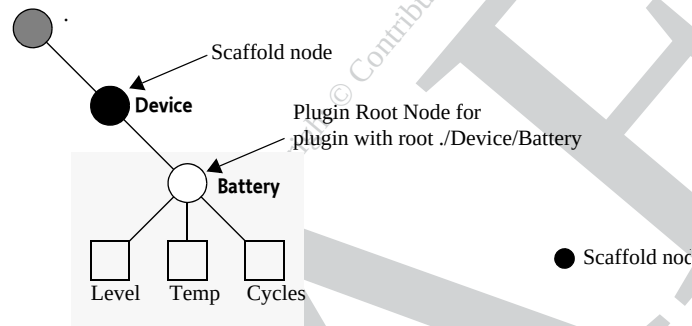
Plugins do not have to support the copy operation. They can throw a Dmt Exception with a code `FEATURE_NOT_SUPPORTED`. In this case, the Dmt Admin service must do the copying node by node. For the clients of the Dmt Admin service, it therefore appears that the copy method is always supported.

117.6.9 Scaffold Nodes

As Plugins can be mapped anywhere into the DMT it is possible that a part of the URI has no corresponding Plugin, such a plugin would not be *reachable* unless the intermediate nodes were provided. A program that would try to discover the DMT would not be able to find the registered Plugins as the intermediate nodes would not be discoverable.

These intermediate nodes that will make all plugins reachable must therefore be provided by the Dmt Admin service, they are called the *scaffold nodes*. The only purpose of the scaffold nodes is to allow every node to be discovered when the DMT is traversed from the root down. Scaffold nodes are provided both for Data Plugins as well as Exec Plugins as well as for Child Plugins that are mounted inside a Parent Plugin, see *Sharing the DMT* on page 30 . In Figure 117.10 the Device node is a scaffold node because there is no plugin associated with it. The Dmt Admin service must, however, provide the Battery node as child node of the Device node.

Figure 117.10 Scaffold Nodes



A scaffold node is always an interior node and has limited functionality, it must have a type of `DDF_SCAFFOLD` . It has no value, it is impossible to add or delete nodes to it, and the methods that are allowed for a scaffold node are specified in the following table.

Table 117.3 Supported Scaffold Node Methods

Method	Description
<code>getNodeAcl(String)</code>	Must inherit from the root node.
<code>getChildNodeNames(String)</code>	Answer the child node names such that plugin's in the sub-tree are reachable.
<code>getMetaNode(String)</code>	Provides the Meta Node defined in Table 117.4
<code>getNodeSize(String)</code>	Must throw a <code>DmtException</code> <code>COMMAND_NOT_ALLOWED</code>
<code>getNodeTitle(String)</code>	null
<code>getNodeTimestamp(String)</code>	Time first created
<code>getNodeType(String)</code>	<code>DDF_SCAFFOLD</code>
<code>isNodeUri(String)</code>	true
<code>isLeafNode(String)</code>	false
<code>getNodeVersion(String)</code>	Away returns 0
<code>copy(String,String,boolean)</code>	Not allowed for a single scaffold node as nodeUri, if the recurse parameter is false the <code>DmtException</code> <code>COMMAND_NOT_ALLOWED</code>

Any other operations must throw a `DmtException` with error code `COMMAND_NOT_ALLOWED` . The scope of a scaffold node is always `PERMANENT` . Scaffold nodes must have a Meta Node provided by the Dmt Admin service. This Meta Node must act as defined in the following table.

Table 117.4 Scaffold Meta Node Supported Methods

Method	Description
<code>can(int)</code>	<code>CMD_GET</code>
<code>getDefault()</code>	null

Method	Description
<code>getDescription()</code>	null
<code>getFormat()</code>	FORMAT_NODE
<code>getMax()</code>	Double.MAX_VALUE
<code>getMaxOccurrence()</code>	1
<code>getMimeType()</code>	DDF_SCAFFOLD
<code>getMin()</code>	Double.MIN_VALUE
<code>getRawFormatNames()</code>	null
<code>getScope()</code>	PERMANENT
<code>getValidNames()</code>	null
<code>getValidValues()</code>	null
<code>isLeaf()</code>	false
<code>isValidName(String)</code>	true
<code>isValidValue(DmtData)</code>	false
<code>isZeroOccurrenceAllowed()</code>	true

If a Plugin is registered then it is possible that a scaffold node becomes a Data Plugin root node. In that case the node and the Meta Node must subsequently be provided by the Data Plugin and can thus become different. Scaffold nodes are virtual, there are therefore no events associated with the life cycle of a scaffold node.

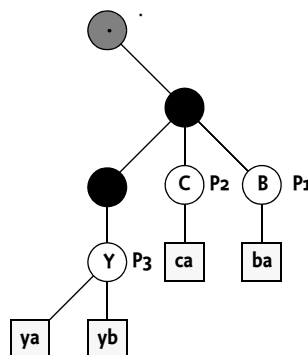
For example, there are three plugins registered:

URI	Plugin	Children
<code>./A/B</code>	P1	ba
<code>./A/C</code>	P2	ca
<code>./A/X/Y</code>	P3	ya, yb

In this example, node B, C, and Y are the plugin roots of the different plugins. As there is no plugin to manage node A and X these must be provided by the Dmt Admin service. In this example, the child names returned from each node are summarized as follows:

Node	Children	Provided by
.	{ A }	Dmt Admin (scaffold node)
A	{ X, C, B }	Dmt Admin (scaffold node)
B	{ ba }	P1
C	{ ca }	P2
X	{ Y }	Dmt Admin (scaffold node)
Y	{ ya, yb }	P3

Figure 117.11 Example Scaffold Nodes



117.7 Sharing the DMT

The Dmt Admin service provides a model to integrate the management of the myriad of components that make up an OSGi device. This integration is achieved by sharing a single namespace: the DMT. Sharing a single namespace requires rules to prevent conflicts and to resolve any conflicts when Plugins register with plugin roots that overlap. It also requires rules for the Dmt Admin service when nodes are accessed for which there is no Plugin available.

This section defines the management of overlapping plugins through the *mount points*, places where a Parent Plugin can allow a Child Plugin to take over.

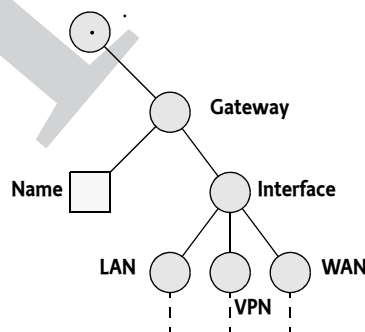
117.7.1 Mount Points

With multiple plugins the DMT is a *shared namespace*. Sharing requires rules to ensure that conflicts are avoided and when they occur, can be resolved in a consistent way. The most powerful and flexible model is to allow general overlapping. However, in practice this flexibility comes at the cost of ordering issues and therefore timing dependent results. A best practice is therefore to strictly control the points where the DMT can be extended for both Data and Exec Plugins.

A *mount point* is such a place. A Dmt Admin service at start up provides virtual mount points anywhere in the DMT and provides scaffold nodes for any intermediate nodes between the root of the DMT and the Plugin's root URL. Once a Plugin is mounted it is free to use its sub-tree (the plugin root and any ancestors) as it sees fit. However, this implies that the Plugin must implement the full sub-tree. In reality, many object models use a pattern where the different levels in the object model map to different domains.

For example, an Internet Gateway could have an object model where the general information, like the name, vendor, etc. is stored in the first level but any attached interfaces are stored in the sub-tree. However, it is highly unlikely that the code that handles the first level with the general information is actually capable of handling the details of, for example, the different network interfaces. It is actually likely that these network interfaces are dynamic. A Virtual Private Network (VPN) can add virtual network interfaces on demand. Such a could have the object model depicted in Figure 117.12.

Figure 117.12 Data Modeling



Forcing these different levels to be implemented by the same plugin violates one of the primary rules of modularity: *cohesion*. Plugins forced to handle all aspects become complex and hard to maintain. A Plugin like the one managing the Gateway node could provide its own Plugin mechanism but that would force a lot of replication and is error prone. For this reason, the Dmt Admin service allows a Plugin to provide *mount points* inside its sub-tree. A Plugin can specify that it has mount points by registering a `MOUNT_POINTS` service property (the constant is defined both in `DataPlugin` and `ExecPlugin` but have the same constant value). The type of this property must be `String+`, each string specifies a mount point. Each mount point is specified as a URI that is relative from the

plugin root. That is, when the plugin root is `./A/B` and the mount point is specified as `C` then the absolute URI of the mount point is `./A/B/C`.

A Plugin that has mount points acts as a *Parent Plugin* to a number of *Child Plugins*. In the previous example, the LAN, VPN, and WAN nodes, can then be provided by separate Child Plugins even though the Gateway/Name node is provided by the Parent Plugin. In this case, the mount points are children of the Interface node.

A mount point can be used by a number of child plugins. In the previous example, there was a Child Plugin for the LAN node, the VPN node, and the WAN node. This model has the implicit problem that it requires coordination to ensure that their names are unique. Such a coordination between independent parties is complicated and error prone. Its is therefore possible to force the Dmt Admin service to provide unique names for these nodes, see *Shared Mount Points* on page 32 .

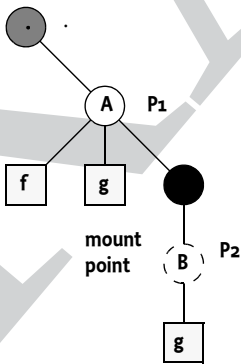
A Parent Plugin is not responsible for any scaffolding nodes to make its Child Plugins reachable. However, Dmt Admin may assume that a Plugin Root node always exists and may not provide a scaffold node on the Plugin Root. A Plugin is recommended to always provide the Plugin Root node to make its Child Plugins reachable. When a Parent Plugin provides the nodes to its mount points, the nodes should be the correct interior nodes to make its Child Plugins reachable.

For example, the following setup of plugins:

Plugin	Plugin Root	Mount Points
P1	<code>./A</code>	<code>X/B</code>
P2	<code>./A/X/B</code>	

This setup is depicted in Figure 117.13 .

Figure 117.13 Example Scaffold Nodes For Child Plugin



If the child node names are requested for the `./A` node then the plugin `P1` is asked for the child node names and must return the names `[f,g]`. However, if plugin `P2` is mapped then the Dmt Admin service must add the scaffold node name that makes this plugin reachable from that level, the returned set must therefore be `[f, g, X]`.

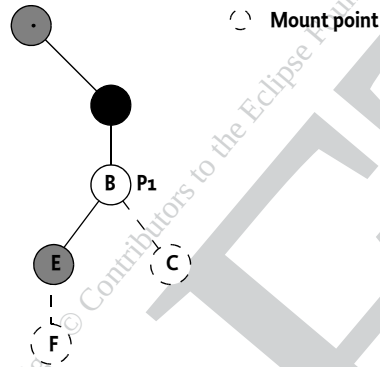
117.7.2 Parent Plugin

If a Plugin is registered with mount points then it is a *Parent Plugin*. A Parent Plugin must register with a single plugin root URI, that is the `DATA_ROOT_URI` or `EXEC_ROOT_URI` service properties must contain only one element. A Parent Plugin is allowed to be a Data and Exec Plugin at the same time. If a Parent Plugin is registered with multiple plugin root URIs then the Dmt Admin service must log an error and ignore the registration of such a Parent Plugin. A Parent Plugin can in itself also be a Child Plugin.

For example, a Plugin `P1` that has a plugin root of `./A/B` and provides a mount point at `./A/B/C` and `./A/B/E/F`. as depicted in Figure 117.14 .

Figure 117.14

Example Mount Points



Registering such a Plugin would have to register the following service properties to allow the example configuration of the DMT:

```
dataRootUri      . /A/B
mountPoints      [ C, E/F ]
```

117.7.3

Shared Mount Points

Mount points can be shared between different Plugins. In the earlier example about the Gateway the Interface node contained a sub-tree of network interfaces. It is very likely in such an example that the Plugins for the VPN interface will be provided by a different organization than the WAN and LAN network interfaces. However, all these network interface plugins must share a single parent node, the Interface node, under which they would have to mount. Sharing therefore requires a prior agreement and a naming scheme.

The naming scheme is defined by using the number sign ('#' \u0023) to specify a *shared mount point*. A plugin root that ends with the number sign, for example `./A/B/#`, indicates that it is willing to get any node under node B, leaving the naming of that node up to the Dmt Admin service. Shared mount points cannot overlap with normal mount points, the first one will become mapped and subsequent ones are in error, they are incompatible with each other. A Parent Plugin must specify a mount point explicitly as a shared mount point by using the number sign at the end of the mount point's relative URI.

A plugin is compatible with other plugins if all other plugins specify a shared mount point to the same URI. It is compatible with its Parent Plugin if the child's plugin root and the mount point are either shared or not.

The Dmt Admin service must provide a name for a plugin root that identifies a shared mount point such that every Plugin on that mount point has a unique integer name for that node level. The integer name must be ≥ 1 . The name must be convertible to an int with the static `Integer.parseInt(String)` method.

A management system in general requires permanent links to nodes. It is therefore necessary to choose the same integer every time a plugin is mapped to a shared mount point. A Child Plugin on a shared mount point must therefore get a permanent integer node name when it registers with a Persistent ID (PID). That is, it registers with the service property `service.pid`. The permanent link is then coupled to the PID and the bundle id since different bundles must be able to use the same PID. If a Plugin is registered with multiple PIDs then the first one must be used. Since permanent links can stay around for a long time implementations must strive to not reuse these integer names.

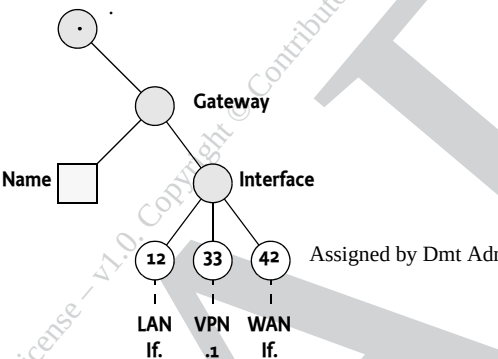
If no PID is provided then the Dmt Admin service must choose a new number that has not been used yet nor matches any persistently stored names that are currently not registered.

The Gateway example would require the following Plugin registrations:

Root URI	Mount Points	Plugin	Role
./Gateway	[Interface/#]	Gateway	Parent
./Gateway/Interface/#	[]	WAN If.	Child
./Gateway/Interface/#	[]	LAN If.	Child
./Gateway/Interface/#	[]	VPN.1	Child

This setup is depicted in Figure 117.15 .

Figure 117.15 Mount Point Sharing



The Meta Node for a Node on the level of the Mount Point can specify either an existing Plugin or it can refer to a non-existing node. If the node exists, the corresponding Plugin must provide the Meta Node. If the node does not exist, the Dmt Admin service must provide the Meta Node. Such a Meta Node must provide the responses as specified in Table 117.4 .

Table 117.5 Shared Mount Point Meta Node Supported Methods

Method	Description
can(int)	CMD_GET
getDefault()	null
getDescription()	null
getFormat()	FORMAT_NODE
getMax()	Double.MAX_VALUE
getMaxOccurrence()	Integer.MAX_VALUE
getMimeTypes()	null
getMin()	Double.MIN_VALUE
getRawFormatNames()	null
getScope()	The scope will depend on the Parent
getValidNames()	null
getValidValues()	null
isLeaf()	false
isValidName(String)	name >=1 && name < Integer.MAX_VALUE
isValidValue(DmtData)	false
isZeroOccurrenceAllowed()	true

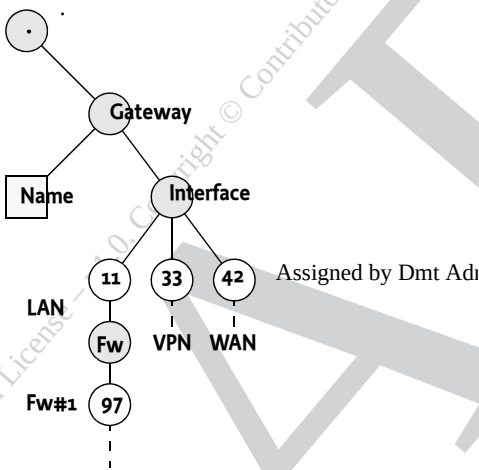
A URI can cross multiple mount points, shared and unshared. For example, if a network interface could be associated with a number of firewall rules then it is possible to register a URI on the designated network interface that refers to the Firewall rules. For the previous example, a Plugin could register a Firewall if the following registrations were done:

Root URI	Mount Points	Plugin	Parent	Name
----------	--------------	--------	--------	------

./Gateway	[Interface/#]	Gw		
./Gateway/Interface/#	[Fw/#]	WAN If. Gw	11	
./Gateway/Interface/#	[]	LAN If. Gw	33	
./Gateway/Interface/#	[]	VPN. 1 Gw	42	
./Gateway/Interface/11/Fw/#	[]	Fw. 1 WAN If.	97	

This example DMT is depicted in Figure 117.16 .

Figure 117.16 Mount Point Multiple Sharing



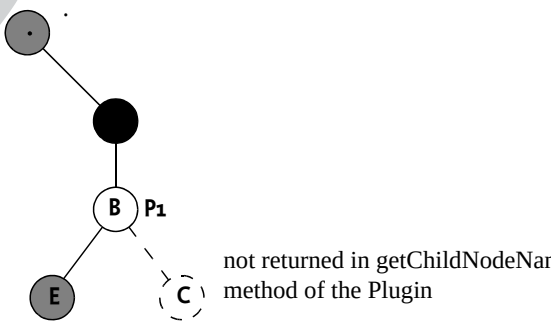
117.7.4 Mount Points are Excluded

Mount nodes are logically not included in the sub-tree of a Plugin. The Dmt Admin service must never ask any information from/about a Mount Point node to its Parent Plugin. A Parent Plugin must also not return the name of a mount point in the list of child node names, the Mount Point and its subtree is logically excluded from the sub-tree. For the Dmt Admin service an unoccupied mount point is a node that does not exist. Its name, must only be discoverable if a Plugin has actually mounted the node. The Dmt Admin service must ensure that the names of the mounted Plugins are included for that level.

In the case of shared mount points the Dmt Admin service must provide the children names of all registered Child Plugins that share that node level.

For example, a Plugin P1 registered with the plugin root of ./A/B, having two leaf nodes E, and a mount point C must not return the name C when the child node names for node B are requested. This is depicted in Figure 117.17 . The Dmt Admin service must ensure that C is returned in the list of names when a Plugin is mounted on that node.

Figure 117.17 Example Exclusion



117.7.5 Mapping a Plugin

A Plugin is not stand alone, its validity can depend on other Plugins. Invalid states make it possible that a Plugin is either *mapped* or *unmapped*. When a Plugin is mapped it is available in the DMT and when it is unmapped it is not available. Any registration, unregistration, or modification of its services properties of a Plugin can potentially alter the mapped state of any related Plugin. A plugin becomes *eligible* for mapping when it is registered.

A plugin can have multiple roots. However, the mapping is described as if there is a single plugin root. Plugins with multiple roots must be treated as multiple plugins that can each independently be mapped or unmapped depending on the context.

If no Parent Plugin is available, the Dmt Admin service must act as a virtual Parent Plugin that allows mount points anywhere in the tree where there is no mapped plugin yet.

When a Plugin becomes eligible then the following assertions must be valid for that Plugin to become mapped:

- If it has one or more mount points then
 - It must have at most one Data and/or Exec Root URI.
 - None of its mount points must overlap.
 - Any already mapped Child Plugins must be compatible with its mount points.
- If no mount points are specified then there must be no Child Plugins already registered.
- The plugin root must be compatible with the corresponding parent's mount point. When a Parent Plugin is available, the plugin root must match exactly to the absolute URI of the parent's mount point.
- The plugin root must be compatible with any other plugins on that mount point.

If either of these assertions fail then the Dmt Admin service must log an error and ignore the registered Plugin, it must not become mapped. If, through the unregistration or modification of the service properties, the assertions can become valid then the Dmt Admin service must retry mapping the Plugin so that it can become available in the DMT. Any mappings and unmappings that affect nodes that are in the sub-tree of an active session must abort that session with a [CONCURRENT_ACCESS](#) exception.

When there are errors in the configuration then the ordering will define which plugins are mapped or not. Since this is an error situation no ordering is defined between conflicting plugins.

For example, a number of Plugins are registered in the given order:

Plugin Root	Children	Mount Points	Plugin
./A/B	E	C	P1
./A/B/C			P2
./A/B/D			P3

The first Plugin P1 will be registered immediately without problems. It has only a single plugin root as required by the fact that it is a Parent Plugin (it has a mount point). There are no Child Plugins yet so it is impossible to have a violation of the mount points. As there is no Parent Plugin registered, the Dmt Admin service will map plugin P1 and automatically provide the scaffold node A.

When Plugin P2 is registered its plugin root maps to a mount point in Plugin P1. As P2 is not a Parent Plugin it is only necessary that it has no Child Plugins. As it has no Child Plugins, the mapping will succeed.

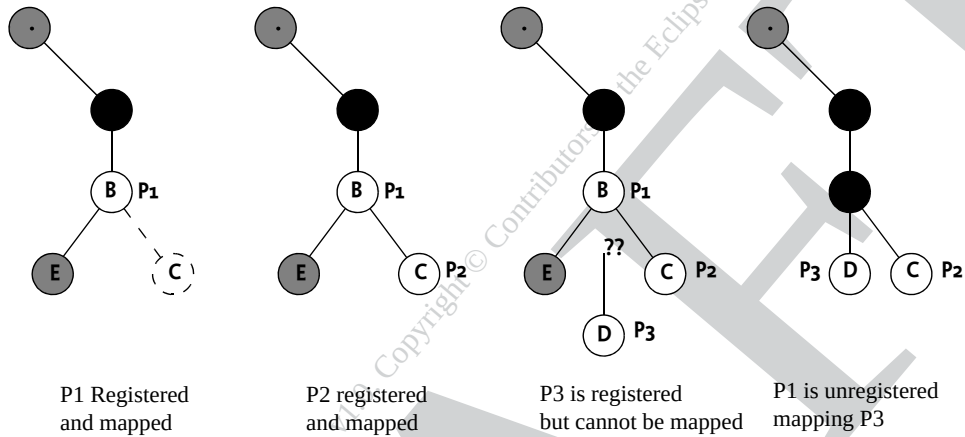
Plugin P3 cannot be mapped because the Parent Plugin is P1 but P1 does not provide a mount point for P3's plugin root ./A/B/D.

If, at a later time P1 is unregistered then the Dmt Admin service must map plugin P3 and leave plugin P2 mapped. This sequence of action is depicted in Figure 117.18.

If plugin P1 becomes registered again at a later time it can then in its turn not be mapped as there would be a child plugin (P3) that would not map to its mount point.

Figure 117.18

Plugin Activation



117.7.6

Mount Plugins

In *Mapping a Plugin* on page 35 it is specified that a Plugin can be *mapped* or not. The mapped state of a Plugin can change depending on other plugins that are registered and unregistered. Plugins require in certain cases to know:

- What is the name of their root node if they mount on a shared mount point.
- What is the mapping state of the Plugin.

To find out these details a Plugin can implement the [MountPlugin](#) interface; this is a mixin interface, it is not necessary to register it as [MountPlugin](#) service. The Dmt Admin service must do an instance of operation on Data Plugin services and Exec Plugin services to detect if they are interested in the mount point information.

The Mount Point interface is used by the Dmt Admin service to notify the Plugin when it becomes mapped and when it becomes unmapped. The Plugin will be informed about each plugin root separately.

The Mount Plugin specifies the following methods that are called synchronously:

- [mountPointAdded\(MountPoint\)](#) - The Dmt Admin service must call this method after it has mapped a plugin root. From this point on the given mount point provides the actual path until the [mountPointRemoved\(MountPoint\)](#) is called with an equal object. The given Mount Point can be used to post events.
- [mountPointRemoved\(MountPoint\)](#) - The Dmt Admin service must call this method after it has unmapped the given mount point. This method must always be called when a plugin root is unmapped, even if this is caused by the unregistration of the plugin.

As the mapping and unmapping of a plugin root can happen any moment in time a Plugin that implements the Mount Plugin interface must be prepared to handle these events at any time on any thread.

The [MountPoint](#) interface has two separate responsibilities:

- *Path* - The path that this Mount Point is associated with. This path is a plugin root of the plugin. This path is identical to the Plugin's root except when it is mounted on a shared mount point; in that case the URI ends in the name chosen by the Dmt Admin service. The [getMountPath\(\)](#) method provides the path.

- *Events* - Post events about the given sub-tree that signal internal changes that occur outside a Dmt Session. The Dmt Admin service must treat these events as they were originated from modifications to the DMT. That is, they need to be forwarded to the Event Admin as well as the Dmt Listeners. For this purpose there are the `postEvent(String,String[],Dictionary)` and `postEvent(String,String[],String[],Dictionary)` methods.

For example, a Data Plugin monitoring one of the batteries registers with the following service properties:

```
dataRootURIs          ". /Device/Battery/#"
```

The Device node is from a Parent Plugin that provided the shared mount point. The Battery Plugin implements the `MountPlugin` interface so it gets called back when it is mapped. This will cause the Dmt Admin service to call the `mountPointAdded(MountPoint)` method on the plugin. In this case, it will get just one mount point, the mount point for its plugin root. If the Dmt Admin service would have assigned the Battery Plugin number 101 then the `getMountPath()` would return:

```
[ ".", "Device", "Battery", "101" ]
```

As the Plugin monitors the charge state of the battery it can detect a significant change. In that case it must send an event to notify any observers. The following code shows how this could be done:

```
@Component( properties="dataRootURIs=. /Device/Battery/#",
              provide=DataPlugin.class)
public class Battery implements DataPlugin, MountPlugin {
    Timer        timer;
    volatile float charge;
    TimerTask     task;

    public void mountPointsAdded(final MountPoint[] mountPoints){
        task = new TimerTask() {
            public void run() {
                float next = measure();
                if (Math.abs(charge - next) > 0.2) {
                    charge = next;
                    mountPoints[0].postEvent(DmtConstants.EVENT_TOPIC_REPLACED,
                                              new String[] { "Charge" }, null);
                }
            }
        };
        timer.schedule(task, 1000);
    }

    public void mountPointsRemoved(MountPoint[] mountPoints){
        task.cancel();
        task = null;
    }
    ... // Other methods
}
```

117.8 Access Control Lists

Each node in the DMT can be protected with an *access control list*, or *ACL*. An ACL is a list of associations between *Principal* and *Operation*:

- *Principal* - The identity that is authorized to use the associated operations. Special principal is the wildcard ('*' \u002A); the operations granted to this principal are called the *global permissions*. The global permissions are available to all principals.
- *Operation* - A list of operations: ADD, DELETE, GET, REPLACE, EXECUTE.

DMT ACLs are defined as strings with an internal syntax in [1] *OMA DM-TND v1.2 draft*. Instances of the ACL class can be created by supplying a valid OMA DM ACL string as its parameter. The syntax of the ACL is presented here in shortened form for convenience:

```
acl      ::= ( acl-entry ( '&' acl-entry)* )
acl-entry ::= command '=' ( principals | '*' )
principals ::= principal ( '+' principal )*
principal ::= -[' ' '&' '*' '+' '\t' '\n' '\r']+
```

The principal name should only use printable characters according to the OMA DM specification.

```
command  ::= 'Add' | 'Delete' | 'Exec' | 'Get' | 'Replace'
```

White space between tokens is not allowed.

Examples:

```
Add=*&Replace=*&Get=*
```

```
Add=www.sonera.fi-8765&Delete=www.sonera.fi-8765& «
Replace=www.sonera.fi-8765+321_ibm.com&Get=*
```

The [Acl\(String\)](#) constructor can be used to construct an ACL from an ACL string. The [toString\(\)](#) method returns a String object that is formatted in the specified form, also called the canonical form. In this form, the principals must be sorted alphabetically and the order of the commands is:

```
ADD,    DELETE,    EXEC,    GET,    REPLACE
```

The Acl class is immutable, meaning that a Acl object can be treated like a string, and that the object cannot be changed after it has been created.

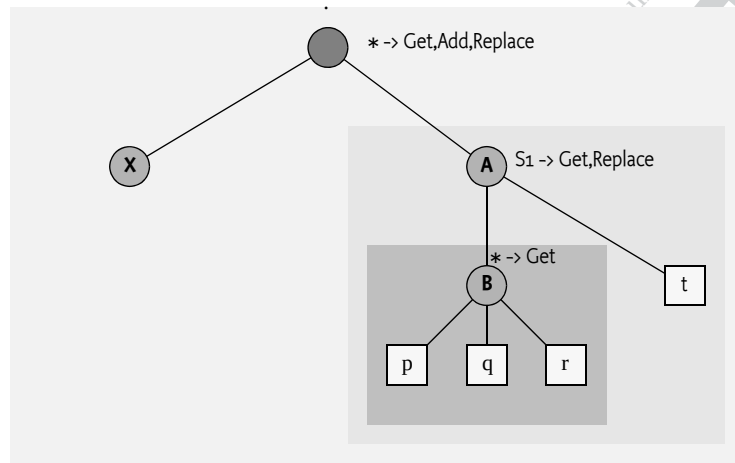
ACLs must only be verified by the Dmt Admin service when the session has an associated principal.

ACLs are properties of nodes. If an ACL is *not set* (i.e. contains no commands nor principals), the *effective* ACL of that node must be the ACL of its first ancestor that has a non-empty ACL. This effective ACL can be acquired with the [getEffectiveNodeAcl\(String\)](#) method. The root node of DMT must always have an ACL associated with it. If this ACL is not explicitly set, it should be set to Add=*&Get=*&Replace=*

This effect is shown in Figure 117.19. This diagram shows the ACLs set on a node and their effect (which is shown by the shaded rectangles). Any principal can get the value of p, q and r, but they cannot replace, add or delete the node. Node t can only be read and replaced by principal S1.

Node X is fully accessible to any authenticated principal because the root node specifies that all principals have Get, Add and Replace access (*->G,A,R).

Figure 117.19 ACL inheritance



The definition and example demonstrate the access rights to the properties of a node, which includes the value.

Changing the ACL property itself has different rules. If a principal has **Replace** access to an interior node, the principal is permitted to change its own ACL property *and* the ACL properties of all its child nodes. Replace access on a leaf node does not allow changing the ACL property itself.

In the previous example, only principal S1 is authorized to change the ACL of node B because it has Replace permission on node B's parent node A.

Figure 117.20 ACLs for the ACL property

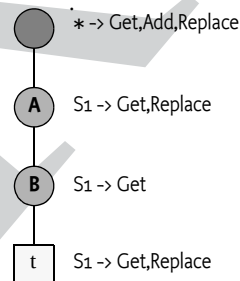


Figure 117.20 demonstrates the effect of this rule with an example. Server S1 can change the ACL properties of all interior nodes. A more detailed analysis:

- *Root* - The root allows all authenticated principals to access it. The root is an interior node so the Replace permission permits the change of the ACL property.
- *Node A* - Server S1 has Replace permission and node A is an interior node so principal S1 can modify the ACL.
- *Node B* - Server S1 has no Replace permission for node B, but the parent node A of node B grants principal S1 Replace permission, and S1 is therefore permitted to change the ACL.
- *Node t* - Server S1 must not be allowed to change the ACL of node t, despite the fact that it has Replace permission on node t. For leaf nodes, permission to change an ACL is defined by the Replace permission in the parent node's ACL. This parent, node B, has no such permission set and thus, access is denied.

The following methods provide access to the ACL property of the node.

- `getNodeAcl(String)` - Return the ACL for the given node, this method must not take any ACL inheritance into account. The ACL may be null if no ACL is set.
- `getEffectiveNodeAcl(String)` - Return the effective ACL for the given node, taking any inheritance into account.
- `setNodeAcl(String,Acl)` - Set the node's ACL. The ACL can be null, in which case the effective permission must be derived from an ancestor. The Dmt Admin service must call `nodeChanged(String[])` on the data session with the given plugin to let the plugin update any timestamps and versions.

The `Acl` class maintains the permissions for a given principal in a bit mask. The following permission masks are defined as constants in the `Acl` class:

- `ADD`
- `DELETE`
- `EXEC`
- `GET`
- `REPLACE`

The class features methods for getting permissions for given principals. A number of methods allow an existing ACL to be modified while creating a new ACL.

- `addPermission(String,int)` - Return a new `Acl` object where the given permissions have been added to permissions of the given principal.
- `deletePermission(String,int)` - Return a new `Acl` object where the given permissions have been removed from the permissions of the given principal.
- `setPermission(String,int)` - Return a new `Acl` object where the permissions of the given principal are overwritten with the given permissions.

Information from a given ACL can be retrieved with:

- `getPermissions(String)` - (int) Return the combined permission mask for this principal.
- `getPrincipals()` - (String[]) Return a list of principals (String objects) that have been granted permissions for this node.

Additionally, the `isPermitted(String,int)` method verifies if the given ACL authorizes the given permission mask. The method returns true if all commands in the mask are allowed by the ACL.

For example:

```
Acl acl = new Acl("Get=S1&Replace=S1");

if ( acl.isPermitted("S1", Acl.GET+Acl.REPLACE ))
    ... // will execute

if ( acl.isPermitted(
    "S1", Acl.GET+Acl.REPLACE+Acl.ADD ))
    ... // will NOT execute
```

117.8.1

Global Permissions

Global permissions are indicated with the '*' and the given permissions apply to all principals. Processing the global permissions, however, has a number of non-obvious side effects:

- Global permissions can be retrieved and manipulated using the special '*' principal: all methods of the `Acl` class that have a principal parameter also accept this principal.

- Global permissions are automatically granted to all specific principals. That is, the result of the `getPermissions` or `isPermitted` methods will be based on the OR of the global permissions and the principal-specific permissions.
- If a global permission is revoked, it is revoked from all specific principals, even if the specific principals already had that permission before it was made global.
- None of the global permissions can be revoked from a specific principal. The OMA DM ACL format does not handle exceptions, which must be enforced by the `deletePermission` and `setPermission` methods.

117.8.2

Ghost ACLs

The ACLs are fully maintained by the Dmt Admin service and enforced when the session has an associated principal. A plugin must be completely unaware of any ACLs. The Dmt Admin service must synchronize the ACLs with any change in the DMT that is made through its service interface. For example, if a node is deleted through the Dmt Admin service, it must also delete an associated ACL.

The DMT nodes, however, are mapped to plugins, and plugins can delete nodes outside the scope of the Dmt Admin service.

As an example, consider a configuration record which is mapped to a DMT node that has an ACL. If the configuration record is deleted using the Configuration Admin service, the data disappears, but the ACL entry in the Dmt Admin service remains. If the configuration dictionary is recreated with the same PID, it will get the old ACL, which is likely not the intended behavior.

This specification does not specify a solution to solve this problem. Suggestions to solve this problem are:

- Use a proprietary callback mechanism from the underlying representation to notify the Dmt Admin service to clean up the related ACLs.
- Implement the services on top of the DMT. For example, the Configuration Admin service could use a plugin that provides general data storage service.

117.9

Notifications

In certain cases it is necessary for some code on the device to alert a remote management server or to initiate a session; this process is called sending a notification or an *alert*. Some examples:

- A Plugin that must send the result of an asynchronous EXEC operation.
- Sending a request to the server to start a management session.
- Notifying the server of completion of a software update operation.

Notifications can be sent to a management server using the `sendNotification(String,int,String,AlertItem[])` method on the Notification Service. This method is on the Notification Service and not on the session, because the session can already be closed when the need for an alert arises. If an alert is related to a session, the session can provide the required principal, even after it is closed.

The remote server is alerted with one or more `AlertItem` objects. The `AlertItem` class describes details of the alert. An alert code is an alert type identifier, usually requiring specifically formatted `AlertItem` objects.

The data syntax and semantics vary widely between various alerts, and so does the optionality of particular parameters of an alert item. If an item, such as source or type, is not defined, the corresponding getter method must return null.

The `AlertItem` class contains the following items. The value of these items must be defined in an alert definition:

- `source` - (String) The URI of a node that is related to this request. This parameter can be null.
- `type` - (String) The type of the item. For example, `x-oma-application:syncml.samplealert` in the Generic Alert example.
- `mark` - (String) Mark field of an alert. Contents depend on the alert type.
- `data` - (DmtData) The payload of the alert with its type.

An `AlertItem` object can be constructed with two different constructors:

- `AlertItem(String,String,String,DmtData)` - This method takes all the previously defined fields.
- `AlertItem(String[],String,String,DmtData)` - Same as previous but with a convenience parameter for a segmented URI.

The Notification Service provides the following method to send `AlertItem` objects to the management server:

- `sendNotification(String,int,String,AlertItem[])` - Send the alert to the server that is associated with the session. The first argument is the name of the principal (identifying the remote management system) or null for implementation defined routing. The `int` argument is the *alert type*. The alert types are defined by *managed object types*. The third argument (String) can be used for the correlation id of a previous execute operation that triggered the alert. The `AlertItem` objects contain the data of the alert. The method will run asynchronously from the caller. The Notification Service must provide a reliable delivery method for these alerts. Alerts must therefore not be re-transmitted.

When this method is called with null correlator, null or empty `AlertItem` array, and a 0 code as values, it should send a protocol specific notification that must initiate a new management session.

Implementers should base the routing on the session or server information provided as a parameter in the `sendNotification(String,int,String,AlertItem[])` method. Routing might even be possible without any routing information if there is a well known remote server for the device.

If the request cannot be routed, the Alert Sender service must immediately throw a Dmt Exception with a code of `ALERT_NOT_ROUTED`. The caller should not attempt to retry the sending of the notification. It is the responsibility of the Notification Service to deliver the notification to the remote management system.

117.9.1

Routing Alerts

The Notification Service allows external parties to route alerts to their destination. This mechanism enables Protocol Adapters to receive any alerts for systems with which they can communicate.

Such a Protocol Adapter should register a Remote Alert Sender service. It should provide the following service property:

- *principals* - (String+) The array of principals to which this Remote Alert Sender service can route alerts. If this property is not registered, the Remote Alert Sender service will be treated as the default sender. The default alert sender is only used when a more specific alert sender cannot be found.

If multiple Remote Alert Sender services register for the same principals highest ranking service is taken as defined in the OSGi Core.

117.10 Exceptions

Most of the methods of this Dmt Admin service API throw Dmt Exceptions whenever an operation fails. The `DmtException` class contains numeric error codes which describe the cause of the error. Some of the error codes correspond to the codes described by the OMA DM spec, while some are introduced by the OSGi Working Group. The documentation of each method describes what codes could potentially be used for that method.

The fatality of the exception decides if a thrown Exception rolls back an atomic session or not. If the `isFatal()` method returns true, the Exception is fatal and the session must be rolled back.

All possible error codes are constants in the `DmtException` class.

117.11 Events

There are the following mechanisms to work with events when using the Dmt Admin service.

- *Event Admin service* - Standard asynchronous notifications
- *Dmt Event Listener service* - A white board model for listener. A registered `DmtEventListener` service can use service properties to filter the received events

In both cases events are delivered asynchronously and ordered per listener unless otherwise specified. Events to the DMT can occur because of modifications made in a session or they can occur because a Plugin changes its internal state and notifies the Dmt Admin service through the `MountPoint` interface.

Changes made through a session always start with a `SESSION_OPENED` event directly after the session is opened. This event must contain the properties defined in *Life Cycle Event Properties* on page 46 .

If events originate from an atomic session then these events must be queued until the sessions is successfully committed, which can happen multiple times over the life time of a session. If the session is rolled back or runs into an error then none of the queued events must be sent.

When a session is closed, which can happen automatically when the session fails, then the `SESSION_CLOSED` event must be sent. This event must happen after any queued events. This closed event must contain the properties defined in *Life Cycle Event Properties* on page 46 .

An event must only be sent when that type of event actually occurred.

117.11.1 Event Admin

Event Admin, when present, must be used to deliver the Dmt Admin events asynchronously. The event types are specified in Table 117.7 on page 44 , the Topic column defines the Event Admin topic. The Table 117.10 on page 46 and Table 117.9 on page 46 define the Life Cycle and Session properties that must be passed as the event properties of Event Admin.

117.11.2 Dmt Event Listeners

To receive the Dmt Admin events it is necessary to register a Dmt Event Listener service. It is possible to filter the events by registering a combination of the service properties defined in the following table.

Table 117.6 *Service Properties for the Dmt Event Listener*

Service Property	Data Type	Default	Description
<code>FILTER_EVENT</code>	Integer	All Events	A bitmap of <code>DmtEvent</code> types: <code>SESSION_OPENED</code> , <code>ADDED</code> , <code>COPIED</code> , <code>DELETED</code> , <code>REPLACED</code> , and <code>SESSION_CLOSED</code> . A Dmt Event's type must occur in the bitmap to be delivered.
<code>FILTER_PRINCIPAL</code>	String+	Any node	Only deliver Dmt Events for which at least one of the given principals has the right to Get that node.
<code>FILTER_SUBTREE</code>	String+	Any node	This property defines a number of sub-trees by specifying the URI of the top nodes of these sub-trees. Only events that occur in one of the sub-trees must be delivered.

A Dmt Event must only be delivered to a Dmt Event Listener if the Bundle that registers the Dmt Event Listener service has the `GET` Dmt Permission for each of the nodes used in the nodes and newNodes properties as tested with the Bundle hasPermission method.

The Dmt Admin service must track Dmt Event Listener services and deliver matching events as long as a Dmt Event Listener service is registered. Any changes in the service properties must be expediently handled.

A Dmt Event Listener must implement the `changeOccurred(DmtEvent)` method. This method is called asynchronously from the actual event occurrence but each listener must receive the events in order.

Events are delivered with a `DmtEvent` object. This object provides access to the properties of the event. Some properties are available as methods others must be retrieved through the `getProperty(String)` method. The methods that provide property information are listed in the property tables, see Table 117.10 on page 46 .

117.11.3

Atomic Sessions and Events

The intent of the events is that a listener can follow the modifications to the DMT from the events alone. However, from an efficiency point of view certain events should be coalesced to minimize the number of events that a listener need to handle. For this reason, the Dmt Admin service must coalesce events if possible.

Two consecutive events can be coalesced when they are of the same type. In that case the nodes and, if present, the newNodes of the second event can be concatenated with the first event and the time-stamp must be derived from the first event. It is not necessary to remove duplicates from the nodes and newNodes. This guarantees that the order of the nodes is in the order of the events.

117.11.4

Event Types

This section describes the events that can be generated by the Dmt Admin service. Table 117.7 enumerates all the events and provides the name of the topic of Event Admin and the Dmt Event type for the listener model.

There are two kinds of events:

- *Life Cycle Events* - The events for session open and closed are the session events.
- *Session Events* - `ADDED`, `DELETED`, `REPLACED`, `RENAMED`, and `COPIED`.

Session and life cycle events have different properties.

Table 117.7 *Event Types*

Event	Topic	Dmt Event Type	Description
<code>SESSION OPENED</code>	<code>org/osgi/service/dmt/DmtEvent/SESSION_OPENED</code>	<code>SESSION_OPENED</code>	A new session was opened. The event must the properties defined in Table 117.9 on page 46 .

Event	Topic	Dmt Event Type	Description
ADDED	org.osgi/service/dmt/DmtEvent/ ADDED	ADDED	One or more nodes were added.
DELETED	org.osgi/service/dmt/DmtEvent/ DELETED	DELETED	One or more existing nodes were deleted.
REPLACED	org.osgi/service/dmt/DmtEvent/RE- PLACED	REPLACED	Values of nodes were replaced.
RENAMED	org.osgi/service/dmt/DmtEvent/RE- NAMED	RENAMED	Existing nodes were renamed.
COPIED	org.osgi/service/dmt/DmtEvent/ COPIED	COPIED	Existing nodes were copied. A copy operation does not trigger an ADDED event (in addition to the COPIED event), even though new node(s) are created. For efficiency reasons, recursive copy and delete operations must only generate a single COPIED and DELETED event for the root of the affected sub-tree.
SESSION CLOSED	org.osgi/service/dmt/DmtEvent/ SESSION_CLOSED	SESSION_CLOSED	A session was closed either because it was closed explicitly or because there was an error detected. The event must have the properties defined in Table 117.9 on page 46 .

117.11.5 General Event Properties

The following properties must be available as the event properties in Event Admin service and the properties in the Dmt Event for Dmt Event Listener services.

Table 117.8

General Event

Property Name	Type	Dmt Event	Description
event.topics	String		Event topic, required by Event Admin but must also be present in the Dmt Events.
session.id	Integer	<code>getSessionId()</code>	A unique identifier for the session that triggered the event. This property has the same value as <code>getSessionId()</code> of the associated DMT session. If this event is generated outside a session then the session id must be -1, otherwise it must be ≥ 1 .
timestamp	Long		The time the event was started as defined by <code>System.currentTimeMillis()</code>
bundle	Bundle		The initiating Bundle, this is the bundle that caused the event. This is either the Bundle that opened the associated session or the Plugin's bundle when there is no session (i.e. the session id is -1).
bundle.signer	String+		The signer of the initiating Bundle
bundle.symbolicName	String		The Bundle Symbolic name of the initiating Bundle
bundle.version	Version		The Bundle version of the initiating Bundle.
bundle.id	Long		The Bundle Id of the initiating Bundle.

117.11.6 Session Event Properties

All session events must have the properties defined in the following table.

Table 117.9 Event Properties For Session Events

Property Name	Type	Dmt Session	Description
session.rooturi	String	getRootUri()	The root URI of the session that triggered the event.
session.principal	String	getPrincipal()	The principal of the session, or absent if no principal is associated with this session. In the latter case the method returns null.
session.locktype	Integer	getLockType()	The lock type of the session. The number is mapped as follows: LOCK_TYPE_SHARED - 0 LOCK_TYPE_EXCLUSIVE - 1 LOCK_TYPE_ATOMIC - 2
session.timeout	Boolean		If the session timed out then this property must be set to true. If it did not time out this property must be false.
exception	Throwable		The name of the actual exception class if the session had a fatal exception.
exception.message	String		Must describe the exception if the session had a fatal exception.
exception.class	String		The name of the actual exception class if the session had a fatal exception.

117.11.7 Life Cycle Event Properties

All Life Cycle events must have the properties defined in the following table.

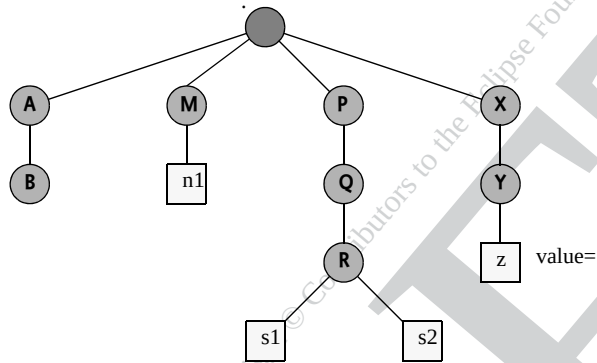
Table 117.10 Event Properties for Life Cycle Events

Property Name	Type	Dmt Event	Description
nodes	String[]	getNodes()	The absolute URIs of each affected node. This is the <code>nodeUri</code> parameter of the Dmt API methods. The order of the URIs in the array corresponds to the chronological order of the operations. In case of a recursive delete or copy, only the session root URI is present in the array.
newnodes	String[]	getNewNodes()	The absolute URIs of new renamed or copied nodes. Only the RENAMED and COPIED events have this property. The <code>newnodes</code> array runs parallel to the <code>nodes</code> array. In case of a rename, <code>newnodes[i]</code> must contain the new name of <code>nodes[i]</code> , and in case of a copy, <code>newnodes[i]</code> is the URI to which <code>nodes[i]</code> was copied.

117.11.8 Example Event Delivery

The example in this section shows the change of a non-trivial tree and the events that these changes will cause.

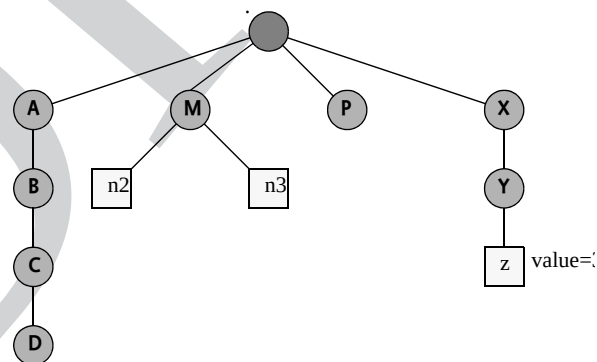
Figure 117.21 Example DMT before



For example, in a given session, when the DMT in Figure 117.21 is modified with the following operations:

- Open atomic session 42 on the root URI
- Add node ./A/B/C
- Add node ./A/B/C/D
- Rename ./M/n1 to ./M/n2
- Copy ./M/n2 to ./M/n3
- Delete node ./P/Q
- Add node ./P/Q
- Delete node ./P/Q
- Replace ./X/Y/z with 3
- Commit
- Close

Figure 117.22 Example DMT after



When the Dmt Session is opened, the following event is published:

```

SESSION_OPENED {
    session.id = 42
    session.rooturi=.
    session.principal=null
    session.locktype=2
  }
  
```

```

        timestamp=1313411544752
        bundle =<Bundle>
        bundle.signer=[]
        bundle.symbolicname"com.acme.bundle"
        bundle.version=1.2.4711
        bundle.id=442
        ...
    }

```

When the Dmt Session is closed (assuming it is atomic), the following events are published:

```

ADDED {
    nodes = [ ./A/B/C, ./A/B/C/D ] # note the coalescing
    session.id = 42
    ...
}
RENAMED {
    nodes = [ ./M/n1 ]
    newnodes = [ ./M/n2 ]
    session.id = 42
    ...
}
COPIED {
    nodes = [ ./M/n2 ]
    newnodes = [ ./M/n3 ]
    session.id = 42
    ...
}
DELETED {
    nodes = [ ./P/Q ]
    session.id = 42
    ...
}
ADDED {
    nodes = [ ./P/Q ]
    session.id = 42
    ...
}
DELETED {
    nodes = [ ./P/Q ]
    session.id = 42
    ...
}
REPLACED {
    nodes = [ ./X/Y/z ]
    session.id = 42
    ...
}
SESSION_CLOSED {
    session.id = 42
    session.rooturi=.
    session.principal=null
    session.locktype=2
    ...
}

```

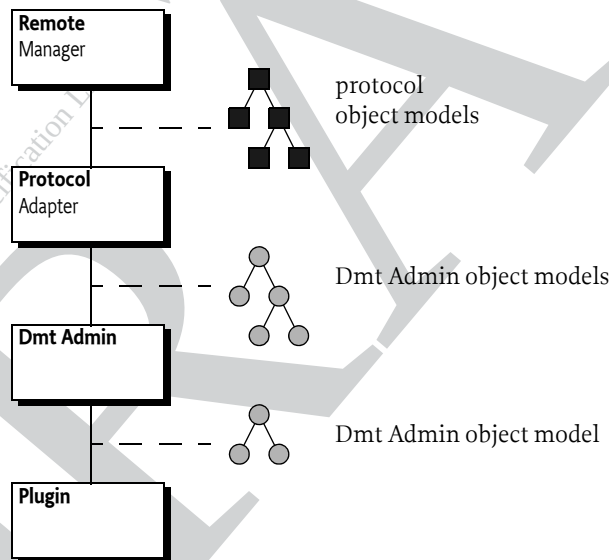
117.12 OSGi Object Modeling

117.12.1 Object Models

Management protocols define only half the picture; the object models associated with a particular protocol are the other half. Object models are always closely associated with a remote management protocol since they are based on the data types and actions that are defined in the protocol. Even small differences between the data types of a protocol and its differences make accurate mapping between protocols virtually impossible. It is therefore necessary to make the distinction between *native* and *foreign* protocols for an object model.

A native protocol for an object model originates from the same specification organization. For example, OMA DM consists of a protocol based on SyncML and a number of object models that define the structure and behavior of the nodes of the DMT. The FOMA specification defines an OMA DM native object model, it defines how firmware management is done. This is depicted in Figure 117.23.

Figure 117.23 Device Management Architecture



If an object implements a standardized data model it must be visible through its *native* Protocol Adapter, that is the Protocol Adapter that belongs to the object model's standard. For example, an ExecutionUnit node defined in UPnP Device Management could be implemented as a bundle, exposed through a Data Plugin for the Dmt Admin service, and then translated by its native UPnP Protocol Adapter.

If an object is present in the Dmt Admin service it is also available to *foreign* Protocol Adapters. A foreign Protocol Adapter is any Protocol Adapter except its native Protocol Adapter. For example, the Broadband Forum's ExecutionUnit could be browsed on the foreign OMA DM protocol.

In a foreign Protocol Adapter the object model should be *browsable* but it would not map to one of its native object models. Browsable means that the information is available to the Protocol Adapter's remote manager but not recognized as a standard model for the manager. Browse can include, potentially limited, manipulation.

In a native Protocol Adapter it is paramount that the mapping from the DMT to the native object is fully correct. It is the purpose of this part of the Dmt Admin service specification to allow the native Protocol Adapter to map the intentions of the Plugin without requiring knowledge of the specific

native object model. That is, a TR-069 Plugin implementing a WAN interface must be available over the TR-069 protocol without the Protocol Adapter having explicit knowledge about the WAN interfaces object models from Broadband Forum.

Therefore, the following use cases are recognized:

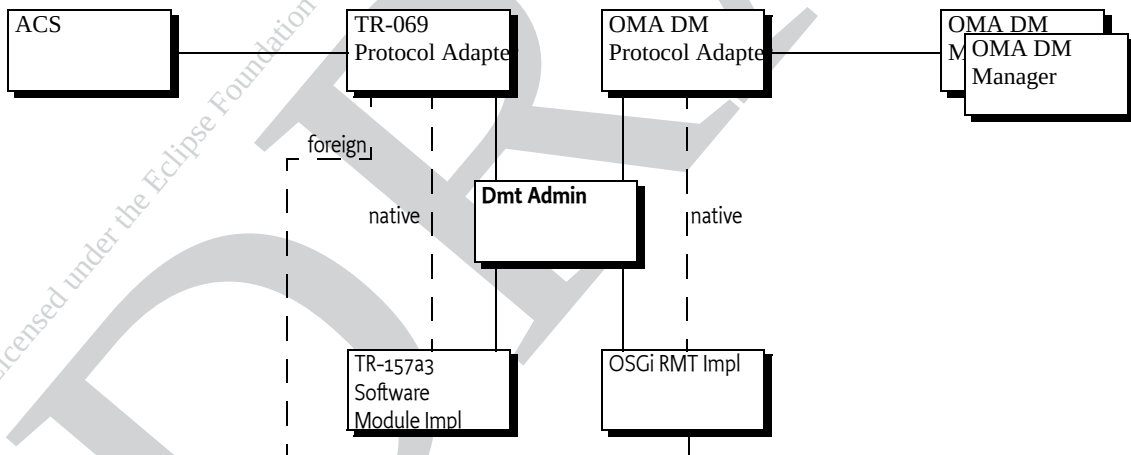
- *Foreign Mapping* - Foreign mapping can be best-effort as there is no object model to follow. Each Protocol Adapter must define how the Dmt Admin model is mapped for this browse mode.
- *Native Mapping* - Native mapping must be 100% correct. As it is impossible to automatically map DMTs to arbitrary protocols this specification provides the concept of a mapping model that allows a Plugin to instruct its native Protocol Adapter using Meta Nodes.

117.12.2 Protocol Mapping

The OSGi Working Group specifies an Execution Environment that can be used as a basis for residential gateways, mobiles, or other devices. This raises the issue how to expose the manageability of an OSGi device and the *objects*, the units of manageability, that are implemented through Plugins. Ideally, an object should be able to expose its management interface once and then Protocol Adapters convert the management interface to specific device management stacks. For example, an object can be exposed through the Dmt Admin service where then a TR-069 Protocol Adapter maps the DMT to the TR-069 Remote Procedure Calls (RPC).

Figure 117.24 shows an example of a TR-069 Protocol Adapter and an OMA DM Protocol Adapter. The TR-069 Protocol Adapter should be able to map native TR-069 objects in the DMT (the Software Modules Impl in the figure) to Broadband Forum's object models. It should also be able to browse the foreign DMT and other objects that are not defined in Broadband forum but can be accessed with the TR-069 RPCs.

Figure 117.24 Implementing & Browsing



A *Protocol Mapping* is a document that describes the default mapping and the native mechanism for exact mapping.

The following sections specify how Plugins must implement an object model that is exposed through the Dmt Admin service. This model is limited from the full Dmt Admin service capabilities so that for each protocol it is possible to specify a default mapping for browsing as well as a mechanism to ensure that special conversion requirements can be communicated from a Plugin to its native Protocol Adapter.

117.12.3 Hierarchy

The Dmt Admin model provides an hierarchy of *nodes*. Each node has a *type* that is reflected by its Meta Node. A node is addressed with a URI. The flexibility of the Dmt Admin service allows a large

number of constructs, for example, the name of the node can be used as a *value*, a feature that some management standards support. To simplify mapping to foreign Protocol Adapters, some of the fundamental constructs have been defined in the following sections.

117.12.4 General Restriction Guidelines

The Dmt Admin service provides a very rich tool to model complex object structures. Many choices can be made that would make it very hard to browse DMTs on non-OMA DM protocols or make the DMT hard to use through the Dmt Admin service. As Plugins can always signal special case handling to their native Protocol Adapter, any object model design should strive to be easy to use for the developers and managers. Therefore, this section provides a number of guidelines for the design of such object models that will improve the browsing experience for many Protocol Adapters.

- *Reading of a node must not change the state of a device* - Management systems must be able to browse a tree without causing any side effects. If reading modified the DMT, a management system would have no way to warn the user that the system is modified. There are a number of technical reasons as well (race conditions, security holes, and eventing) but the most important reason is the browsability of the device.
- *No use of recursive structures* - The Dmt Admin service provides a very rich tree model that has no problem with recursion. However, this does not have to be true for other models. To increase the changes that a model is browsable on another device it is strongly recommended to prevent recursive models. For example, TR-069 cannot handle recursive models.
- *Only a single format per meta node* - Handling different types in different nodes simplifies the data conversion for both foreign and native protocols. Having a single choice from the Meta Node makes the conversion straightforward and does not require guessing.
- *All nodes must provide a Meta Node* - Conversion without a Meta Node makes the conversion very hard since object model schemas are often not available in the Protocol Adapter.
- *Naming* - Structured node members must have names only consisting of [a-zA-Z0-9] and must always start with a character [a-zA-Z]. Member names must be different regardless of the case, that is Abc and ABC must not both be members of the same structured node. The reason for this restriction is that it makes it more likely that the chosen names are compatible with the supported protocols and do not require escaping.
- *Typing* - Restrict the used formats to formats that maximize both the interoperability as the ease of use for Java developers. The following type are widely supported and are easy to use from Java:
 - `FORMAT_STRING`
 - `FORMAT_BOOLEAN`
 - `FORMAT_INTEGER`
 - `FORMAT_LONG`
 - `FORMAT_FLOAT`
 - `FORMAT_DATE_TIME`
 - `FORMAT_BINARY`

117.12.5 DDF

The Data Description Format is part of OMA DM; it provides a description language for the object model. The following table provides an example of the Data Description Format as used in the OSGi specifications.

Name	Actions	Type	Card.	S	Description
FaultType	Get	integer	1	P	...

The columns have the following meanings:

- *Name* - The name of the node

- *Actions* - The set of actions that can be executed on the node, see *Operations* on page 22 .
- *Type* - The type of the node. All lower case are primitives, a name starting with an upper case is an interior node type. MAP, LIST, and SCAFFOLD are the special types. The NODE type is like an ANY type. Other type names are then further specified in the document. See *Types* on page 52 .
- *Cardinality* - The number of occurrences of the node, see *Cardinality* on page 23 .
- *Scope* - The scope of the node, see *Scope* on page 22 .
- *Description* - A description of the node.

117.12.6 Types

Each node is considered to have a *type*. The Dmt Admin service has a number of constructs that have typing like behavior. There are therefore the following *kind* of types:

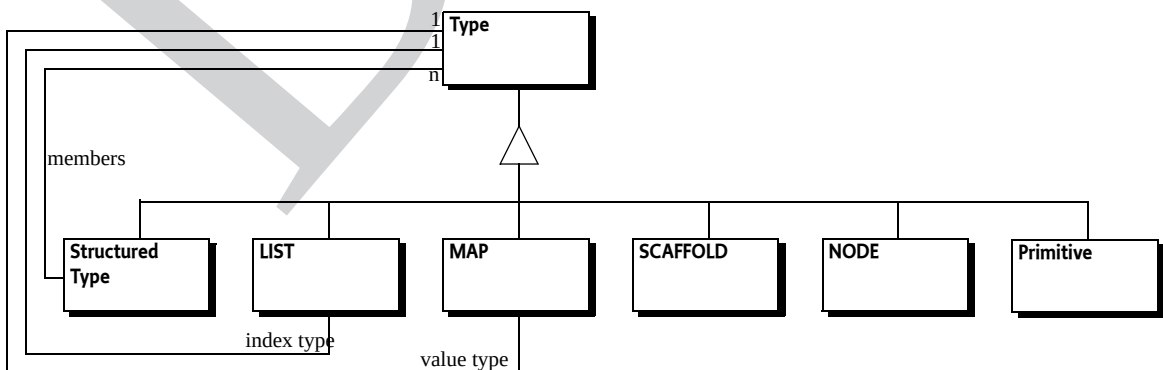
- *Primitives* - Primitives are data types like integers and strings; they include all the Dmt Admin data formats. See *Primitives* on page 53 . Primitive type names are always lower case to distinguish them from the interior node type names.
- *Structured Types* - A structured type types a structured node. See *Structured Nodes* on page 53 . A structured type has a type name that starts with an uppercase. Object models generally consist of defining these types.
- *NODE* - A general unqualified Dmt Admin node.
- *LIST* - A node that represents a homogeneous collection of child nodes; the name of the child nodes is the index in the collection. See *LIST Nodes* on page 53 .
- *MAP* - A node that represents a mapping from a key, the name of the child node, and a value, the value of the child node. All values have the same type. See *MAP Nodes* on page 55 .
- *SCAFFOLD* - A node provided by the Dmt Admin service or a Parent Plugin to make it possible to discover a DMT, see *Scaffold Nodes* on page 27 .

Nodes are treated as if there is a single type system. However, the Dmt Admin type system has the following mechanisms to type a node:

- *Format* - The Dmt Admin primitive types used for leaf nodes, as defined on Dmt Data.
- *MIME* - A MIME type on a leaf node which is available through [getNodeTypes\(String\)](#).
- *DDF Document URI* - A Data Description Format URI that provides a type name for an interior node. The URI provides a similar role as the MIME type for the leaf node and is also available through [getNodeTypes\(String\)](#).

The Dmt Admin service provides the MIME type for leaf nodes and the DDF Document URI for interior nodes through the [getNodeTypes\(String\)](#) method. As both are strings they can both be used as type identifiers. The different types are depicted in Figure 117.25 .

Figure 117.25 Type inheritance and structure



117.12.7 Primitives

A primitive is a value stored in a leaf node. In the Dmt Admin service, the type of the primitive is called the *format*. The Dmt Admin service supports a large number of types that have semantic overlap. A Protocol Mapping must provide a unique mapping from each Dmt Admin format to the corresponding protocol type and provide conversion from a protocol type to the corresponding Dmt Admin types defined in a Meta Node.

Primitives are documented in OSGi object models with a lower case name that is the last part of their format definition. For example, for `FORMAT_STRING` the DDF type name is string. A primitive DDF for an integer leaf node therefore looks like:

Name	Act	Type	Card.	S	Description
FaultType	Get	integer	1	P	...

117.12.8 Structured Nodes

A *structured node* is like a struct in C or a class in an object oriented languages. A structured node is an interior node with a set of members (child nodes) with fixed names, it is never possible to add or remove such members dynamically. The meaning of each named node and its type is usually defined in a management specification. For example, a node representing the OSGi Bundle could have a BundleId child-node that maps to the getBundleId() method on the Bundle interface.

It is an error to add or delete members to a Structured node, this must be reflected in the corresponding Meta Node, that is, Structured nodes must never have the Add or Delete action.

A structured node is defined in a *structured type* to allow the reuse of the same information in different places in an object model. A structured type defines the members and their behaviors. A structured type can be referred by its name. The name of the type is often, but not required, the name of the member.

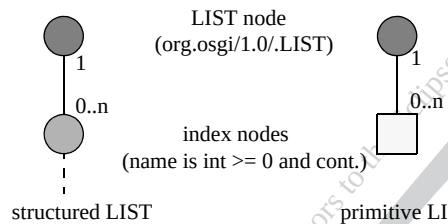
For example, a Unit structured type could look like:

Name	Act	Type	Card.	S	Description
Id	Get	long	1	P	...
URL	Get Set	string	1	P	...
Name	Get	string	1	P	...
Certificate	Get	LIST	1	P	
[index]	Get	Certificate	1	D	Note the use of a structured type.

117.12.9 LIST Nodes

A LIST node is an interior node representing a *collection* of elements. The elements are stored in the child nodes of the LIST node, they are called the *index nodes*. All index nodes must have the same type. The names of the index nodes are synthesized and represent the index of the index node. The first node is always named 0 and the sibling is 1, 2, etc. The sequence must be continuous and must have no missing indexes. A node name is always a string, it is therefore the responsibility of the plugin to provide the proper names. The index is assumed to be a signed positive integer limiting the LIST nodes size to Integer.MAX_VALUE elements.

Figure 117.26 LIST Nodes



Index nodes should only be used for types where the value of the index node is the identity. For example, a network interface has an identity; a manager will expect that a node representing such as a network interface node will always have the same URI even if other interfaces are added and deleted. Since LIST nodes renumber the index node names when an element is deleted or added, the URI would fail if a network interface was added or removed. If such a case, a MAP node should be used, see *MAP Nodes* on page 55 , as they allow the key to be managed by the remote manager.

LIST nodes can be mutable if the Meta Node of its index nodes support the Add or Delete action. A LIST node is modeled after a java.util.List that can automatically accommodate new elements. Get and Replace operations use the node name to index in this list.

To rearrange the list the local manager can Add and Delete nodes or rename them as it sees fit. At any moment in time the underlying implementation must maintain a list that runs from 0 to max(index) (inclusive), where index is the name of the LIST child nodes. Inserting a node requires renaming all subsequent nodes. Any missing indexes must automatically be provided by the plugin when the child node names are retrieved.

For example, a LIST node named L contains the following nodes:

L/0 A
L/1 B
L/2 C

To insert a node after B, L/2 must be renamed to L/3. This will automatically extend the LIST node to 4 elements. That is, even though L/2 is renamed, the implementation must automatically provide a new L/2 node. The value of this node depends on the underlying implementation. The value of the list will therefore then be: [A,B,?,C]. If node 1 is deleted, then the list will be [A,?,C]. If a node L/5 is added then the list will be [A,?,C,?,?,?]. It is usually easiest to use the LIST node as a complex value, this is discussed in the next section.

117.12.9.1 Complex Collections

An implementation of a LIST node must support a complex node value if its members are primitive; the interior node must then have a value of a Java object implementing the Collection interface from java.util. The elements in this map must be converted according to the following table.

Table 117.11 Conversion for Collections

Format	Associated Java Type
FORMAT_STRING	String
FORMAT_BOOLEAN	Boolean
FORMAT_INTEGER	Integer
FORMAT_LONG	Long
FORMAT_FLOAT	Float
FORMAT_DATE_TIME	Date
FORMAT_BINARY	byte[]

Alternatively, the Collection may contain Dmt Data objects but the collection must be homogeneous. The collection must always be a copy and changes made to the collection must not affect the DMT.

For example, a LIST type for a list of URIs could look like:

Name	Act	Type	Card.	S	Description
URIs	Get	LIST	1	P	A List of URIs
[index]	Get Set Add Del	string	0..n	D	A primitive index node

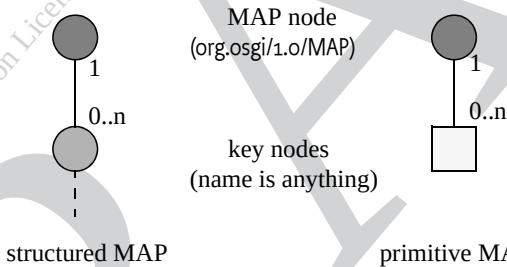
Replacing a complex value will generate a single `EVENT_TOPIC_REPLACED` event for the LIST node.

117.12.10

MAP Nodes

A MAP node represents a mapping from a *key* to a *value*. The key is the name of the node and the value is the node's value. A MAP node performs the same functions as a Java Map. See Figure 117.27 .

Figure 117.27 MAP Nodes



A MAP node has *key nodes* as children. A key node is an association between the name of the key node (which is the key) and the value of the key node. Key nodes are depicted with [*<type>*], where the *<type>* indicates the type used for the string name. For example, a long type will have node names that can be converted to a long. A key type must always be one of the primitive types. For example, a list of Bundle locations can be handled with a MAP with [string] key nodes that have a value type of string. Since the key is used in URIs it must always be escaped, see *The DMT Addressing URI* on page 13 .

For example:

Name	Act	Type	Card.	S	Description
Location	Get	MAP	1	P	A MAP of location where the index node is the Bundle Id.
[long]	Get Set Add Del	string	0..n	D	Name is the Bundle Id and the value is the location.

117.12.10.1

Complex Value

An implementation of a MAP node must support an interior node value if its child nodes are primitive; the interior node must then be associated with a Java object implementing the Map interface from java.util. The values in this Map must homogeneous and be converted according to Table 117.11 or the given values must of type DmtData. The Map object must a copy and does not track changes in the DMT or vice-versa.

Replacing a complex value will generate a single `EVENT_TOPIC_REPLACED` event for that node.

117.12.11 Instance Id

Some protocols cannot handle arbitrary names in the access URI, they need a well defined *instance id* to index in a table or put severe restrictions on the node name's character set, length, or other aspects. For example, TR-069 can access an object with the following URI:

Device.VOIP.12.Name

The more natural model for the DMT is to use:

Device.VOIP.<Name>...

To provide assistance to these protocols this section defines a mechanism that can be used by Protocol Adapters to simplify access.

An Object Model can define a child node `InstanceId`. The `InstanceId` node, if present, holds a long value that has the following qualities:

- Its value must be between 1 and `Long.MAX_VALUE`.
- No other index/key node on the same level must have the same value for the `InstanceId` node
- The value must be persistent between sessions and restarts of the plugin
- A value must not be reused when a node is deleted until the number space is exhausted

Protocol Adapters can use this information to provide alternative access paths for the DMT.

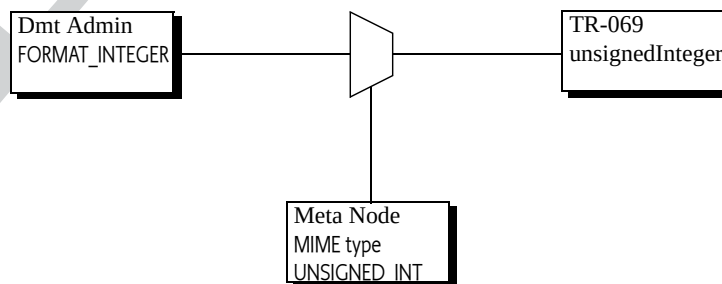
117.12.12 Conversions

Each Protocol Mapping document should define a default conversion from the Dmt Admin data formats to the protocol types and vice versa, including the `LIST` and `MAP` nodes. However, this default mapping is likely to be too constraining in real world models since different protocols support different data types and a 1:1 mapping is likely to be impossible.

For this reason, the Protocol Mapping document should define a number of protocol specific MIME types for each unique data type that they support. A Data Plugin can associate such a MIME type with a node. The Protocol Adapter can then look for this MIME type. If none of the Protocol Adapter specific MIME types are available in a node the default conversion is used.

For example, in the TR-069 Protocol Adapter specification there is a MIME type for each TR-069 data type. If for a given leaf node the Meta Node's type specifies `TR069_MIME_UNSIGNED_INT` and the node specifies the format `FORMAT_INTEGER` then the Protocol Adapter must convert the integer to an unsigned integer and encode the value as such in the response message. The Protocol Adapter there does not have to have specific knowledge of the object model, the Plugin drives the Protocol Adapter by providing the protocol specific MIME types on the leaf node Meta Nodes. This model is depicted in Figure 117.28.

Figure 117.28 Conversions



Since a Meta Node can contain multiple MIME types, there is no restrictions on the number of Protocol Adapters; a Plugin can specify the MIME types of multiple Protocol Adapters.

117.12.13

Extensions

All interior nodes in this specification can have a node named Ext. These nodes are the *extension* nodes. If an implementation needs to expose additional details about an interior node then they should expose these extensions under the corresponding Ext node. To reduce name conflicts, it is recommended to group together implementation specific extensions under a unique name, recommended is to use the reverse domain name. For example, the following DDF defines an Ext node with extensions for the ACME provider.

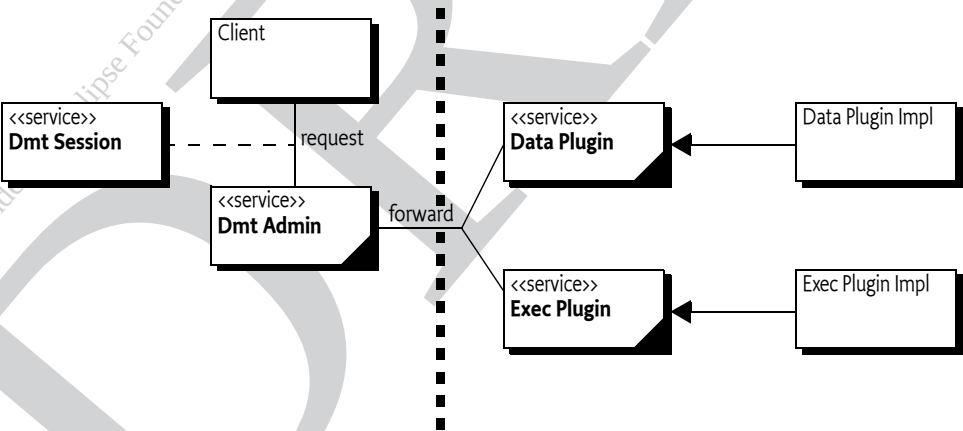
Name	Act	Type	Card.	S	Description
Framework	Get	Framework	1	P	...
Ext	Get		1	P	Extension node
com.acme	Get	AcmeFrameworkExt	1	P	The node for the ACME extensions
Transactional	Get	boolean	1	P	...

117.13

Security

A key aspect of the Dmt Admin service model is the separation from DMT clients and plugins. The Dmt Admin service receives all the operation requests and, after verification of authority, forwards the requests to the plugins.

Figure 117.29 Separation of clients and plugins



This architecture makes it straightforward to use the OSGi security architecture to protect the different actors.

117.13.1

Principals

The caller of the `getSession(String,String,int)` method must have the Dmt Principal Permission with a target that matches the given principal. This Dmt Principal Permission is used to enforce that only trusted entities can act on behalf of remote managers.

The Dmt Admin service must verify that all operations from a session with a principal can be executed on the given nodes using the available ACLs.

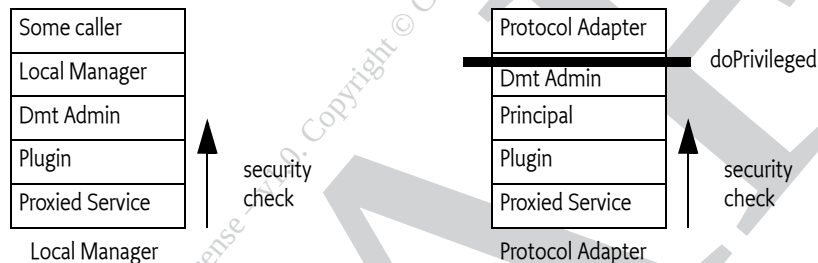
The other two forms of the `getSession` method are meant for local management applications where no principal is available. No special permission is defined to restrict the usage of these methods. The callers that want to execute device management commands, however, need to have the appropriate Dmt Permissions.

117.13.2 Operational Permissions

The operational security of a Local Manager and a remote manager is distinctly different. The distinction is made on the principal. Protocol Adapters should use the `getSession` method that takes an authenticated principal. Local Managers should not specify a principal.

Figure 117.30

Access control context, for Local Manager and Protocol Adapter operation



117.13.3 Protocol Adapters

A Protocol Adapter must provide a principal to the Dmt Admin service when it gets a session. It must use the `getSession(String,String,int)` method. The Protocol Adapter must have Dmt Principal Permission for the given principal. The Dmt Admin service must then use this principal to determine the *security scope* of the given principal. This security scope is a set of permissions. How these permissions are found is not defined in this specification; they are usually in the management tree of a device. For example, the Mobile Specification stores these under the `$/Policy/Java/DmtPrincipalPermission` sub-tree.

Additionally, a Dmt Session with a principal implies that the Dmt Admin service must verify the ACLs on the node for all operations.

Any operation that is requested by a Protocol Adapter must be executed in a `doPrivileged` block that takes the principal's security scope. The `doPrivileged` block effectively hides the permissions of the Protocol Adapter; all operations must be performed under the security scope of the principal.

The security check for a Protocol Adapter is therefore as follows:

- The operation method calls `doPrivileged` with the security scope of the principal.
- The operation is forwarded to the appropriate plugin. The underlying service must perform its normal security checks. For example, the Configuration Admin service must check for the appropriate Configuration Permission.

The Access Control context is shown in Figure 117.30 within the Protocol Adapter column.

This principal-based security model allows for minimal permissions on the Protocol Adapter, because the Dmt Admin service performs a `doPrivileged` on behalf of the principal, inserting the permissions for the principal on the call stack. This model does not guard against malicious Protocol Adapters, though the Protocol Adapter must have the appropriate Dmt Principal Permission.

The Protocol Adapter is responsible for the authentication of the principal. The Dmt Admin service must trust that the Protocol Adapter has correctly verified the identity of the other party. This specification does not address the type of authentication mechanisms that can be used. Once it has permission to use that principal, it can use any DMT command that is permitted for that principal at any time.

117.13.4 Local Manager

A Local Manager does not specify a principal. Security checks are therefore performed against the security scope of the Local Manager bundle, as shown in Figure 117.30 with the Local Manager stack. An operation is checked only with a Dmt Permission for the given node URI and operation. A thrown Security Exception must be passed unmodified to the caller of the operation method. The Dmt Admin service must not check the ACLs when no principal is set.

A Local Manager, and all its callers, must therefore have sufficient permission to handle the DMT operations as well as the permissions required by the plugins when they proxy other services (which is likely an extensive set of Permissions).

117.13.5 Plugin Security

Plugins are required to hold the maximum security scope for any services they proxy. For example, the plugin that manages the Configuration Admin service must have ConfigurationPermission("*","*") to be effective.

Plugins should not make doPrivileged calls, but should use the caller's context on the stack for permission checks.

117.13.6 Events and Permissions

Dmt Event Listener services must have the appropriate Dmt Permission to receive the event since this must be verified with the hasPermission() method on Bundle.

The Dmt Event Listener services registered with a [FILTER_PRINCIPAL](#) service property requires Dmt Principal Permission for the given principal. In this case, the principal must have Get access to see the nodes for the event. Any nodes that the listener does not have access to must be removed from the event.

Plugins are not required to have access to the Event Admin service. If they send an event through the [MountPoint](#) interface then the Dmt Admin service must use a doPrivileged block to send the event to the Event Admin service.

117.13.7 Dmt Principal Permission

Execution of the getSession methods of the Dmt Admin service featuring an explicit principal name is guarded by the Dmt Principal Permission. This permission must be granted only to Protocol Adapters that open Dmt Sessions on behalf of remote management servers.

The DmtPrincipalPermission class does not have defined actions; it must always be created with a * to allow future extensions. The target is the principal name. A wildcard character is allowed at the end of the string to match a prefix.

Example:

```
new DmtPrincipalPermission("com.acme.dep*", "*" )
```

117.13.8 Dmt Permission

The Dmt Permission controls access to management objects in the DMT. It is intended to control only the *local* access to the DMT. The Dmt Permission target string identifies the target node's URI (absolute path is required, starting with the './' prefix) and the action field lists the management commands that are permitted on the node.

The URI can end in a wildcard character * to indicate it is a prefix that must be matched. This comparison is string based so that node boundaries can be ignored.

The following actions are defined:

- [ADD](#)

- DELETE
- EXEC
- GET
- REPLACE

For example, the following code creates a Dmt Permission for a bundle to add and replace nodes in any URI that starts with ./D.

```
new DmtPermission("./D*", "Add, Replace")
```

This permission must imply the following permission:

```
new DmtPermission("./Dev/Operator/Name", "Replace")
```

117.13.9

Alert Permission

The Alert Permission permits the holder of this permission to send a notification to a specific *target principal*. The target is identical to *Dmt Principal Permission* on page 59 . No actions are defined for Alert Permission.

117.13.10

Security Summary

117.13.10.1

Dmt Admin Service and Notification Service

The Dmt Admin service is likely to require All Permission. This requirement is caused by the plug-in model. Any permission required by any of the plugins must be granted to the Dmt Admin service. This set of permissions is large and hard to define. The following list shows the minimum permissions required if the plugin permissions are left out.

ServicePermission	..DmtAdmin	REGISTER
ServicePermission	..NotificationService	REGISTER
ServicePermission	..DataPlugin	GET
ServicePermission	..ExecPlugin	GET
ServicePermission	..EventAdmin	GET
ServicePermission	..RemoteAlertSender	GET
ServicePermission	..DmtEventListener	GET
DmtPermission	*	*
DmtPrincipalPermission	*	*
PackagePermission	org.osgi.service.dmt	EXPORTONLY
PackagePermission	org.osgi.service.dmt.spi	EXPORTONLY
PackagePermission	org.osgi.service.dmt.notification	EXPORTONLY
PackagePermission	org.osgi.service.dmt.notification.spi	EXPORTONLY
PackagePermission	org.osgi.service.dmt.registry	EXPORTONLY
PackagePermission	org.osgi.service.dmt.security	EXPORTONLY

117.13.10.2

Dmt Event Listener Service

ServicePermission	..DmtEventListener	REGISTER
PackagePermission	org.osgi.service.dmt	IMPORT

Dmt Event Listeners must have the appropriate DmtPermission to see the nodes in the events. If they are registered with a principal then they also need DmtPrincipalPermission for the given principals.

117.13.10.3

Data and Exec Plugin

ServicePermission	..NotificationService	GET
-------------------	-----------------------	-----

ServicePermission	..DataPlugin	REGISTER
ServicePermission	..ExecPlugin	REGISTER
PackagePermission	org.osgi.service.dmt	IMPORT
PackagePermission	org.osgi.service.dmt.notification	IMPORT
PackagePermission	org.osgi.service.dmt.spi	IMPORT
PackagePermission	org.osgi.service.dmt.security	IMPORT

The plugin is also required to have any permissions to call its underlying services.

117.13.10.4**Local Manager**

ServicePermission	..DmtAdmin	GET
PackagePermission	org.osgi.service.dmt	IMPORT
PackagePermission	org.osgi.service.dmt.security	IMPORT
DmtPermission	<scope>	...

Additionally, the Local Manager requires all permissions that are needed by the plugins it addresses.

117.13.10.5**Protocol Adapter**

The Protocol Adapter only requires Dmt Principal Permission for the instances that it is permitted to manage. The other permissions are taken from the security scope of the principal.

ServicePermission	..DmtAdmin	GET
ServicePermission	RemoteAlertSender	REGISTER
PackagePermission	org.osgi.service.dmt	IMPORT
PackagePermission	org.osgi.service.dmt.notification.spi	IMPORT
PackagePermission	org.osgi.service.dmt.notification	IMPORT
DmtPrincipalPermission	<scope>	

117.14**org.osgi.service.dmt**

Device Management Tree Package Version 2.0.

This package contains the public API for the Device Management Tree manipulations. Permission classes are provided by the {@code org.osgi.service.dmt.security} package, and DMT plugin interfaces can be found in the {@code org.osgi.service.dmt.spi} package. Asynchronous notifications to remote management servers can be sent using the interfaces in the {@code org.osgi.service.dmt.notification} package.

Bundles wishing to use this package must list the package in the Import-Package header of the bundle's manifest. This package has two types of users: the consumers that use the API in this package and the providers that implement the API in this package.

Example import for consumers using the API in this package:

```
{@code Import-Package: org.osgi.service.dmt; version="[2.0,3.0)"}
```

Example import for providers implementing the API in this package:

```
{@code Import-Package: org.osgi.service.dmt; version="[2.0,2.1)"}
```

117.14.1**Summary**

- Acl - {@code Acl} is an immutable class representing structured access to DMT ACLs.
- DmtAdmin - An interface providing methods to open sessions and register listeners.
- DmtConstants - Defines standard names for {@code DmtAdmin}.
- DmtData - An immutable data structure representing the contents of a leaf or interior node.

- `DmtEvent` - Event class storing the details of a change in the tree.
- `DmtEventListener` - Registered implementations of this class are notified via `{@link DmtEvent}` objects about important changes in the tree.
- `DmtException` - Checked exception received when a DMT operation fails.
- `DmtIllegalStateException` - Unchecked illegal state exception.
- `DmtSession` - `DmtSession` provides concurrent access to the DMT.
- `MetaNode` - The `MetaNode` contains meta data as standardized by OMA DM but extends it (without breaking the compatibility) to provide for better DMT data quality in an environment where many software components manipulate this data.
- `Uri` - This class contains static utility methods to manipulate DMT URIs.

117.14.2 public final class `Acl`

`{@code Acl}` is an immutable class representing structured access to DMT ACLs. Under OMA DM the ACLs are defined as strings with an internal syntax.

The methods of this class taking a principal as parameter accept remote server IDs (as passed to `{@link DmtAdmin#getSession(String, String, int) DmtAdmin.getSession()}`), as well as "`{@code *}`" indicating any principal.

The syntax for valid remote server IDs:

`<server-identifier> ::=` All printable characters except `{@code '='}`, `{@code '&'}`, `{@code '*'}`, `{@code '+'}` or white-space characters.

117.14.2.1 public static final int `ADD = 2`

Principals holding this permission can issue ADD commands on the node having this ACL.

117.14.2.2 public static final int `ALL_PERMISSION = 31`

Principals holding this permission can issue any command on the node having this ACL. This permission is the logical OR of `{@link #ADD}`, `{@link #DELETE}`, `{@link #EXEC}`, `{@link #GET}` and `{@link #REPLACE}` permissions.

117.14.2.3 public static final int `DELETE = 8`

Principals holding this permission can issue DELETE commands on the node having this ACL.

117.14.2.4 public static final int `EXEC = 16`

Principals holding this permission can issue EXEC commands on the node having this ACL.

117.14.2.5 public static final int `GET = 1`

Principals holding this permission can issue GET command on the node having this ACL.

117.14.2.6 public static final int `REPLACE = 4`

Principals holding this permission can issue REPLACE commands on the node having this ACL.

117.14.2.7 public `Acl(String acl)`

acl The string representation of the ACL as defined in OMA DM. If `{@code null}` or empty then it represents an empty list of principals with no permissions.

- Create an instance of the ACL from its canonical string representation.

Throws `IllegalArgumentException`—if *acl* is not a valid OMA DM ACL string

117.14.2.8 public `Acl(String[][] principals, int[][] permissions)`

principals The array of principals

permissions The array of permissions

- Creates an instance with a specified list of principals and the permissions they hold. The two arrays run in parallel, that is {@code principals[i]} will hold {@code permissions[i]} in the ACL.

A principal name may not appear multiple times in the 'principals' argument. If the "*" principal appears in the array, the corresponding permissions will be granted to all principals (regardless of whether they appear in the array or not).

Throws `IllegalArgumentException` – if the length of the two arrays are not the same, if any array element is invalid, or if a principal appears multiple times in the {@code principals} array

117.14.2.9 **public synchronized Acl addPermission(String principal, int permissions)**

principal The entity to which permissions should be granted, or "*" to grant permissions to all principals.

permissions The permissions to be given. The parameter can be a logical {@code or} of more permission constants defined in this class.

- Create a new {@code Acl} instance from this {@code Acl} with the given permission added for the given principal. The already existing permissions of the principal are not affected.

Returns a new {@code Acl} instance

Throws `IllegalArgumentException` – if {@code principal} is not a valid principal name or if {@code permissions} is not a valid combination of the permission constants defined in this class

117.14.2.10 **public synchronized Acl deletePermission(String principal, int permissions)**

principal The entity from which permissions should be revoked, or "*" to revoke permissions from all principals.

permissions The permissions to be revoked. The parameter can be a logical {@code or} of more permission constants defined in this class.

- Create a new {@code Acl} instance from this {@code Acl} with the given permission revoked from the given principal. Other permissions of the principal are not affected.

Note, that it is not valid to revoke a permission from a specific principal if that permission is granted globally to all principals.

Returns a new {@code Acl} instance

Throws `IllegalArgumentException` – if {@code principal} is not a valid principal name, if {@code permissions} is not a valid combination of the permission constants defined in this class, or if a globally granted permission would have been revoked from a specific principal

117.14.2.11 **public boolean equals(Object obj)**

obj the object to compare with this {@code Acl} instance

- Checks whether the given object is equal to this {@code Acl} instance. Two {@code Acl} instances are equal if they allow the same set of permissions for the same set of principals.

Returns {@code true} if the parameter represents the same ACL as this instance

117.14.2.12 **public synchronized int getPermissions(String principal)**

principal The entity whose permissions to query, or "*" to query the permissions that are granted globally, to all principals

- Get the permissions associated to a given principal.

Returns The permissions of the given principal. The returned {@code int} is a bitmask of the permission constants defined in this class

Throws `IllegalArgumentException` – if {@code principal} is not a valid principal name

117.14.2.13 public String[][] getPrincipals()

- Get the list of principals who have any kind of permissions on this node. The list only includes those principals that have been explicitly assigned permissions (so "*" is never returned), globally set permissions naturally apply to all other principals as well.

Returns The array of principals having permissions on this node.

117.14.2.14 public int hashCode()

- Returns the hash code for this ACL instance. If two {@code Acl} instances are equal according to the {@code link #equals(Object)} method, then calling this method on each of them must produce the same integer result.

Returns hash code for this ACL

117.14.2.15 public synchronized boolean isPermitted(String principal, int permissions)

principal The entity to check, or "*" to check whether the given permissions are granted to all principals globally

permissions The permissions to check

- Check whether the given permissions are granted to a certain principal. The requested permissions are specified as a bitfield, for example {@code (Acl.ADD | Acl.DELETE | Acl.GET)}.

Returns {@code true} if the principal holds all the given permissions

Throws `IllegalArgumentException` – if {@code principal} is not a valid principal name or if {@code permissions} is not a valid combination of the permission constants defined in this class

117.14.2.16 public synchronized Acl setPermission(String principal, int permissions)

principal The entity to which permissions should be granted, or "*" to globally grant permissions to all principals.

permissions The set of permissions to be given. The parameter is a bitmask of the permission constants defined in this class.

- Create a new {@code Acl} instance from this {@code Acl} where all permissions for the given principal are overwritten with the given permissions.

Note, that when changing the permissions of a specific principal, it is not allowed to specify a set of permissions stricter than the global set of permissions (that apply to all principals).

Returns a new {@code Acl} instance

Throws `IllegalArgumentException` – if {@code principal} is not a valid principal name, if {@code permissions} is not a valid combination of the permission constants defined in this class, or if a globally granted permission would have been revoked from a specific principal

117.14.2.17 public synchronized String toString()

- Give the canonical string representation of this ACL. The operations are in the following order: {Add, Delete, Exec, Get, Replace}, principal names are sorted alphabetically.

Returns The string representation as defined in OMA DM.

117.14.3 public abstract DmtAdmin

An interface providing methods to open sessions and register listeners. The implementation of {@code DmtAdmin} should register itself in the OSGi service registry as a service. {@code DmtAdmin} is the entry point for applications to use the DMT API.

The {@code getSession} methods are used to open a session on a specified subtree of the DMT. A typical way of usage:

```

serviceRef = context.getServiceReference(DmtAdmin.class.getName());
DmtAdmin admin = (DmtAdmin) context.getService(serviceRef);
DmtSession session = admin.getSession("/OSGi/Configuration");
session.createInteriorNode("/OSGi/Configuration/my.table");

```

The methods for opening a session take a node URI (the session root) as a parameter. All segments of the given URI must be within the segment length limit of the implementation, and the special characters '/' and '\' must be escaped (preceded by a '\').

See the {@link Uri#encode(String)} method for support on escaping invalid characters in a URI.

It is possible to specify a lock mode when opening the session (see lock type constants in {@link DmtSession}). This determines whether the session can run in parallel with other sessions, and the kinds of operations that can be performed in the session. All Management Objects constituting the device management tree must support read operations on their nodes, while support for write operations depends on the Management Object. Management Objects supporting write access may support transactional write, non-transactional write or both. Users of {@code DmtAdmin} should consult the Management Object specification and implementation for the supported update modes. If Management Object definition permits, implementations are encouraged to support both update modes.

117.14.3.1 **public abstract DmtSession getSession(String subtreeUri) throws DmtException**

subtreeUri the subtree on which DMT manipulations can be performed within the returned session

- Opens a {@code DmtSession} for local usage on a given subtree of the DMT with non transactional write lock. This call is equivalent to the following: {@code getSession(null, subtreeUri, DmtSession.LOCK_TYPE_EXCLUSIVE)}

The {@code subtreeUri} parameter must contain an absolute URI. It can also be {@code null}, in this case the session is opened with the default session root, ".", that gives access to the whole tree.

To perform this operation the caller must have {@code DmtPermission} for the {@code subtreeUri} node with the Get action present.

Returns a {@code DmtSession} object for the requested subtree

Throws DmtException– with the following possible error codes:

- {@code INVALID_URI} if {@code subtreeUri} is syntactically invalid
- {@code URI_TOO_LONG} if {@code subtreeUri} is longer than accepted by the {@code DmtAdmin} implementation (especially on systems with limited resources)
- {@code NODE_NOT_FOUND} if {@code subtreeUri} specifies a non-existing node
- {@code SESSION_CREATION_TIMEOUT} if the operation timed out because of another ongoing session
- {@code COMMAND_FAILED} if {@code subtreeUri} specifies a relative URI, or some unspecified error is encountered while attempting to complete the command

SecurityException– if the caller does not have {@code DmtPermission} for the given root node with the Get action present

117.14.3.2 **public abstract DmtSession getSession(String subtreeUri, int lockMode) throws DmtException**

subtreeUri the subtree on which DMT manipulations can be performed within the returned session

lockMode one of the lock modes specified in {@code DmtSession}

- Opens a {@code DmtSession} for local usage on a specific DMT subtree with a given lock mode. This call is equivalent to the following: {@code getSession(null, subtreeUri, lockMode)}

The {@code subtreeUri} parameter must contain an absolute URI. It can also be {@code null}, in this case the session is opened with the default session root, ".", that gives access to the whole tree.

To perform this operation the caller must have {@code DmtPermission} for the {@code subtreeUri} node with the Get action present.

Returns a {@code DmtSession} object for the requested subtree

Throws `DmtException`– with the following possible error codes:

- {@code INVALID_URI} if {@code subtreeUri} is syntactically invalid
- {@code URI_TOO_LONG} if {@code subtreeUri} is longer than accepted by the {@code DmtAdmin} implementation (especially on systems with limited resources)
- {@code NODE_NOT_FOUND} if {@code subtreeUri} specifies a non-existing node
- {@code FEATURE_NOT_SUPPORTED} if atomic sessions are not supported by the implementation and {@code lockMode} requests an atomic session
- {@code SESSION_CREATION_TIMEOUT} if the operation timed out because of another ongoing session
- {@code COMMAND_FAILED} if {@code subtreeUri} specifies a relative URI, if {@code lockMode} is unknown, or some unspecified error is encountered while attempting to complete the command

`SecurityException`– if the caller does not have {@code DmtPermission} for the given root node with the Get action present

117.14.3.3

public abstract DmtSession getSession(String principal, String subtreeUri, int lockMode) throws DmtException

principal the identifier of the remote server on whose behalf the data manipulation is performed, or {@code null} for local sessions

subtreeUri the subtree on which DMT manipulations can be performed within the returned session

lockMode one of the lock modes specified in {@code DmtSession}

- Opens a {@code DmtSession} on a specific DMT subtree using a specific lock mode on behalf of a remote principal. If local management applications are using this method then they should provide {@code null} as the first parameter. Alternatively they can use other forms of this method without providing a principal string.

The {@code subtreeUri} parameter must contain an absolute URI. It can also be {@code null}, in this case the session is opened with the default session root, ".", that gives access to the whole tree.

This method is guarded by {@code DmtPrincipalPermission} in case of remote sessions. In addition, the caller must have Get access rights (ACL in case of remote sessions, {@code DmtPermission} in case of local sessions) on the {@code subtreeUri} node to perform this operation.

Returns a {@code DmtSession} object for the requested subtree

Throws `DmtException`– with the following possible error codes:

- {@code INVALID_URI} if {@code subtreeUri} is syntactically invalid
- {@code URI_TOO_LONG} if {@code subtreeUri} is longer than accepted by the {@code DmtAdmin} implementation (especially on systems with limited resources)
- {@code NODE_NOT_FOUND} if {@code subtreeUri} specifies a non-existing node
- {@code PERMISSION_DENIED} if {@code principal} is not {@code null} and the ACL of the node does not allow the {@code Get} operation for the principal on the given root node
- {@code FEATURE_NOT_SUPPORTED} if atomic sessions are not supported by the implementation and {@code lockMode} requests an atomic session
- {@code SESSION_CREATION_TIMEOUT} if the operation timed out because of another ongoing session

- `{@code COMMAND_FAILED}` if `{@code subtreeUri}` specifies a relative URI, if `{@code lock-Mode}` is unknown, or some unspecified error is encountered while attempting to complete the command

`SecurityException` – in case of remote sessions, if the caller does not have the required `{@code Dmt-PrincipalPermission}` with a target matching the `{@code principal}` parameter, or in case of local sessions, if the caller does not have `{@code DmtPermission}` for the given root node with the Get action present

117.14.4 **public class DmtConstants**

Defines standard names for `{@code DmtAdmin}`.

Since `@since 2.0`

117.14.4.1 **public static final String DDF_LIST = "org.osgi/1.0/LIST"**

A string defining a DDF URI, indicating that the node is a LIST node.

117.14.4.2 **public static final String DDF_MAP = "org.osgi/1.0/MAP"**

A string defining a DDF URI, indicating that the node is a MAP node.

117.14.4.3 **public static final String DDF_SCAFFOLD = "org.osgi/1.0/SCAFFOLD"**

A string defining a DDF URI, indicating that the node is a SCAFFOLD node.

117.14.4.4 **public static final String EVENT_PROPERTY_NEW_NODES = "newnodes"**

A string defining the property key for the `{@code newnodes}` property in node related events.

117.14.4.5 **public static final String EVENT_PROPERTY_NODES = "nodes"**

A string defining the property key for the `@{code nodes}` property in node related events.

117.14.4.6 **public static final String EVENT_PROPERTY_SESSION_ID = "session.id"**

A string defining the property key for the `{@code session.id}` property in node related events.

117.14.4.7 **public static final String EVENT_TOPIC_ADDED = "org/osgi/service/dmt/DmtEvent/ADDED"**

A string defining the topic for the event that is sent for added nodes.

117.14.4.8 **public static final String EVENT_TOPIC_COPIED = "org/osgi/service/dmt/DmtEvent/COPIED"**

A string defining the topic for the event that is sent for copied nodes.

117.14.4.9 **public static final String EVENT_TOPIC_DELETED = "org/osgi/service/dmt/DmtEvent/DELETED"**

A string defining the topic for the event that is sent for deleted nodes.

117.14.4.10 **public static final String EVENT_TOPIC_RENAMED = "org/osgi/service/dmt/DmtEvent/RENAMED"**

A string defining the topic for the event that is sent for renamed nodes.

117.14.4.11 **public static final String EVENT_TOPIC_REPLACED = "org/osgi/service/dmt/DmtEvent/REPLACED"**

A string defining the topic for the event that is sent for replaced nodes.

117.14.4.12 **public static final String EVENT_TOPIC_SESSION_CLOSED = "org/osgi/service/dmt/DmtEvent/SESSION_CLOSED"**

A string defining the topic for the event that is sent for a closed session.

117.14.4.13 **public static final String EVENT_TOPIC_SESSION_OPENED = "org/osgi/service/dmt/DmtEvent/SESSION_OPENED"**

A string defining the topic for the event that is sent for a newly opened session.

117.14.5 public final class DmtData

An immutable data structure representing the contents of a leaf or interior node. This structure represents only the value and the format property of the node, all other properties (like MIME type) can be set and read using the {@code DmtSession} interface.

Different constructors are available to create nodes with different formats. Nodes of {@code null} format can be created using the static {@code NULL_VALUE} constant instance of this class.

{@code FORMAT_RAW_BINARY} and {@code FORMAT_RAW_STRING} enable the support of future data formats. When using these formats, the actual format name is specified as a {@code String}. The application is responsible for the proper encoding of the data according to the specified format.

Concurrency Immutable

117.14.5.1 public static final DmtData FALSE_VALUE

Constant instance representing a boolean {@code false} value.

Since @since 2.0

117.14.5.2 public static final int FORMAT_BASE64 = 128

The node holds an OMA DM {@code b64} value. Like {@code FORMAT_BINARY}, this format is also represented by the Java {@code byte[]} type, the difference is only in the corresponding OMA DM format. This format does not affect the internal storage format of the data as {@code byte[]}. It is intended as a hint for the external representation of this data. Protocol Adapters can use this hint for their further processing.

117.14.5.3 public static final int FORMAT_BINARY = 64

The node holds an OMA DM {@code bin} value. The value of the node corresponds to the Java {@code byte[]} type.

117.14.5.4 public static final int FORMAT_BOOLEAN = 8

The node holds an OMA DM {@code bool} value.

117.14.5.5 public static final int FORMAT_DATE = 16

The node holds an OMA DM {@code date} value.

117.14.5.6 public static final int FORMAT_DATE_TIME = 16384

The node holds a Date object. If the `getTime()` equals zero then the date time is not known. If the `getTime()` is negative it must be interpreted as a relative number of milliseconds.

Since @since 2.0

117.14.5.7 public static final int FORMAT_FLOAT = 2

The node holds an OMA DM {@code float} value.

117.14.5.8 public static final int FORMAT_INTEGER = 1

The node holds an OMA DM {@code int} value.

117.14.5.9 public static final int FORMAT_LONG = 8192

The node holds a long value. The {@code getFormatName()} method can be used to get the actual format name.

Since @since 2.0

117.14.5.10 public static final int FORMAT_NODE = 1024

Format specifier of an internal node. An interior node can hold a Java object as value (see {@link DmtData#DmtData(Object)} and {@link DmtData#getNode()}). This value can be used by Java programs that know a specific URI understands the associated Java type. This type is further used as a return value of the {@link MetaNode#getFormat()} method for interior nodes.

117.14.5.11 public static final int FORMAT_NULL = 512

The node holds an OMA DM {@code null} value. This corresponds to the Java {@code null} type.

117.14.5.12 public static final int FORMAT_RAW_BINARY = 4096

The node holds raw protocol data encoded in binary format. The {@link #getFormatName()} method can be used to get the actual format name.

117.14.5.13 public static final int FORMAT_RAW_STRING = 2048

The node holds raw protocol data encoded as {@code String}. The {@link #getFormatName()} method can be used to get the actual format name.

117.14.5.14 public static final int FORMAT_STRING = 4

The node holds an OMA DM {@code chr} value.

117.14.5.15 public static final int FORMAT_TIME = 32

The node holds an OMA DM {@code time} value.

117.14.5.16 public static final int FORMAT_XML = 256

The node holds an OMA DM {@code xml} value.

117.14.5.17 public static final DmtData NULL_VALUE

Constant instance representing a leaf node of {@code null} format.

117.14.5.18 public static final DmtData TRUE_VALUE

Constant instance representing a boolean {@code true} value.

Since @since 2.0

117.14.5.19 public DmtData(String string)

string the string value to set

- Create a {@code DmtData} instance of {@code chr} format with the given string value. The {@code null} string argument is valid.

117.14.5.20 public DmtData(Date date)

date the Date object to set

- Create a {@code DmtData} instance of {@code dateTime} format with the given Date value. The given Date value must be a non-null {@code Date} object.

117.14.5.21 public DmtData(Object complex)

complex the complex data object to set

- Create a {@code DmtData} instance of {@code node} format with the given object value. The value represents complex data associated with an interior node.

Certain interior nodes can support access to their subtrees through such complex values, making it simpler to retrieve or update all leaf nodes in a subtree.

The given value must be a non-{@code null} immutable object.

117.14.5.22 **public DmtData(String value, int format)**

value the string, XML, date, or time value to set

format the format of the {@code DmtData} instance to be created, must be one of the formats specified above

- Create a {@code DmtData} instance of the specified format and set its value based on the given string. Only the following string-based formats can be created using this constructor:
 - {@link #FORMAT_STRING} - value can be any string
 - {@link #FORMAT_XML} - value must contain an XML fragment (the validity is not checked by this constructor)
 - {@link #FORMAT_DATE} - value must be parsable to an ISO 8601 calendar date in complete representation, basic format (pattern {@code CCYYMMDD})
 - {@link #FORMAT_TIME} - value must be parsable to an ISO 8601 time of day in either local time, complete representation, basic format (pattern {@code hhmmss}) or Coordinated Universal Time, basic format (pattern {@code hhmmssZ})

* The {@code null} string argument is only valid if the format is string or XML.

Throws `IllegalArgumentException` - if {@code format} is not one of the allowed formats, or {@code value} is not a valid string for the given format

`NullPointerException` - if a string, XML, date, or time is constructed and {@code value} is {@code null}

117.14.5.23 **public DmtData(int integer)**

integer the integer value to set

- Create a {@code DmtData} instance of {@code int} format and set its value.

117.14.5.24 **public DmtData(float flt)**

flt the float value to set

- Create a {@code DmtData} instance of {@code float} format and set its value.

117.14.5.25 **public DmtData(long lng)**

lng the long value to set

- Create a {@code DmtData} instance of {@code long} format and set its value.

Since @since 2.0

117.14.5.26 **public DmtData(boolean bool)**

bool the boolean value to set

- Create a {@code DmtData} instance of {@code bool} format and set its value.

117.14.5.27 **public DmtData(byte[][] bytes)**

bytes the byte array to set, must not be {@code null}

- Create a {@code DmtData} instance of {@code bin} format and set its value.

Throws `NullPointerException` - if {@code bytes} is {@code null}

117.14.5.28 **public DmtData(byte[][] bytes, boolean base64)**

bytes the byte array to set, must not be {@code null}

base64 if {@code true}, the new instance will have {@code b64} format, if {@code false}, it will have {@code bin} format

- Create a {@code DmtData} instance of {@code bin} or {@code b64} format and set its value. The chosen format is specified by the {@code base64} parameter.

Throws `NullPointerException`– if {@code bytes} is {@code null}

117.14.5.29 **public DmtData(byte[][] bytes, int format)**

bytes the byte array to set, must not be {@code null}

format the format of the `DmtData` instance to be created, must be one of the formats specified above

- Create a {@code DmtData} instance of the specified format and set its value based on the given {@code byte[]}. Only the following {@code byte[]} based formats can be created using this constructor:
 - {@link #FORMAT_BINARY}
 - {@link #FORMAT_BASE64}

Throws `IllegalArgumentException`– if format is not one of the allowed formats

`NullPointerException`– if {@code bytes} is {@code null}

117.14.5.30 **public DmtData(String formatName, String data)**

formatName the name of the format, must not be {@code null}

data the data encoded according to the specified format, must not be {@code null}

- Create a {@code DmtData} instance in {@link #FORMAT_RAW_STRING} format. The data is provided encoded as a {@code String}. The actual data format is specified in {@code formatName}. The encoding used in {@code data} must conform to this format.

Throws `NullPointerException`– if {@code formatName} or {@code data} is {@code null}

117.14.5.31 **public DmtData(String formatName, byte[][] data)**

formatName the name of the format, must not be {@code null}

data the data encoded according to the specified format, must not be {@code null}

- Create a {@code DmtData} instance in {@link #FORMAT_RAW_BINARY} format. The data is provided encoded as binary. The actual data format is specified in {@code formatName}. The encoding used in {@code data} must conform to this format.

Throws `NullPointerException`– if {@code formatName} or {@code data} is {@code null}

117.14.5.32 **public boolean equals(Object obj)**

obj the object to compare with this {@code DmtData}

- Compares the specified object with this {@code DmtData} instance. Two {@code DmtData} objects are considered equal if their format is the same, and their data (selected by the format) is equal.

In case of {@link #FORMAT_RAW_BINARY} and {@link #FORMAT_RAW_STRING} the textual name of the data format - as returned by {@link #getFormatName()} - must be equal as well.

Returns true if the argument represents the same {@code DmtData} as this object

117.14.5.33 **public byte[][] getBase64()**

- Gets the value of a node with base 64 ({@code b64}) format.

Returns the binary value

Throws `DmtIllegalStateException`– if the format of the node is not base 64.

117.14.5.34 public byte[][] getBinary()

- Gets the value of a node with binary ({@code bin}) format.

Returns the binary value

Throws `DmtIllegalStateException` – if the format of the node is not binary

117.14.5.35 public boolean getBoolean()

- Gets the value of a node with boolean ({@code bool}) format.

Returns the boolean value

Throws `DmtIllegalStateException` – if the format of the node is not boolean

117.14.5.36 public String getDate()

- Gets the value of a node with date format. The returned date string is formatted according to the ISO 8601 definition of a calendar date in complete representation, basic format (pattern {@code CCYYMMDD}).

Returns the date value

Throws `DmtIllegalStateException` – if the format of the node is not date

117.14.5.37 public Date getDateTime()

- Gets the value of a node with {@code dateTime} format.

Returns the Date value

Throws `DmtIllegalStateException` – if the format of the node is not time

Since @since 2.0

117.14.5.38 public float getFloat()

- Gets the value of a node with {@code float} format.

Returns the float value

Throws `DmtIllegalStateException` – if the format of the node is not {@code float}

117.14.5.39 public int getFormat()

- Get the node's format, expressed in terms of type constants defined in this class. Note that the 'format' term is a legacy from OMA DM, it is more customary to think of this as 'type'.

Returns the format of the node

117.14.5.40 public String getFormatName()

- Returns the format of this {@code DmtData} as {@code String}. For the predefined data formats this is the OMA DM defined name of the format. For {@link #FORMAT_RAW_STRING} and {@link #FORMAT_RAW_BINARY} this is the format specified when the object was created.

Returns the format name as {@code String}

117.14.5.41 public int getInt()

- Gets the value of a node with integer ({@code int}) format.

Returns the integer value

Throws `DmtIllegalStateException` – if the format of the node is not integer

117.14.5.42 public long getLong()

- Gets the value of a node with long format.

Returns the long value

Throws `DmtIllegalStateException`– if the format of the node is not long

Since @since 2.0

117.14.5.43 `public Object getNode()`

- Gets the complex data associated with an interior node ({@code node} format).

Certain interior nodes can support access to their subtrees through complex values, making it simpler to retrieve or update all leaf nodes in the subtree.

Returns the data object associated with an interior node

Throws `DmtIllegalStateException`– if the format of the data is not {@code node}

117.14.5.44 `public byte[][] getRawBinary()`

- Gets the value of a node in raw binary ({@link #FORMAT_RAW_BINARY}) format.

Returns the data value in raw binary format

Throws `DmtIllegalStateException`– if the format of the node is not raw binary

117.14.5.45 `public String getRawString()`

- Gets the value of a node in raw {@code String} ({@link #FORMAT_RAW_STRING}) format.

Returns the data value in raw {@code String} format

Throws `DmtIllegalStateException`– if the format of the node is not raw {@code String}

117.14.5.46 `public int getSize()`

- Get the size of the data. The returned value depends on the format of data in the node:
 - {@link #FORMAT_STRING}, {@link #FORMAT_XML}, {@link #FORMAT_BINARY}, {@link #FORMAT_BASE64}, {@link #FORMAT_RAW_STRING}, and {@link #FORMAT_RAW_BINARY}: the length of the stored data, or 0 if the data is {@code null}
 - {@link #FORMAT_INTEGER} and {@link #FORMAT_FLOAT}: 4
 - {@link #FORMAT_LONG} and {@link #FORMAT_DATE_TIME}: 8
 - {@link #FORMAT_DATE} and {@link #FORMAT_TIME}: the length of the date or time in its string representation
 - {@link #FORMAT_BOOLEAN}: 1
 - {@link #FORMAT_NODE}: -1 (unknown)
 - {@link #FORMAT_NULL}: 0

Returns the size of the data stored by this object

117.14.5.47 `public String getString()`

- Gets the value of a node with string ({@code chr}) format.

Returns the string value

Throws `DmtIllegalStateException`– if the format of the node is not string

117.14.5.48 `public String getTime()`

- Gets the value of a node with time format. The returned time string is formatted according to the ISO 8601 definition of the time of day. The exact format depends on the value the object was initialized with: either local time, complete representation, basic format (pattern {@code hhmmss}) or Coordinated Universal Time, basic format (pattern {@code hhmmssZ}).

Returns the time value

Throws `DmtIllegalStateException` – if the format of the node is not time

117.14.5.49 **public String getXml()**

- Gets the value of a node with {@code xml} format.

Returns the XML value

Throws `DmtIllegalStateException` – if the format of the node is not {@code xml}

117.14.5.50 **public int hashCode()**

- Returns the hash code value for this {@code DmtData} instance. The hash code is calculated based on the data (selected by the format) of this object.

Returns the hash code value for this object

117.14.5.51 **public String toString()**

- Gets the string representation of the {@code DmtData}. This method works for all formats.

For string format data - including {@link #FORMAT_RAW_STRING} - the string value itself is returned, while for XML, date, time, integer, float, boolean, long and node formats the string form of the value is returned. Binary - including {@link #FORMAT_RAW_BINARY} - base64 data is represented by two-digit hexadecimal numbers for each byte separated by spaces. The {@link #NULL_VALUE} data has the string form of "`null`". Data of string or XML format containing the Java `null` value is represented by an empty string. `DateTime` data is formatted as {@code yyyy-MM-dd'T'HH:mm:ss'Z'}).

Returns the string representation of this {@code DmtData} instance

117.14.6 **public abstract DmtEvent**

Event class storing the details of a change in the tree. {@code DmtEvent} is used by {@code DmtAdmin} to notify registered {@link DmtEventListener EventListeners} services about important changes. Events are generated after every successful DMT change, and also when sessions are opened or closed. If a {@link DmtSession} is opened in atomic mode, DMT events are only sent when the session is committed, when the changes are actually performed.

The {@code type} of the event describes the change that triggered the event delivery. Each event carries the unique identifier of the session in which the described change happened or -1 when the change originated outside a session. The events describing changes in the DMT carry the list of affected nodes. In case of {@link #COPIED} or {@link #RENAMED} events, the event carries the list of new nodes as well.

117.14.6.1 **public static final int ADDED = 1**

Event type indicating nodes that were added.

117.14.6.2 **public static final int COPIED = 2**

Event type indicating nodes that were copied.

117.14.6.3 **public static final int DELETED = 4**

Event type indicating nodes that were deleted.

117.14.6.4 **public static final int RENAMED = 8**

Event type indicating nodes that were renamed.

117.14.6.5 **public static final int REPLACED = 16**

Event type indicating nodes that were replaced.

117.14.6.6 public static final int SESSION_CLOSED = 64

Event type indicating that a session was closed. This type of event is sent when the session is closed by the client or becomes inactive for any other reason (session timeout, fatal errors in business methods, etc.).

117.14.6.7 public static final int SESSION_OPENED = 32

Event type indicating that a new session was opened.

117.14.6.8 public abstract String[][] getNewNodes()

- This method can be used to query the new nodes, when the type of the event is {@link #COPIED} or {@link #RENAMED}. For all other event types this method returns {@code null}.

The array returned by this method runs parallel to the array returned by {@link #getNodes()}, the elements in the two arrays contain the source and destination URIs for the renamed or copied nodes in the same order. All returned URIs are absolute.

This method returns only those nodes where the caller has the GET permission for the source or destination node of the operation. Therefore, it is possible that the method returns an empty array.

Returns the array of newly created nodes

117.14.6.9 public abstract String[][] getNodes()

- This method can be used to query the subject nodes of this event. The method returns {@code null} for {@link #SESSION_OPENED} and {@link #SESSION_CLOSED}.

The method returns only those affected nodes that the caller has the GET permission for (or in case of {@link #COPIED} or {@link #RENAMED} events, where the caller has GET permissions for either the source or the destination nodes). Therefore, it is possible that the method returns an empty array. All returned URIs are absolute.

Returns the array of affected nodes

See Also `getNewNodes()`

117.14.6.10 public abstract Object getProperty(String key)

key the name of the requested property

- This method can be used to get the value of a single event property.

Returns the requested property value or null, if the key is not contained in the properties

See Also `getPropertyNames()`

Since @since 2.0

117.14.6.11 public abstract String[][] getPropertyNames()

- This method can be used to query the names of all properties of this event.

The returned names can be used as key value in subsequent calls to {@link #getProperty(String)}.

Returns the array of property names

See Also `getProperty(String)`

Since @since 2.0

117.14.6.12 public abstract int getSessionId()

- This method returns the identifier of the session in which this event took place. The ID is guaranteed to be unique on a machine.

For events that do not result from a session, the session id is -1.

The availability of a session.id can also be checked by using {@code getProperty()} with "session.id" as key.

Returns the unique identifier of the session that triggered the event or -1 if there is no session associated

117.14.6.13 **public abstract int getType()**

- This method returns the type of this event.

Returns the type of this event.

117.14.7 **public abstract DmtEventListener**

Registered implementations of this class are notified via {@link DmtEvent} objects about important changes in the tree. Events are generated after every successful DMT change, and also when sessions are opened or closed. If a {@link DmtSession} is opened in atomic mode, DMT events are only sent when the session is committed, when the changes are actually performed.

Dmt Event Listener services must have permission {@link DmtPermission#GET} for the nodes in the {@code nodes} and {@code newNodes} property in the Dmt Event.

117.14.7.1 **public static final String FILTER_EVENT = "osgi.filter.event"**

A number of event types packed in a bitmap. If this service property is provided with a Dmt Event Listener service registration then that listener must only receive events where one of the Dmt Event types occur in the bitmap. The type of this service property must be {@code Integer}.

117.14.7.2 **public static final String FILTER_PRINCIPAL = "osgi.filter.principal"**

A number of names of principals. If this service property is provided with a Dmt Event Listener service registration then that listener must only receive events for which at least one of the given principals has {@code Get} rights. The type of this service property is {@code String+}.

117.14.7.3 **public static final String FILTER_SUBTREE = "osgi.filter.subtree"**

A number of sub-tree top nodes that define the scope of the Dmt Event Listener. If this service property is registered then the service must only receive events for nodes that are part of one of the subtrees. The type of this service property is {@code String+}.

117.14.7.4 **public abstract void changeOccurred(DmtEvent event)**

event the {@code DmtEvent} describing the change in detail

- {@code DmtAdmin} uses this method to notify the registered listeners about the change. This method is called asynchronously from the actual event occurrence.

117.14.8 **public class DmtException extends Exception**

Checked exception received when a DMT operation fails. Beside the exception message, a {@code DmtException} always contains an error code (one of the constants specified in this class), and may optionally contain the URI of the related node, and information about the cause of the exception.

Some of the error codes defined in this class have a corresponding error code defined in OMA DM, in these cases the name and numerical value from OMA DM is used. Error codes without counterparts in OMA DM were given numbers from a different range, starting from 1.

The cause of the exception (if specified) can either be a single {@code Throwable} instance, or a list of such instances if several problems occurred during the execution of a method. An example for the latter is the {@code close} method of {@code DmtSession} that tries to close multiple plugins, and has to report the exceptions of all failures.

Each constructor has two variants, one accepts a {@code String} node URI, the other accepts a {@code String[]} node path. The former is used by the DmtAdmin implementation, the latter by the

plugins, who receive the node URI as an array of segment names. The constructors are otherwise identical.

Getter methods are provided to retrieve the values of the additional parameters, and the {@code printStackTrace(PrintWriter)} method is extended to print the stack trace of all causing throwables as well.

117.14.8.1 `public static final int ALERT_NOT_ROUTED = 5`

An alert can not be sent from the device to the given principal. This can happen if there is no Remote Alert Sender willing to forward the alert to the given principal, or if no principal was given and the DmtAdmin did not find an appropriate default destination.

This error code does not correspond to any OMA DM response status code. It should be translated to the code 500 "Command Failed" when transferring over OMA DM.

117.14.8.2 `public static final int COMMAND_FAILED = 500`

The recipient encountered an error which prevented it from fulfilling the request.

This error code is only used in situations not covered by any of the other error codes that a method may use. Some methods specify more specific error situations for this code, but it can generally be used for any unexpected condition that causes the command to fail.

This error code corresponds to the OMA DM response status code 500 "Command Failed".

117.14.8.3 `public static final int COMMAND_NOT_ALLOWED = 405`

The requested command is not allowed on the target node. This includes the following situations:

- an interior node operation is requested for a leaf node, or vice versa (e.g. trying to retrieve the children of a leaf node)
- an attempt is made to create a node where the parent is a leaf node
- an attempt is made to rename or delete the root node of the tree
- an attempt is made to rename or delete the root node of the session
- a write operation (other than setting the ACL) is performed in a non-atomic write session on a node provided by a plugin that is read-only or does not support non-atomic writing
- a node is copied to its descendant
- the ACL of the root node is changed not to include Add rights for all principals

This error code corresponds to the OMA DM response status code 405 "Command not allowed".

117.14.8.4 `public static final int CONCURRENT_ACCESS = 4`

An error occurred related to concurrent access of nodes. This can happen for example if a configuration node was deleted directly through the Configuration Admin service, while the node was manipulated via the tree.

This error code does not correspond to any OMA DM response status code. It should be translated to the code 500 "Command Failed" when transferring over OMA DM.

117.14.8.5 `public static final int DATA_STORE_FAILURE = 510`

An error related to the recipient data store occurred while processing the request. This error code may be thrown by any of the methods accessing the tree, but whether it is really used depends on the implementation, and the data store it uses.

This error code corresponds to the OMA DM response status code 510 "Data store failure".

117.14.8.6 `public static final int FEATURE_NOT_SUPPORTED = 406`

The requested command failed because an optional feature required by the command is not supported. For example, opening an atomic session might return this error code if the DmtAdmin im-

plementation does not support transactions. Similarly, accessing the optional node properties (Title, Timestamp, Version, Size) might not succeed if either the DmtAdmin implementation or the underlying plugin does not support the property.

When getting or setting values for interior nodes (an optional optimization feature), a plugin can use this error code to indicate that the given interior node does not support values.

This error code corresponds to the OMA DM response status code 406 "Optional feature not supported".

117.14.8.7 **public static final int INVALID_URI = 3**

The requested command failed because the target URI or node name is {@code null} or syntactically invalid. This covers the following cases:

- the URI or node name ends with the '\' or '/' character
- the URI is an empty string (only invalid if the method does not accept relative URIs)
- the URI contains the segment "{@code .}" at a position other than the beginning of the URI
- the node name is "{@code ..}" or the URI contains such a segment
- the node name contains an unescaped '/' character

See the {@link Uri#encode(String)} method for support on escaping invalid characters in a URI.

This code is only used if the URI or node name does not match any of the criteria for {@link #URI_TOO_LONG}. This error code does not correspond to any OMA DM response status code. It should be translated to the code 404 "Not Found" when transferring over OMA DM.

117.14.8.8 **public static final int LIMIT_EXCEEDED = 413**

The requested operation failed because a specific limit was exceeded, e.g. if a requested resource exceeds a size limit.

This error code corresponds to the OMA DM response status code 413 "Request entity too large".

Since @since 2.0

117.14.8.9 **public static final int METADATA_MISMATCH = 2**

Operation failed because of meta data restrictions. This covers any attempted deviation from the parameters defined by the {@code MetaNode} objects of the affected nodes, for example in the following situations:

- creating, deleting or renaming a permanent node, or modifying its type
- creating an interior node where the meta-node defines it as a leaf, or vice versa
- any operation on a node which does not have the required access type (e.g. executing a node that lacks the {@code MetaNode.CMD_EXECUTE} access type)
- any node creation or deletion that would violate the cardinality constraints
- any leaf node value setting that would violate the allowed formats, values, mime types, etc.
- any node creation that would violate the allowed node names

This error code can also be used to indicate any other meta data violation, even if it cannot be described by the {@code MetaNode} class. For example, detecting a multi-node constraint violation while committing an atomic session should result in this error.

This error code does not correspond to any OMA DM response status code. It should be translated to the code 405 "Command not allowed" when transferring over OMA DM.

117.14.8.10 public static final int NODE_ALREADY_EXISTS = 418

The requested node creation operation failed because the target already exists. This can occur if the node is created directly (with one of the {@code create...} methods), or indirectly (during a {@code copy} operation).

This error code corresponds to the OMA DM response status code 418 "Already exists".

117.14.8.11 public static final int NODE_NOT_FOUND = 404

The requested target node was not found. No indication is given as to whether this is a temporary or permanent condition, unless otherwise noted.

This is only used when the requested node name is valid, otherwise the more specific error codes {@link #URI_TOO_LONG} or {@link #INVALID_URI} are used. This error code corresponds to the OMA DM response status code 404 "Not Found".

117.14.8.12 public static final int PERMISSION_DENIED = 425

The requested command failed because the principal associated with the session does not have adequate access control permissions (ACL) on the target. This can only appear in case of remote sessions, i.e. if the session is associated with an authenticated principal.

This error code corresponds to the OMA DM response status code 425 "Permission denied".

117.14.8.13 public static final int REMOTE_ERROR = 1

A device initiated remote operation failed. This is used when the protocol adapter fails to send an alert for any reason.

Alert routing errors (that occur while looking for the proper protocol adapter to use) are indicated by {@link #ALERT_NOT_ROUTED}, this code is only for errors encountered while sending the routed alert. This error code does not correspond to any OMA DM response status code. It should be translated to the code 500 "Command Failed" when transferring over OMA DM.

117.14.8.14 public static final int ROLLBACK_FAILED = 516

The rollback command was not completed successfully. The tree might be in an inconsistent state after this error.

This error code corresponds to the OMA DM response status code 516 "Atomic roll back failed".

117.14.8.15 public static final int SESSION_CREATION_TIMEOUT = 7

Creation of a session timed out because of another ongoing session. The length of time while the DmtAdmin waits for the blocking session(s) to finish is implementation dependent.

This error code does not correspond to any OMA DM response status code. OMA has several status codes related to timeout, but these are meant to be used when a request times out, not if a session can not be established. This error code should be translated to the code 500 "Command Failed" when transferring over OMA DM.

117.14.8.16 public static final int TRANSACTION_ERROR = 6

A transaction-related error occurred in an atomic session. This error is caused by one of the following situations:

- an updating method within an atomic session can not be executed because the underlying plugin is read-only or does not support atomic writing
- a commit operation at the end of an atomic session failed because one of the underlying plugins failed to close

The latter case may leave the tree in an inconsistent state due to the lack of a two-phase commit system, see {@link DmtSession#commit()} for details.

This error code does not correspond to any OMA DM response status code. It should be translated to the code 500 "Command Failed" when transferring over OMA DM.

117.14.8.17 **public static final int UNAUTHORIZED = 401**

The originator's authentication credentials specify a principal with insufficient rights to complete the command.

This status code is used as response to device originated sessions if the remote management server cannot authorize the device to perform the requested operation.

This error code corresponds to the OMA DM response status code 401 "Unauthorized".

117.14.8.18 **public static final int URI_TOO_LONG = 414**

The requested command failed because the target URI is too long for what the recipient is able or willing to process.

This error code corresponds to the OMA DM response status code 414 "URI too long".

See Also OSGi Service Platform, Mobile Specification Release 4

117.14.8.19 **public DmtException(String uri, int code, String message)**

uri the node on which the failed DMT operation was issued, or {@code null} if the operation is not associated with a node

code the error code of the failure

message the message associated with the exception, or {@code null} if there is no error message

- Create an instance of the exception. The {@code uri} and {@code message} parameters are optional. No originating exception is specified.

117.14.8.20 **public DmtException(String uri, int code, String message, Throwable cause)**

uri the node on which the failed DMT operation was issued, or {@code null} if the operation is not associated with a node

code the error code of the failure

message the message associated with the exception, or {@code null} if there is no error message

cause the originating exception, or {@code null} if there is no originating exception

- Create an instance of the exception, specifying the cause exception. The {@code uri}, {@code message} and {@code cause} parameters are optional.

117.14.8.21 **public DmtException(String uri, int code, String message, Vector<? extends Throwable> causes, boolean fatal)**

uri the node on which the failed DMT operation was issued, or {@code null} if the operation is not associated with a node

code the error code of the failure

message the message associated with the exception, or {@code null} if there is no error message

causes the list of originating exceptions, or empty list or {@code null} if there are no originating exceptions

fatal whether the exception is fatal

- Create an instance of the exception, specifying the list of cause exceptions and whether the exception is a fatal one. This constructor is meant to be used by plugins wishing to indicate that a serious error occurred which should invalidate the ongoing atomic session. The {@code uri}, {@code message} and {@code causes} parameters are optional.

If a fatal exception is thrown, no further business methods will be called on the originator plugin. In case of atomic sessions, all other open plugins will be rolled back automatically, except if the fatal exception was thrown during commit.

117.14.8.22 `public DmtException(String[][] path, int code, String message)`

path the path of the node on which the failed DMT operation was issued, or {@code null} if the operation is not associated with a node

code the error code of the failure

message the message associated with the exception, or {@code null} if there is no error message

- Create an instance of the exception, specifying the target node as an array of path segments. This method behaves in exactly the same way as if the path was given as a URI string.

See Also `DmtException(String, int, String)`

117.14.8.23 `public DmtException(String[][] path, int code, String message, Throwable cause)`

path the path of the node on which the failed DMT operation was issued, or {@code null} if the operation is not associated with a node

code the error code of the failure

message the message associated with the exception, or {@code null} if there is no error message

cause the originating exception, or {@code null} if there is no originating exception

- Create an instance of the exception, specifying the target node as an array of path segments, and specifying the cause exception. This method behaves in exactly the same way as if the path was given as a URI string.

See Also `DmtException(String, int, String, Throwable)`

117.14.8.24 `public DmtException(String[][] path, int code, String message, Vector<? extends Throwable> causes, boolean fatal)`

path the path of the node on which the failed DMT operation was issued, or {@code null} if the operation is not associated with a node

code the error code of the failure

message the message associated with the exception, or {@code null} if there is no error message

causes the list of originating exceptions, or empty list or {@code null} if there are no originating exceptions

fatal whether the exception is fatal

- Create an instance of the exception, specifying the target node as an array of path segments, the list of cause exceptions, and whether the exception is a fatal one. This method behaves in exactly the same way as if the path was given as a URI string.

See Also `DmtException(String, int, String, Vector, boolean)`

117.14.8.25 `public Throwable getCause()`

- Get the cause of this exception. Returns non-{@code null}, if this exception is caused by one or more other exceptions (like a {@code NullPointerException} in a `DmtPlugin`). If there are more than one cause exceptions, the first one is returned.

Returns the cause of this exception, or {@code null} if no cause was given

117.14.8.26 `public Throwable[][] getCauses()`

- Get all causes of this exception. Returns the causing exceptions in an array. If no cause was specified, an empty array is returned.

Returns the list of causes of this exception

117.14.8.27 **public int getCode()**

- Get the error code associated with this exception. Most of the error codes within this exception correspond to OMA DM error codes.

Returns the error code

117.14.8.28 **public String getMessage()**

- Get the message associated with this exception. The returned string also contains the associated URI (if any) and the exception code. The resulting message has the following format (parts in square brackets are only included if the field inside them is not {@code null}):

```
<exception_code>[: '<uri>'][: <error_message>]
```

Returns the error message in the format described above

117.14.8.29 **public String getURI()**

- Get the node on which the failed DMT operation was issued. Some operations like {@code DmtSession.close()} don't require an URI, in this case this method returns {@code null}.

Returns the URI of the node, or {@code null}

117.14.8.30 **public boolean isFatal()**

- Check whether this exception is marked as fatal in the session. Fatal exceptions trigger an automatic rollback of atomic sessions.

Returns whether the exception is marked as fatal

117.14.8.31 **public void printStackTrace(PrintStream s)**

s {@code PrintStream} to use for output

- Prints the exception and its stacktrace to the specified print stream. Any causes that were specified for this exception are also printed, together with their stacktraces.

117.14.9 **public class DmtIllegalStateException extends RuntimeException**

Unchecked illegal state exception. This class is used in DMT because java.lang.IllegalStateException does not exist in CLDC.

117.14.9.1 **public DmtIllegalStateException()**

- Create an instance of the exception with no message.

117.14.9.2 **public DmtIllegalStateException(String message)**

message the reason for the exception

- Create an instance of the exception with the specified message.

117.14.9.3 **public DmtIllegalStateException(Throwable cause)**

cause the cause of the exception

- Create an instance of the exception with the specified cause exception and no message.

117.14.9.4 **public DmtIllegalStateException(String message, Throwable cause)**

message the reason for the exception

cause the cause of the exception

- Create an instance of the exception with the specified message and cause exception.

117.14.10 public abstract DmtSession

DmtSession provides concurrent access to the DMT. All DMT manipulation commands for management applications are available on the {@code DmtSession} interface. The session is associated with a root node which limits the subtree in which the operations can be executed within this session.

Most of the operations take a node URI as parameter, which can be either an absolute URI (starting with "/") or a URI relative to the root node of the session. The empty string as relative URI means the root URI the session was opened with. All segments of a URI must be within the segment length limit of the implementation, and the special characters "/" and "\" must be escaped (preceded by a "\").

See the {@link Uri#encode(String)} method for support on escaping invalid characters in a URI.

If the URI specified does not correspond to a legitimate node in the tree an exception is thrown. The only exception is the {@link #isNodeUri(String)} method which returns {@code false} in case of an invalid URI.

Each method of {@code DmtSession} that accesses the tree in any way can throw {@code DmtIllegalStateException} if the session has been closed or invalidated (due to timeout, fatal exceptions, or unexpectedly unregistered plugins).

117.14.10.1 public static final int LOCK_TYPE_ATOMIC = 2

{@code LOCK_TYPE_ATOMIC} is an exclusive lock with transactional functionality. Commands of an atomic session will either fail or succeed together, if a single command fails then the whole session will be rolled back.

117.14.10.2 public static final int LOCK_TYPE_EXCLUSIVE = 1

{@code LOCK_TYPE_EXCLUSIVE} lock guarantees full access to the tree, but can not be shared with any other locks.

117.14.10.3 public static final int LOCK_TYPE_SHARED = 0

Sessions created with {@code LOCK_TYPE_SHARED} lock allows read-only access to the tree, but can be shared between multiple readers.

117.14.10.4 public static final int STATE_CLOSED = 1

The session is closed, DMT manipulation operations are not available, they throw {@code DmtIllegalStateException} if tried.

117.14.10.5 public static final int STATE_INVALID = 2

The session is invalid because a fatal error happened. Fatal errors include the timeout of the session, any DmtException with the 'fatal' flag set, or the case when a plugin service is unregistered while in use by the session. DMT manipulation operations are not available, they throw {@code DmtIllegalStateException} if tried.

117.14.10.6 public static final int STATE_OPEN = 0

The session is open, all session operations are available.

117.14.10.7 public abstract void close() throws DmtException

- Closes a session. If the session was opened with atomic lock mode, the {@code DmtSession} must first persist the changes made to the DMT by calling {@code commit()} on all (transactional) plugins participating in the session. See the documentation of the {@code commit()} method for details and possible errors during this operation.

The state of the session changes to {@code DmtSession.STATE_CLOSED} if the close operation completed successfully, otherwise it becomes {@code DmtSession.STATE_INVALID}.

Throws `DmtException`– with the following possible error codes:

- `{@code METADATA_MISMATCH}` in case of atomic sessions, if the commit operation failed because of meta-data restrictions
- `{@code CONCURRENT_ACCESS}` in case of atomic sessions, if the commit operation failed because of some modification outside the scope of the DMT to the nodes affected in the session
- `{@code TRANSACTION_ERROR}` in case of atomic sessions, if an underlying plugin failed to commit
- `{@code DATA_STORE_FAILURE}` if an error occurred while accessing the data store
- `{@code COMMAND_FAILED}` if an underlying plugin failed to close, or if some unspecified error is encountered while attempting to complete the command

`DmtIllegalStateException`– if the session is already closed or invalidated

`SecurityException`– if the caller does not have the necessary permissions to execute the underlying management operation

117.14.10.8 **public abstract void commit() throws DmtException**

- Commits a series of DMT operations issued in the current atomic session since the last transaction boundary. Transaction boundaries are the creation of this object that starts the session, and all subsequent `{@link #commit()}` and `{@link #rollback()}` calls.

This method can fail even if all operations were successful. This can happen due to some multi-node semantic constraints defined by a specific implementation. For example, node A can be required to always have children A/B, A/C and A/D. If this condition is broken when `{@code commit()}` is executed, the method will fail, and throw a `{@code METADATA_MISMATCH}` exception.

An error situation can arise due to the lack of a two phase commit mechanism in the underlying plugins. As an example, if plugin A has committed successfully but plugin B failed, the whole session must fail, but there is no way to undo the commit performed by A. To provide predictable behavior, the commit operation should continue with the remaining plugins even after detecting a failure. All exceptions received from failed commits are aggregated into one `{@code TRANSACTION_ERROR}` exception thrown by this method.

In many cases the tree is not the only way to manage a given part of the system. It may happen that while modifying some nodes in an atomic session, the underlying settings are modified in parallel outside the scope of the DMT. If this is detected during commit, an exception with the code `{@code CONCURRENT_ACCESS}` is thrown.

Throws `DmtException`– with the following possible error codes:

- `{@code METADATA_MISMATCH}` if the operation failed because of meta-data restrictions
- `{@code CONCURRENT_ACCESS}` if it is detected that some modification has been made outside the scope of the DMT to the nodes affected in the session's operations
- `{@code TRANSACTION_ERROR}` if an error occurred during the commit of any of the underlying plugins
- `{@code DATA_STORE_FAILURE}` if an error occurred while accessing the data store
- `{@code COMMAND_FAILED}` if some unspecified error is encountered while attempting to complete the command

`DmtIllegalStateException`– if the session was not opened using the `{@code LOCK_TYPE_ATOMIC}` lock type, or if the session is already closed or invalidated

`SecurityException`– if the caller does not have the necessary permissions to execute the underlying management operation

117.14.10.9 **public abstract void copy(String nodeUri, String newNodeUri, boolean recursive) throws DmtException**

nodeUri the node or root of a subtree to be copied

newNodeUri the URI of the new node or root of a subtree

recursive {@code false} if only a single node is copied, {@code true} if the whole subtree is copied

- Create a copy of a node or a whole subtree. Beside the structure and values of the nodes, most properties are also copied, with the exception of the ACL (Access Control List), Timestamp and Version properties.

The copy method is essentially a convenience method that could be substituted with a sequence of retrieval and update operations. This determines the permissions required for copying. However, some optimization can be possible if the source and target nodes are all handled by DmtAdmin or by the same plugin. In this case, the handler might be able to perform the underlying management operation more efficiently: for example, a configuration table can be copied at once instead of reading each node for each entry and creating it in the new tree.

This method may result in any of the errors possible for the contributing operations. Most of these are collected in the exception descriptions below, but for the full list also consult the documentation of {@link #getChildNodeNames(String)}, {@link #isLeafNode(String)}, {@link #getNodeValue(String)}, {@link #getNodeTypes(String)}, {@link #getNodeTitle(String)}, {@link #setNodeTitle(String, String)}, {@link #createLeafNode(String, DmtData, String)} and {@link #createInteriorNode(String, String)}.

Throws DmtException– with the following possible error codes:

- {@code INVALID_URI} if {@code nodeUri} or {@code newNodeUri} is {@code null} or syntactically invalid
- {@code URI_TOO_LONG} if {@code nodeUri} is longer than accepted by the {@code DmtAdmin} implementation (especially on systems with limited resources)
- {@code NODE_NOT_FOUND} if {@code nodeUri} points to a non-existing node, or if {@code newNodeUri} points to a node that cannot exist in the tree according to the meta-data (see {@link #getMetaNode(String)})
- {@code NODE_ALREADY_EXISTS} if {@code newNodeUri} points to a node that already exists
- {@code PERMISSION_DENIED} if the session is associated with a principal and the ACL of the copied node(s) does not allow the {@code Get} operation, or the ACL of the parent of the target node does not allow the {@code Add} operation for the associated principal
- {@code COMMAND_NOT_ALLOWED} if {@code nodeUri} is an ancestor of {@code newNodeUri}, or if any of the implied retrieval or update operations are not allowed
- {@code METADATA_MISMATCH} if any of the meta-data constraints of the implied retrieval or update operations are violated
- {@code TRANSACTION_ERROR} in an atomic session if the underlying plugin is read-only or does not support atomic writing
- {@code DATA_STORE_FAILURE} if an error occurred while accessing the data store
- {@code COMMAND_FAILED} if either URI is not within the current session's subtree, or if some unspecified error is encountered while attempting to complete the command

DmtIllegalStateException– if the session was opened using the {@code LOCK_TYPE_SHARED} lock type, or if the session is already closed or invalidated

SecurityException– if the caller does not have the necessary permissions to execute the underlying management operation, or, in case of local sessions, if the caller does not have {@code DmtPermission} for the copied node(s) with the Get action present, or for the parent of the target node with the Add action

117.14.10.10 **public abstract void createInteriorNode(String nodeUri) throws DmtException**

nodeUri the URI of the node to create

- Create an interior node. If the parent node does not exist, it is created automatically, as if this method were called for the parent URI. This way all missing ancestor nodes leading to the specified node are created. Any exceptions encountered while creating the ancestors are propagated to the caller of this method, these are not explicitly listed in the error descriptions below.

If meta-data is available for the node, several checks are made before creating it. The node must have {@code MetaNode.CMD_ADD} access type, it must be defined as a non-permanent interior node, the node name must conform to the valid names, and the creation of the new node must not cause the maximum occurrence number to be exceeded.

If the meta-data cannot be retrieved because the given node cannot possibly exist in the tree (it is not defined in the specification), the {@code NODE_NOT_FOUND} error code is returned (see {@link #getMetaNode(String)}).

Throws `DmtException` – with the following possible error codes:

- {@code INVALID_URI} if {@code nodeUri} is {@code null} or syntactically invalid
- {@code URI_TOO_LONG} if {@code nodeUri} is longer than accepted by the {@code DmtAdmin} implementation (especially on systems with limited resources)
- {@code NODE_NOT_FOUND} if {@code nodeUri} points to a node that cannot exist in the tree (see above)
- {@code NODE_ALREADY_EXISTS} if {@code nodeUri} points to a node that already exists
- {@code PERMISSION_DENIED} if the session is associated with a principal and the ACL of the parent node does not allow the {@code Add} operation for the associated principal
- {@code COMMAND_NOT_ALLOWED} if the parent node is not an interior node, or in non-atomic sessions if the underlying plugin is read-only or does not support non-atomic writing
- {@code METADATA_MISMATCH} if the node could not be created because of meta-data restrictions (see above)
- {@code TRANSACTION_ERROR} in an atomic session if the underlying plugin is read-only or does not support atomic writing
- {@code DATA_STORE_FAILURE} if an error occurred while accessing the data store
- {@code COMMAND_FAILED} if the URI is not within the current session's subtree, or if some unspecified error is encountered while attempting to complete the command

`DmtIllegalStateException` – if the session was opened using the {@code LOCK_TYPE_SHARED} lock type, or if the session is already closed or invalidated

`SecurityException` – if the caller does not have the necessary permissions to execute the underlying management operation, or, in case of local sessions, if the caller does not have {@code DmtPermission} for the parent node with the Add action present

117.14.10.11 **public abstract void createInteriorNode(String nodeUri, String type) throws DmtException**

nodeUri the URI of the node to create

type the type URI of the interior node, can be {@code null} if no node type is defined

- Create an interior node with a given type. The type of interior node, if specified, is a URI identifying a DDF document. If the parent node does not exist, it is created automatically, as if {@link #createInteriorNode(String)} were called for the parent URI. This way all missing ancestor nodes leading to the specified node are created. Any exceptions encountered while creating the ancestors are propagated to the caller of this method, these are not explicitly listed in the error descriptions below.

If meta-data is available for the node, several checks are made before creating it. The node must have {@code MetaNode.CMD_ADD} access type, it must be defined as a non-permanent interior node, the node name must conform to the valid names, and the creation of the new node must not cause the maximum occurrence number to be exceeded.

If the meta-data cannot be retrieved because the given node cannot possibly exist in the tree (it is not defined in the specification), the {@code NODE_NOT_FOUND} error code is returned (see {@link #getMetaNode(String)}).

Interior node type identifiers must follow the format defined in section 7.7.7.2 of the OMA Device Management Tree and Description document. Checking the validity of the type string does not have to be done by the DmtAdmin, this can be left to the plugin handling the node (if any), to avoid unnecessary double-checks.

Throws DmtException– with the following possible error codes:

- {@code INVALID_URI} if {@code nodeUri} is {@code null} or syntactically invalid
- {@code URI_TOO_LONG} if {@code nodeUri} is longer than accepted by the {@code DmtAdmin} implementation (especially on systems with limited resources)
- {@code NODE_NOT_FOUND} if {@code nodeUri} points to a node that cannot exist in the tree (see above)
- {@code NODE_ALREADY_EXISTS} if {@code nodeUri} points to a node that already exists
- {@code PERMISSION_DENIED} if the session is associated with a principal and the ACL of the parent node does not allow the {@code Add} operation for the associated principal
- {@code COMMAND_NOT_ALLOWED} if the parent node is not an interior node, or in non-atomic sessions if the underlying plugin is read-only or does not support non-atomic writing
- {@code METADATA_MISMATCH} if the node could not be created because of meta-data restrictions (see above)
- {@code TRANSACTION_ERROR} in an atomic session if the underlying plugin is read-only or does not support atomic writing
- {@code DATA_STORE_FAILURE} if an error occurred while accessing the data store
- {@code COMMAND_FAILED} if the URI is not within the current session's subtree, if the type string is invalid (see above), or if some unspecified error is encountered while attempting to complete the command

DmtIllegalStateException– if the session was opened using the {@code LOCK_TYPE_SHARED} lock type, or if the session is already closed or invalidated

SecurityException– if the caller does not have the necessary permissions to execute the underlying management operation, or, in case of local sessions, if the caller does not have {@code DmtPermission} for the parent node with the Add action present

See Also createInteriorNode(String), OMA Device Management Tree and Description v1.2 draft [http://member.openmobilealliance.org/ftp/public_documents/dm/Permanent_documents/OMA-TS-DM-TND-V1_2-20050615-C.zip]

117.14.10.12 public abstract void createLeafNode(String nodeUri) throws DmtException

nodeUri the URI of the node to create

- Create a leaf node with default value and MIME type. If a node does not have a default value or MIME type, this method will throw a {@code DmtException} with error code {@code METADATA_MISMATCH}. Note that a node might have a default value or MIME type even if there is no meta-data for the node or its meta-data does not specify the default.

If the parent node does not exist, it is created automatically, as if {@link #createInteriorNode(String)} were called for the parent URI. This way all missing ancestor nodes leading to the specified node are created. Any exceptions encountered while creating the ancestors are propagated to the caller of this method, these are not explicitly listed in the error descriptions below.

If meta-data is available for a node, several checks are made before creating it. The node must have {@code MetaNode.CMD_ADD} access type, it must be defined as a non-permanent leaf node, the

node name must conform to the valid names, and the creation of the new node must not cause the maximum occurrence number to be exceeded.

If the meta-data cannot be retrieved because the given node cannot possibly exist in the tree (it is not defined in the specification), the {@code NODE_NOT_FOUND} error code is returned (see {@link #getMetaNode(String)}).

Throws `DmtException` – with the following possible error codes:

- {@code INVALID_URI} if {@code nodeUri} is {@code null} or syntactically invalid
- {@code URI_TOO_LONG} if {@code nodeUri} is longer than accepted by the {@code DmtAdmin} implementation (especially on systems with limited resources)
- {@code NODE_NOT_FOUND} if {@code nodeUri} points to a node that cannot exist in the tree (see above)
- {@code NODE_ALREADY_EXISTS} if {@code nodeUri} points to a node that already exists
- {@code PERMISSION_DENIED} if the session is associated with a principal and the ACL of the parent node does not allow the {@code Add} operation for the associated principal
- {@code COMMAND_NOT_ALLOWED} if the parent node is not an interior node, or in non-atomic sessions if the underlying plugin is read-only or does not support non-atomic writing
- {@code METADATA_MISMATCH} if the node could not be created because of meta-data restrictions (see above)
- {@code TRANSACTION_ERROR} in an atomic session if the underlying plugin is read-only or does not support atomic writing
- {@code DATA_STORE_FAILURE} if an error occurred while accessing the data store
- {@code COMMAND_FAILED} if the URI is not within the current session's subtree, or if some unspecified error is encountered while attempting to complete the command

`DmtIllegalStateException` – if the session was opened using the {@code LOCK_TYPE_SHARED} lock type, or if the session is already closed or invalidated

`SecurityException` – if the caller does not have the necessary permissions to execute the underlying management operation, or, in case of local sessions, if the caller does not have {@code DmtPermission} for the parent node with the Add action present

See Also `createLeafNode(String, DmtData)`

117.14.10.13 **public abstract void createLeafNode(String nodeUri, DmtData value) throws DmtException**

nodeUri the URI of the node to create

value the value to be given to the new node, can be {@code null}

- Create a leaf node with a given value and the default MIME type. If the specified value is {@code null}, the default value is taken. If the node does not have a default MIME type or value (if needed), this method will throw a {@code DmtException} with error code {@code METADATA_MISMATCH}. Note that a node might have a default value or MIME type even if there is no meta-data for the node or its meta-data does not specify the default.

If the parent node does not exist, it is created automatically, as if {@link #createInteriorNode(String)} were called for the parent URI. This way all missing ancestor nodes leading to the specified node are created. Any exceptions encountered while creating the ancestors are propagated to the caller of this method, these are not explicitly listed in the error descriptions below.

If meta-data is available for a node, several checks are made before creating it. The node must have {@code MetaNode.CMD_ADD} access type, it must be defined as a non-permanent leaf node, the node name must conform to the valid names, the node value must conform to the value constraints, and the creation of the new node must not cause the maximum occurrence number to be exceeded.

If the meta-data cannot be retrieved because the given node cannot possibly exist in the tree (it is not defined in the specification), the {@code NODE_NOT_FOUND} error code is returned (see {@link #getMetaNode(String)}).

Nodes of {@code null} format can be created by using {@link DmtData#NULL_VALUE} as second argument.

Throws `DmtException`– with the following possible error codes:

- {@code INVALID_URI} if {@code nodeUri} is {@code null} or syntactically invalid
- {@code URI_TOO_LONG} if {@code nodeUri} is longer than accepted by the {@code DmtAdmin} implementation (especially on systems with limited resources)
- {@code NODE_NOT_FOUND} if {@code nodeUri} points to a node that cannot exist in the tree (see above)
- {@code NODE_ALREADY_EXISTS} if {@code nodeUri} points to a node that already exists
- {@code PERMISSION_DENIED} if the session is associated with a principal and the ACL of the parent node does not allow the {@code Add} operation for the associated principal
- {@code COMMAND_NOT_ALLOWED} if the parent node is not an interior node, or in non-atomic sessions if the underlying plugin is read-only or does not support non-atomic writing
- {@code METADATA_MISMATCH} if the node could not be created because of meta-data restrictions (see above)
- {@code TRANSACTION_ERROR} in an atomic session if the underlying plugin is read-only or does not support atomic writing
- {@code DATA_STORE_FAILURE} if an error occurred while accessing the data store
- {@code COMMAND_FAILED} if the URI is not within the current session's subtree, or if some unspecified error is encountered while attempting to complete the command

`DmtIllegalStateException`– if the session was opened using the {@code LOCK_TYPE_SHARED} lock type, or if the session is already closed or invalidated

`SecurityException`– if the caller does not have the necessary permissions to execute the underlying management operation, or, in case of local sessions, if the caller does not have {@code DmtPermission} for the parent node with the Add action present

117.14.10.14 **public abstract void createLeafNode(String nodeUri, DmtData value, String mimeType) throws DmtException**

nodeUri the URI of the node to create

value the value to be given to the new node, can be {@code null}

mimeType the MIME type to be given to the new node, can be {@code null}

- Create a leaf node with a given value and MIME type. If the specified value or MIME type is {@code null}, their default values are taken. If the node does not have the necessary defaults, this method will throw a {@code DmtException} with error code {@code METADATA_MISMATCH}. Note that a node might have a default value or MIME type even if there is no meta-data for the node or its meta-data does not specify the default.

If the parent node does not exist, it is created automatically, as if {@link #createInteriorNode(String)} were called for the parent URI. This way all missing ancestor nodes leading to the specified node are created. Any exceptions encountered while creating the ancestors are propagated to the caller of this method, these are not explicitly listed in the error descriptions below.

If meta-data is available for a node, several checks are made before creating it. The node must have {@code MetaNode.CMD_ADD} access type, it must be defined as a non-permanent leaf node, the node name must conform to the valid names, the node value must conform to the value constraints, the MIME type must be among the listed types, and the creation of the new node must not cause the maximum occurrence number to be exceeded.

If the meta-data cannot be retrieved because the given node cannot possibly exist in the tree (it is not defined in the specification), the {@code NODE_NOT_FOUND} error code is returned (see {@code #getMetaNode(String)}).

Nodes of {@code null} format can be created by using {@code DmtData#NULL_VALUE} as second argument.

The MIME type string must conform to the definition in RFC 2045. Checking its validity does not have to be done by the DmtAdmin, this can be left to the plugin handling the node (if any), to avoid unnecessary double-checks.

Throws DmtException– with the following possible error codes:

- {@code INVALID_URI} if {@code nodeUri} is {@code null} or syntactically invalid
- {@code URI_TOO_LONG} if {@code nodeUri} is longer than accepted by the {@code DmtAdmin} implementation (especially on systems with limited resources)
- {@code NODE_NOT_FOUND} if {@code nodeUri} points to a node that cannot exist in the tree (see above)
- {@code NODE_ALREADY_EXISTS} if {@code nodeUri} points to a node that already exists
- {@code PERMISSION_DENIED} if the session is associated with a principal and the ACL of the parent node does not allow the {@code Add} operation for the associated principal
- {@code COMMAND_NOT_ALLOWED} if the parent node is not an interior node, or in non-atomic sessions if the underlying plugin is read-only or does not support non-atomic writing
- {@code METADATA_MISMATCH} if the node could not be created because of meta-data restrictions (see above)
- {@code TRANSACTION_ERROR} in an atomic session if the underlying plugin is read-only or does not support atomic writing
- {@code DATA_STORE_FAILURE} if an error occurred while accessing the data store
- {@code COMMAND_FAILED} if the URI is not within the current session's subtree, if {@code mimeType} is not a proper MIME type string (see above), or if some unspecified error is encountered while attempting to complete the command

DmtIllegalStateException– if the session was opened using the {@code LOCK_TYPE_SHARED} lock type, or if the session is already closed or invalidated

SecurityException– if the caller does not have the necessary permissions to execute the underlying management operation, or, in case of local sessions, if the caller does not have {@code DmtPermission} for the parent node with the Add action present

See Also createLeafNode(String, DmtData), RFC 2045 [http://www.ietf.org/rfc/rfc2045.txt]

117.14.10.15 **public abstract void deleteNode(String nodeUri) throws DmtException**

nodeUri the URI of the node

- Delete the given node. Deleting interior nodes is recursive, the whole subtree under the given node is deleted. It is not allowed to delete the root node of the session.

If meta-data is available for a node, several checks are made before deleting it. The node must be non-permanent, it must have the {@code MetaNode.CMD_DELETE} access type, and if zero occurrences of the node are not allowed, it must not be the last one.

Throws DmtException– with the following possible error codes:

- {@code INVALID_URI} if {@code nodeUri} is {@code null} or syntactically invalid
- {@code URI_TOO_LONG} if {@code nodeUri} is longer than accepted by the {@code DmtAdmin} implementation (especially on systems with limited resources)
- {@code NODE_NOT_FOUND} if {@code nodeUri} points to a non-existing node

- {@code PERMISSION_DENIED} if the session is associated with a principal and the ACL of the node does not allow the {@code Delete} operation for the associated principal
- {@code COMMAND_NOT_ALLOWED} if the target node is the root of the session, or in non-atomic sessions if the underlying plugin is read-only or does not support non-atomic writing
- {@code METADATA_MISMATCH} if the node could not be deleted because of meta-data restrictions (see above)
- {@code TRANSACTION_ERROR} in an atomic session if the underlying plugin is read-only or does not support atomic writing
- {@code DATA_STORE_FAILURE} if an error occurred while accessing the data store
- {@code COMMAND_FAILED} if the URI is not within the current session's subtree, or if some unspecified error is encountered while attempting to complete the command

DmtIllegalStateException– if the session was opened using the {@code LOCK_TYPE_SHARED} lock type, or if the session is already closed or invalidated

SecurityException– if the caller does not have the necessary permissions to execute the underlying management operation, or, in case of local sessions, if the caller does not have {@code DmtPermission} for the node with the Delete action present

117.14.10.16 **public abstract void execute(String nodeUri, String data) throws DmtException**

nodeUri the node on which the execute operation is issued

data the parameter of the execute operation, can be {@code null}

- Executes a node. This corresponds to the EXEC operation in OMA DM. This method cannot be called in a read-only session.

The semantics of an execute operation and the data parameter it takes depends on the definition of the managed object on which the command is issued.

Throws **DmtException**– with the following possible error codes:

- {@code INVALID_URI} if {@code nodeUri} is {@code null} or syntactically invalid
- {@code URI_TOO_LONG} if {@code nodeUri} is longer than accepted by the {@code DmtAdmin} implementation (especially on systems with limited resources)
- {@code NODE_NOT_FOUND} if the node does not exist
- {@code PERMISSION_DENIED} if the session is associated with a principal and the ACL of the node does not allow the {@code Execute} operation for the associated principal
- {@code COMMAND_NOT_ALLOWED} if the specified node is a scaffold node
- {@code METADATA_MISMATCH} if the node cannot be executed according to the meta-data (does not have {@code MetaNode.CMD_EXECUTE} access type)
- {@code DATA_STORE_FAILURE} if an error occurred while accessing the data store
- {@code COMMAND_FAILED} if the URI is not within the current session's subtree, if no **DmtExecPlugin** is associated with the node and the **DmtAdmin** can not execute the node, or if some unspecified error is encountered while attempting to complete the command

DmtIllegalStateException– if the session was opened using the {@code LOCK_TYPE_SHARED} lock type, or if the session is already closed or invalidated

SecurityException– if the caller does not have the necessary permissions to execute the underlying management operation, or, in case of local sessions, if the caller does not have {@code DmtPermission} for the node with the Exec action present

See Also `execute(String, String, String)`

117.14.10.17 **public abstract void execute(String nodeUri, String correlator, String data) throws DmtException**

nodeUri the node on which the execute operation is issued

correlator an identifier to associate this operation with any notifications sent in response to it, can be {@code null} if not needed

data the parameter of the execute operation, can be {@code null}

- Executes a node, also specifying a correlation ID for use in response notifications. This operation corresponds to the EXEC command in OMA DM. This method cannot be called in a read-only session.

The semantics of an execute operation and the data parameter it takes depends on the definition of the managed object on which the command is issued. If a correlation ID is specified, it should be used as the {@code correlator} parameter for notifications sent in response to this execute operation.

Throws `DmtException`– with the following possible error codes:

- {@code INVALID_URI} if {@code nodeUri} is {@code null} or syntactically invalid
- {@code URI_TOO_LONG} if {@code nodeUri} is longer than accepted by the {@code DmtAdmin} implementation (especially on systems with limited resources)
- {@code NODE_NOT_FOUND} if the node does not exist
- {@code PERMISSION_DENIED} if the session is associated with a principal and the ACL of the node does not allow the {@code Execute} operation for the associated principal
- {@code COMMAND_NOT_ALLOWED} if the specified node is a scaffold node
- {@code METADATA_MISMATCH} if the node cannot be executed according to the meta-data (does not have {@code MetaNode.CMD_EXECUTE} access type)
- {@code DATA_STORE_FAILURE} if an error occurred while accessing the data store
- {@code COMMAND_FAILED} if the URI is not within the current session's subtree, if no `DmtExecPlugin` is associated with the node, or if some unspecified error is encountered while attempting to complete the command

`DmtIllegalStateException`– if the session was opened using the {@code LOCK_TYPE_SHARED} lock type, or if the session is already closed or invalidated

`SecurityException`– if the caller does not have the necessary permissions to execute the underlying management operation, or, in case of local sessions, if the caller does not have {@code DmtPermission} for the node with the Exec action present

See Also `execute(String, String)`

117.14.10.18 **public abstract String[][] getChildNodeNames(String nodeUri) throws DmtException**

nodeUri the URI of the node

- Get the list of children names of a node. The returned array contains the names - not the URIs - of the immediate children nodes of the given node. The elements are in no particular order. The returned array must not contain {@code null} entries.

Returns the list of child node names as a string array or an empty string array if the node has no children

Throws `DmtException`– with the following possible error codes:

- {@code INVALID_URI} if {@code nodeUri} is {@code null} or syntactically invalid
- {@code URI_TOO_LONG} if {@code nodeUri} is longer than accepted by the {@code DmtAdmin} implementation (especially on systems with limited resources)
- {@code NODE_NOT_FOUND} if {@code nodeUri} points to a non-existing node
- {@code PERMISSION_DENIED} if the session is associated with a principal and the ACL of the node does not allow the {@code Get} operation for the associated principal
- {@code COMMAND_NOT_ALLOWED} if the specified node is not an interior node

- {@code METADATA_MISMATCH} if node information cannot be retrieved according to the meta-data (it does not have {@code MetaNode.CMD_GET} access type)
- {@code DATA_STORE_FAILURE} if an error occurred while accessing the data store
- {@code COMMAND_FAILED} if the URI is not within the current session's subtree, or if some unspecified error is encountered while attempting to complete the command

DmtIllegalStateException– if the session is already closed or invalidated

SecurityException– if the caller does not have the necessary permissions to execute the underlying management operation, or, in case of local sessions, if the caller does not have {@code DmtPermission} for the node with the Get action present

117.14.10.19 **public abstract Acl getEffectiveNodeAcl(String nodeUri) throws DmtException**

nodeUri the URI of the node

- Gives the Access Control List in effect for a given node. The returned {@code Acl} takes inheritance into account, that is if there is no ACL defined for the node, it will be derived from the closest ancestor having an ACL defined.

Returns the Access Control List belonging to the node

Throws DmtException– with the following possible error codes:

- {@code INVALID_URI} if {@code nodeUri} is {@code null} or syntactically invalid
- {@code URI_TOO_LONG} if {@code nodeUri} is longer than accepted by the {@code DmtAdmin} implementation (especially on systems with limited resources)
- {@code NODE_NOT_FOUND} if {@code nodeUri} points to a non-existing node
- {@code PERMISSION_DENIED} if the session is associated with a principal and the ACL of the node does not allow the {@code Get} operation for the associated principal
- {@code METADATA_MISMATCH} if node information cannot be retrieved according to the meta-data (the node does not have {@code MetaNode.CMD_GET} access type)
- {@code DATA_STORE_FAILURE} if an error occurred while accessing the data store
- {@code COMMAND_FAILED} if the URI is not within the current session's subtree, or if some unspecified error is encountered while attempting to complete the command

DmtIllegalStateException– if the session is already closed or invalidated

SecurityException– in case of local sessions, if the caller does not have {@code DmtPermission} for the node with the Get action present

See Also `getNodeAcl(String)`

117.14.10.20 **public abstract int getLockType()**

- Gives the type of lock the session has.

Returns the lock type of the session, one of {@link #LOCK_TYPE_SHARED}, {@link #LOCK_TYPE_EXCLUSIVE} and {@link #LOCK_TYPE_ATOMIC}

117.14.10.21 **public abstract MetaNode getMetaNode(String nodeUri) throws DmtException**

nodeUri the URI of the node

- Get the meta data which describes a given node. Meta data can only be inspected, it can not be changed.

The {@code MetaNode} object returned to the client is the combination of the meta data returned by the data plugin (if any) plus the meta data returned by the DmtAdmin. If there are differences in the meta data elements known by the plugin and the DmtAdmin then the plugin specific elements take precedence.

Note, that a node does not have to exist for having meta-data associated with it. This method may provide meta-data for any node that can possibly exist in the tree (any node defined in the specification). For nodes that are not defined, it may throw {@code DmtException} with the error code {@code NODE_NOT_FOUND}. To allow easier implementation of plugins that do not provide meta-data, it is allowed to return {@code null} for any node, regardless of whether it is defined or not.

Returns a `MetaNode` which describes meta data information, can be {@code null} if there is no meta data available for the given node

Throws `DmtException`– with the following possible error codes:

- {@code INVALID_URI} if {@code nodeUri} is {@code null} or syntactically invalid
- {@code URI_TOO_LONG} if {@code nodeUri} is longer than accepted by the {@code DmtAdmin} implementation (especially on systems with limited resources)
- {@code NODE_NOT_FOUND} if {@code nodeUri} points to a node that is not defined in the tree (see above)
- {@code PERMISSION_DENIED} if the session is associated with a principal and the ACL of the node does not allow the {@code Get} operation for the associated principal
- {@code DATA_STORE_FAILURE} if an error occurred while accessing the data store
- {@code COMMAND_FAILED} if the URI is not within the current session's subtree, or if some unspecified error is encountered while attempting to complete the command

`DmtIllegalStateException`– if the session is already closed or invalidated

`SecurityException`– if the caller does not have the necessary permissions to execute the underlying management operation, or, in case of local sessions, if the caller does not have {@code DmtPermission} for the node with the `Get` action present

117.14.10.22 **public abstract Acl getNodeAcl(String nodeUri) throws DmtException**

nodeUri the URI of the node

- Get the Access Control List associated with a given node. The returned {@code Acl} object does not take inheritance into account, it gives the ACL specifically given to the node.

Returns the Access Control List belonging to the node or {@code null} if none defined

Throws `DmtException`– with the following possible error codes:

- {@code INVALID_URI} if {@code nodeUri} is {@code null} or syntactically invalid
- {@code URI_TOO_LONG} if {@code nodeUri} is longer than accepted by the {@code DmtAdmin} implementation (especially on systems with limited resources)
- {@code NODE_NOT_FOUND} if {@code nodeUri} points to a non-existing node
- {@code PERMISSION_DENIED} if the session is associated with a principal and the ACL of the node does not allow the {@code Get} operation for the associated principal
- {@code METADATA_MISMATCH} if node information cannot be retrieved according to the meta-data (the node does not have {@code MetaNode.CMD_GET} access type)
- {@code DATA_STORE_FAILURE} if an error occurred while accessing the data store
- {@code COMMAND_FAILED} if the URI is not within the current session's subtree, or if some unspecified error is encountered while attempting to complete the command

`DmtIllegalStateException`– if the session is already closed or invalidated

`SecurityException`– in case of local sessions, if the caller does not have {@code DmtPermission} for the node with the `Get` action present

See Also `getEffectiveNodeAcl(String)`

117.14.10.23 public abstract int getNodeSize(String nodeUri) throws DmtException

nodeUri the URI of the leaf node

- Get the size of the data in a leaf node. The returned value depends on the format of the data in the node, see the description of the {@code DmtData#getSize()} method for the definition of node size for each format.

Returns the size of the data in the node

Throws DmtException– with the following possible error codes:

- {@code INVALID_URI} if {@code nodeUri} is {@code null} or syntactically invalid
- {@code URI_TOO_LONG} if {@code nodeUri} is longer than accepted by the {@code DmtAdmin} implementation (especially on systems with limited resources)
- {@code NODE_NOT_FOUND} if {@code nodeUri} points to a non-existing node
- {@code PERMISSION_DENIED} if the session is associated with a principal and the ACL of the node does not allow the {@code Get} operation for the associated principal
- {@code COMMAND_NOT_ALLOWED} if the specified node is not a leaf node
- {@code METADATA_MISMATCH} if node information cannot be retrieved according to the meta-data (it does not have {@code MetaNode.CMD_GET} access type)
- {@code FEATURE_NOT_SUPPORTED} if the Size property is not supported by the DmtAdmin implementation or the underlying plugin
- {@code DATA_STORE_FAILURE} if an error occurred while accessing the data store
- {@code COMMAND_FAILED} if the URI is not within the current session's subtree, or if some unspecified error is encountered while attempting to complete the command

DmtIllegalStateException– if the session is already closed or invalidated

SecurityException– if the caller does not have the necessary permissions to execute the underlying management operation, or, in case of local sessions, if the caller does not have {@code DmtPermission} for the node with the Get action present

See Also DmtData.getSize()

117.14.10.24 public abstract Date getNodeTimestamp(String nodeUri) throws DmtException

nodeUri the URI of the node

- Get the timestamp when the node was created or last modified.

Returns the timestamp of the last modification

Throws DmtException– with the following possible error codes:

- {@code INVALID_URI} if {@code nodeUri} is {@code null} or syntactically invalid
- {@code URI_TOO_LONG} if {@code nodeUri} is longer than accepted by the {@code DmtAdmin} implementation (especially on systems with limited resources)
- {@code NODE_NOT_FOUND} if {@code nodeUri} points to a non-existing node
- {@code PERMISSION_DENIED} if the session is associated with a principal and the ACL of the node does not allow the {@code Get} operation for the associated principal
- {@code METADATA_MISMATCH} if node information cannot be retrieved according to the meta-data (it does not have {@code MetaNode.CMD_GET} access type)
- {@code FEATURE_NOT_SUPPORTED} if the Timestamp property is not supported by the DmtAdmin implementation or the underlying plugin
- {@code DATA_STORE_FAILURE} if an error occurred while accessing the data store
- {@code COMMAND_FAILED} if the URI is not within the current session's subtree, or if some unspecified error is encountered while attempting to complete the command

`DmtIllegalStateException` – if the session is already closed or invalidated

`SecurityException` – if the caller does not have the necessary permissions to execute the underlying management operation, or, in case of local sessions, if the caller does not have {@code DmtPermission} for the node with the Get action present

117.14.10.25 **public abstract String getNodeTitle(String nodeUri) throws DmtException**

nodeUri the URI of the node

- Get the title of a node. There might be no title property set for a node.

Returns the title of the node, or {@code null} if the node has no title

Throws `DmtException` – with the following possible error codes:

- {@code INVALID_URI} if {@code nodeUri} is {@code null} or syntactically invalid
- {@code URI_TOO_LONG} if {@code nodeUri} is longer than accepted by the {@code DmtAdmin} implementation (especially on systems with limited resources)
- {@code NODE_NOT_FOUND} if {@code nodeUri} points to a non-existing node
- {@code PERMISSION_DENIED} if the session is associated with a principal and the ACL of the node does not allow the {@code Get} operation for the associated principal
- {@code METADATA_MISMATCH} if node information cannot be retrieved according to the meta-data (it does not have {@code MetaNode.CMD_GET} access type)
- {@code FEATURE_NOT_SUPPORTED} if the Title property is not supported by the `DmtAdmin` implementation or the underlying plugin
- {@code DATA_STORE_FAILURE} if an error occurred while accessing the data store
- {@code COMMAND_FAILED} if the URI is not within the current session's subtree, or if some unspecified error is encountered while attempting to complete the command

`DmtIllegalStateException` – if the session is already closed or invalidated

`SecurityException` – if the caller does not have the necessary permissions to execute the underlying management operation, or, in case of local sessions, if the caller does not have {@code DmtPermission} for the node with the Get action present

117.14.10.26 **public abstract String getNodeType(String nodeUri) throws DmtException**

nodeUri the URI of the node

- Get the type of a node. The type of leaf node is the MIME type of the data it contains. The type of an interior node is a URI identifying a DDF document; a {@code null} type means that there is no DDF document overriding the tree structure defined by the ancestors.

Returns the type of the node, can be {@code null}

Throws `DmtException` – with the following possible error codes:

- {@code INVALID_URI} if {@code nodeUri} is {@code null} or syntactically invalid
- {@code URI_TOO_LONG} if {@code nodeUri} is longer than accepted by the {@code DmtAdmin} implementation (especially on systems with limited resources)
- {@code NODE_NOT_FOUND} if {@code nodeUri} points to a non-existing node
- {@code PERMISSION_DENIED} if the session is associated with a principal and the ACL of the node does not allow the {@code Get} operation for the associated principal
- {@code METADATA_MISMATCH} if node information cannot be retrieved according to the meta-data (it does not have {@code MetaNode.CMD_GET} access type)
- {@code DATA_STORE_FAILURE} if an error occurred while accessing the data store
- {@code COMMAND_FAILED} if the URI is not within the current session's subtree, or if some unspecified error is encountered while attempting to complete the command

DmtIllegalStateException– if the session is already closed or invalidated

SecurityException– if the caller does not have the necessary permissions to execute the underlying management operation, or, in case of local sessions, if the caller does not have {@code DmtPermission} for the node with the Get action present

117.14.10.27 **public abstract DmtData getNodeValue(String nodeUri) throws DmtException**

nodeUri the URI of the node to retrieve

- Get the data contained in a leaf or interior node. When retrieving the value associated with an interior node, the caller must have rights to read all nodes in the subtree under the given node.

Returns the data of the node, can not be {@code null}

Throws DmtException– with the following possible error codes:

- {@code INVALID_URI} if {@code nodeUri} is {@code null} or syntactically invalid
- {@code URI_TOO_LONG} if {@code nodeUri} is longer than accepted by the {@code DmtAdmin} implementation (especially on systems with limited resources)
- {@code NODE_NOT_FOUND} if {@code nodeUri} points to a non-existing node
- {@code PERMISSION_DENIED} if the session is associated with a principal and the ACL of the node (and the ACLs of all its descendants in case of interior nodes) do not allow the {@code Get} operation for the associated principal
- {@code METADATA_MISMATCH} if the node value cannot be retrieved according to the meta-data (it does not have {@code MetaNode.CMD_GET} access type)
- {@code FEATURE_NOT_SUPPORTED} if the specified node is an interior node and does not support Java object values
- {@code DATA_STORE_FAILURE} if an error occurred while accessing the data store
- {@code COMMAND_FAILED} if the URI is not within the current session's subtree, or if some unspecified error is encountered while attempting to complete the command

DmtIllegalStateException– if the session is already closed or invalidated

SecurityException– if the caller does not have the necessary permissions to execute the underlying management operation, or, in case of local sessions, if the caller does not have {@code DmtPermission} for the node (and all its descendants in case of interior nodes) with the Get action present

117.14.10.28 **public abstract int getNodeVersion(String nodeUri) throws DmtException**

nodeUri the URI of the node

- Get the version of a node. The version can not be set, it is calculated automatically by the device. It is incremented modulo 0x10000 at every modification of the value or any other property of the node, for both leaf and interior nodes. When a node is created the initial value is 0.

Returns the version of the node

Throws DmtException– with the following possible error codes:

- {@code INVALID_URI} if {@code nodeUri} is {@code null} or syntactically invalid
- {@code URI_TOO_LONG} if {@code nodeUri} is longer than accepted by the {@code DmtAdmin} implementation (especially on systems with limited resources)
- {@code NODE_NOT_FOUND} if {@code nodeUri} points to a non-existing node
- {@code PERMISSION_DENIED} if the session is associated with a principal and the ACL of the node does not allow the {@code Get} operation for the associated principal
- {@code METADATA_MISMATCH} if node information cannot be retrieved according to the meta-data (it does not have {@code MetaNode.CMD_GET} access type)
- {@code FEATURE_NOT_SUPPORTED} if the Version property is not supported by the DmtAdmin implementation or the underlying plugin

- {@code DATA_STORE_FAILURE} if an error occurred while accessing the data store
- {@code COMMAND_FAILED} if the URI is not within the current session's subtree, or if some unspecified error is encountered while attempting to complete the command

DmtIllegalStateException – if the session is already closed or invalidated

SecurityException – if the caller does not have the necessary permissions to execute the underlying management operation, or, in case of local sessions, if the caller does not have {@code DmtPermission} for the node with the Get action present

117.14.10.29 **public abstract String getPrincipal()**

- Gives the name of the principal on whose behalf the session was created. Local sessions do not have an associated principal, in this case {@code null} is returned.

Returns the identifier of the remote server that initiated the session, or {@code null} for local sessions

117.14.10.30 **public abstract String getRootUri()**

- Get the root URI associated with this session. Gives "{@code .}" if the session was created without specifying a root, which means that the target of this session is the whole DMT.

Returns the root URI

117.14.10.31 **public abstract int getSessionId()**

- The unique identifier of the session. The ID is generated automatically, and it is guaranteed to be unique on a machine for a specific Dmt Admin. A session id must be larger than 0.

Returns the session identification number

117.14.10.32 **public abstract int getState()**

- Get the current state of this session.

Returns the state of the session, one of {@link #STATE_OPEN}, {@link #STATE_CLOSED} and {@link #STATE_INVALID}

117.14.10.33 **public abstract boolean isLeafNode(String nodeUri) throws DmtException**

nodeUri the URI of the node

- Tells whether a node is a leaf or an interior node of the DMT.

Returns true if the given node is a leaf node

Throws DmtException – with the following possible error codes:

- {@code INVALID_URI} if {@code nodeUri} is {@code null} or syntactically invalid
- {@code URI_TOO_LONG} if {@code nodeUri} is longer than accepted by the {@code DmtAdmin} implementation (especially on systems with limited resources)
- {@code NODE_NOT_FOUND} if {@code nodeUri} points to a non-existing node
- {@code PERMISSION_DENIED} if the session is associated with a principal and the ACL of the node does not allow the {@code Get} operation for the associated principal
- {@code METADATA_MISMATCH} if node information cannot be retrieved according to the meta-data (it does not have {@code MetaNode.CMD_GET} access type)
- {@code DATA_STORE_FAILURE} if an error occurred while accessing the data store
- {@code COMMAND_FAILED} if the URI is not within the current session's subtree, or if some unspecified error is encountered while attempting to complete the command

DmtIllegalStateException – if the session is already closed or invalidated

SecurityException– if the caller does not have the necessary permissions to execute the underlying management operation, or, in case of local sessions, if the caller does not have {@code DmtPermission} for the node with the Get action present

117.14.10.34 **public abstract boolean isNodeUri(String nodeUri)**

nodeUri the URI to check

- Check whether the specified URI corresponds to a valid node in the DMT.

Returns true if the given node exists in the DMT

Throws DmtIllegalStateException– if the session is already closed or invalidated

SecurityException– if the caller does not have the necessary permissions to execute the underlying management operation, or, in case of local sessions, if the caller does not have {@code DmtPermission} for the node with the Get action present

117.14.10.35 **public abstract void renameNode(String nodeUri, String newName) throws DmtException**

nodeUri the URI of the node to rename

newName the new name property of the node

- Rename a node. This operation only changes the name of the node (updating the timestamp and version properties if they are supported), the value and the other properties are not changed. The new name of the node must be provided, the new URI is constructed from the base of the old URI and the given name. It is not allowed to rename the root node of the session.

If available, the meta-data of the original and the new nodes are checked before performing the rename operation. Neither node can be permanent, their leaf/interior property must match, and the name change must not violate any of the cardinality constraints. The original node must have the {@code MetaNode.CMD_REPLACE} access type, and the name of the new node must conform to the valid names.

Throws DmtException– with the following possible error codes:

- {@code INVALID_URI} if {@code nodeUri} or {@code newName} is {@code null} or syntactically invalid
- {@code URI_TOO_LONG} if {@code nodeUri} is longer than accepted by the {@code DmtAdmin} implementation (especially on systems with limited resources)
- {@code NODE_NOT_FOUND} if {@code nodeUri} points to a non-existing node, or if the new node is not defined in the tree according to the meta-data (see {@code #getMetaNode(String)})
- {@code NODE_ALREADY_EXISTS} if there already exists a sibling of {@code nodeUri} with the name {@code newName}
- {@code PERMISSION_DENIED} if the session is associated with a principal and the ACL of the node does not allow the {@code Replace} operation for the associated principal
- {@code COMMAND_NOT_ALLOWED} if the target node is the root of the session, or in non-atomic sessions if the underlying plugin is read-only or does not support non-atomic writing
- {@code METADATA_MISMATCH} if the node could not be renamed because of meta-data restrictions (see above)
- {@code TRANSACTION_ERROR} in an atomic session if the underlying plugin is read-only or does not support atomic writing
- {@code DATA_STORE_FAILURE} if an error occurred while accessing the data store
- {@code COMMAND_FAILED} if the URI is not within the current session's subtree, or if some unspecified error is encountered while attempting to complete the command

DmtIllegalStateException– if the session was opened using the {@code LOCK_TYPE_SHARED} lock type, or if the session is already closed or invalidated

SecurityException– if the caller does not have the necessary permissions to execute the underlying management operation, or, in case of local sessions, if the caller does not have {@code DmtPermission} for the node with the Replace action present

117.14.10.36 **public abstract void rollback() throws DmtException**

- Rolls back a series of DMT operations issued in the current atomic session since the last transaction boundary. Transaction boundaries are the creation of this object that starts the session, and all subsequent {@code #commit()} and {@code #rollback()} calls.

Throws DmtException– with the error code {@code ROLLBACK_FAILED} in case the rollback did not succeed

DmtIllegalStateException– if the session was not opened using the {@code LOCK_TYPE_ATOMIC} lock type, or if the session is already closed or invalidated

SecurityException– if the caller does not have the necessary permissions to execute the underlying management operation

117.14.10.37 **public abstract void setDefaultNodeValue(String nodeUri) throws DmtException**

nodeUri the URI of the node

- Set the value of a leaf or interior node to its default. The default can be defined by the node's {@code MetaNode}. The method throws a {@code METADATA_MISMATCH} exception if the node does not have a default value.

Throws DmtException– with the following possible error codes:

- {@code INVALID_URI} if {@code nodeUri} is {@code null} or syntactically invalid
- {@code URI_TOO_LONG} if {@code nodeUri} is longer than accepted by the {@code DmtAdmin} implementation (especially on systems with limited resources)
- {@code NODE_NOT_FOUND} if {@code nodeUri} points to a non-existing node
- {@code PERMISSION_DENIED} if the session is associated with a principal and the ACL of the node does not allow the {@code Replace} operation for the associated principal
- {@code COMMAND_NOT_ALLOWED} in non-atomic sessions if the underlying plugin is read-only or does not support non-atomic writing
- {@code METADATA_MISMATCH} if the node is permanent or cannot be modified according to the meta-data (does not have the {@code MetaNode.CMD_REPLACE} access type), or if there is no default value defined for this node
- {@code FEATURE_NOT_SUPPORTED} if the specified node is an interior node and does not support Java object values
- {@code TRANSACTION_ERROR} in an atomic session if the underlying plugin is read-only or does not support atomic writing
- {@code DATA_STORE_FAILURE} if an error occurred while accessing the data store
- {@code COMMAND_FAILED} if the URI is not within the current session's subtree, or if some unspecified error is encountered while attempting to complete the command

DmtIllegalStateException– if the session was opened using the {@code LOCK_TYPE_SHARED} lock type, or if the session is already closed or invalidated

SecurityException– if the caller does not have the necessary permissions to execute the underlying management operation, or, in case of local sessions, if the caller does not have {@code DmtPermission} for the node with the Replace action present

See Also `setNodeValue(String, DmtData)`

117.14.10.38 **public abstract void setNodeAcl(String nodeUri, Acl acl) throws DmtException**

nodeUri the URI of the node

acl the Access Control List to be set on the node, can be {@code null}

- Set the Access Control List associated with a given node. To perform this operation, the caller needs to have replace rights ({@code Acl.REPLACE}) or the corresponding Java permission depending on the session type) as described below:
 - if {@code nodeUri} specifies a leaf node, replace rights are needed on the parent of the node
 - if {@code nodeUri} specifies an interior node, replace rights on either the node or its parent are sufficient

If the given {@code acl} is {@code null} or an empty ACL (not specifying any permissions for any principals), then the ACL of the node is deleted, and the node will inherit the ACL from its parent node.

Throws `DmtException`– with the following possible error codes:

- {@code INVALID_URI} if {@code nodeUri} is {@code null} or syntactically invalid
- {@code URI_TOO_LONG} if {@code nodeUri} is longer than accepted by the {@code DmtAdmin} implementation (especially on systems with limited resources)
- {@code NODE_NOT_FOUND} if {@code nodeUri} points to a non-existing node
- {@code PERMISSION_DENIED} if the session is associated with a principal and the ACL of the node or its parent (see above) does not allow the {@code Replace} operation for the associated principal
- {@code COMMAND_NOT_ALLOWED} if the command attempts to set the ACL of the root node not to include Add rights for all principals
- {@code DATA_STORE_FAILURE} if an error occurred while accessing the data store
- {@code COMMAND_FAILED} if the URI is not within the current session's subtree, or if some unspecified error is encountered while attempting to complete the command

`DmtIllegalStateException`– if the session was opened using the {@code LOCK_TYPE_SHARED} lock type, or if the session is already closed or invalidated

`SecurityException`– in case of local sessions, if the caller does not have {@code DmtPermission} for the node or its parent (see above) with the Replace action present

117.14.10.39 **public abstract void setNodeTitle(String nodeUri, String title) throws DmtException**

nodeUri the URI of the node

title the title text of the node, can be {@code null}

- Set the title property of a node. The length of the title string in UTF-8 encoding must not exceed 255 bytes.

Throws `DmtException`– with the following possible error codes:

- {@code INVALID_URI} if {@code nodeUri} is {@code null} or syntactically invalid
- {@code URI_TOO_LONG} if {@code nodeUri} is longer than accepted by the {@code DmtAdmin} implementation (especially on systems with limited resources)
- {@code NODE_NOT_FOUND} if {@code nodeUri} points to a non-existing node
- {@code PERMISSION_DENIED} if the session is associated with a principal and the ACL of the node does not allow the {@code Replace} operation for the associated principal
- {@code COMMAND_NOT_ALLOWED} in non-atomic sessions if the underlying plugin is read-only or does not support non-atomic writing
- {@code METADATA_MISMATCH} if the node cannot be modified according to the meta-data (does not have the {@code MetaNode.CMD_REPLACE} access type)
- {@code FEATURE_NOT_SUPPORTED} if the Title property is not supported by the `DmtAdmin` implementation or the underlying plugin

- {@code TRANSACTION_ERROR} in an atomic session if the underlying plugin is read-only or does not support atomic writing
- {@code DATA_STORE_FAILURE} if an error occurred while accessing the data store
- {@code COMMAND_FAILED} if the title string is too long, if the URI is not within the current session's subtree, or if some unspecified error is encountered while attempting to complete the command

DmtIllegalStateException – if the session was opened using the {@code LOCK_TYPE_SHARED} lock type, or if the session is already closed or invalidated

SecurityException – if the caller does not have the necessary permissions to execute the underlying management operation, or, in case of local sessions, if the caller does not have {@code DmtPermission} for the node with the Replace action present

117.14.10.40 **public abstract void setNodeType(String nodeUri, String type) throws DmtException**

nodeUri the URI of the node

type the type of the node, can be {@code null}

- Set the type of a node. The type of leaf node is the MIME type of the data it contains. The type of an interior node is a URI identifying a DDF document.

For interior nodes, a {@code null} type string means that there is no DDF document overriding the tree structure defined by the ancestors. For leaf nodes, it requests that the default MIME type is used for the given node. If the node does not have a default MIME type this method will throw a {@code DmtException} with error code {@code METADATA_MISMATCH}. Note that a node might have a default MIME type even if there is no meta-data for the node or its meta-data does not specify the default.

MIME types must conform to the definition in RFC 2045. Interior node type identifiers must follow the format defined in section 7.7.2 of the OMA Device Management Tree and Description document. Checking the validity of the type string does not have to be done by the *DmtAdmin*, this can be left to the plugin handling the node (if any), to avoid unnecessary double-checks.

Throws *DmtException* – with the following possible error codes:

- {@code INVALID_URI} if {@code nodeUri} is {@code null} or syntactically invalid
- {@code URI_TOO_LONG} if {@code nodeUri} is longer than accepted by the {@code DmtAdmin} implementation (especially on systems with limited resources)
- {@code NODE_NOT_FOUND} if {@code nodeUri} points to a non-existing node
- {@code PERMISSION_DENIED} if the session is associated with a principal and the ACL of the node does not allow the {@code Replace} operation for the associated principal
- {@code COMMAND_NOT_ALLOWED} in non-atomic sessions if the underlying plugin is read-only or does not support non-atomic writing
- {@code METADATA_MISMATCH} if the node is permanent or cannot be modified according to the meta-data (does not have the {@code MetaNode.CMD_REPLACE} access type), and in case of leaf nodes, if {@code null} is given and there is no default MIME type, or the given MIME type is not allowed
- {@code TRANSACTION_ERROR} in an atomic session if the underlying plugin is read-only or does not support atomic writing
- {@code DATA_STORE_FAILURE} if an error occurred while accessing the data store
- {@code COMMAND_FAILED} if the URI is not within the current session's subtree, if the type string is invalid (see above), or if some unspecified error is encountered while attempting to complete the command

DmtIllegalStateException – if the session was opened using the {@code LOCK_TYPE_SHARED} lock type, or if the session is already closed or invalidated

SecurityException– if the caller does not have the necessary permissions to execute the underlying management operation, or, in case of local sessions, if the caller does not have {@code DmtPermission} for the node with the Replace action present

See Also RFC 2045 [http://www.ietf.org/rfc/rfc2045.txt], OMA Device Management Tree and Description v1.2 draft [http://member.openmobilealliance.org/ftp/public_documents/dm/Permanent_documents/OMA-TS-DM-TND-V1_2-20050615-C.zip]

117.14.10.41 **public abstract void setNodeValue(String nodeUri, DmtData data) throws DmtException**

nodeUri the URI of the node

data the data to be set, can be {@code null}

- Set the value of a leaf or interior node. The format of the node is contained in the {@code DmtData} object. For interior nodes, the format must be {@code FORMAT_NODE}, while for leaf nodes this format must not be used.

If the specified value is {@code null}, the default value is taken. In this case, if the node does not have a default value, this method will throw a {@code DmtException} with error code {@code METADATA_MISMATCH}. Nodes of {@code null} format can be set by using {@code DmtData#NULL_VALUE} as second argument.

An Event of type REPLACE is sent out for a leaf node. A replaced interior node sends out events for each of its children in depth first order and node names sorted with `Arrays.sort(String[])`. When setting a value on an interior node, the values of the leaf nodes under it can change, but the structure of the subtree is not modified by the operation.

Throws DmtException– with the following possible error codes:

- {@code INVALID_URI} if {@code nodeUri} is {@code null} or syntactically invalid
- {@code URI_TOO_LONG} if {@code nodeUri} is longer than accepted by the {@code DmtAdmin} implementation (especially on systems with limited resources)
- {@code NODE_NOT_FOUND} if {@code nodeUri} points to a non-existing node
- {@code PERMISSION_DENIED} if the session is associated with a principal and the ACL of the node does not allow the {@code Replace} operation for the associated principal
- {@code COMMAND_NOT_ALLOWED} if the given data has {@code FORMAT_NODE} format but the node is a leaf node (or vice versa), or in non-atomic sessions if the underlying plugin is read-only or does not support non-atomic writing
- {@code METADATA_MISMATCH} if the node is permanent or cannot be modified according to the meta-data (does not have the {@code MetaNode.CMD_REPLACE} access type), or if the given value does not conform to the meta-data value constraints
- {@code FEATURE_NOT_SUPPORTED} if the specified node is an interior node and does not support Java object values
- {@code TRANSACTION_ERROR} in an atomic session if the underlying plugin is read-only or does not support atomic writing
- {@code DATA_STORE_FAILURE} if an error occurred while accessing the data store
- {@code COMMAND_FAILED} if the URI is not within the current session's subtree, or if some unspecified error is encountered while attempting to complete the command

DmtIllegalStateException– if the session was opened using the {@code LOCK_TYPE_SHARED} lock type, or if the session is already closed or invalidated

SecurityException– if the caller does not have the necessary permissions to execute the underlying management operation, or, in case of local sessions, if the caller does not have {@code DmtPermission} for the node with the Replace action present

117.14.11 public abstract MetaNode

The MetaNode contains meta data as standardized by OMA DM but extends it (without breaking the compatibility) to provide for better DMT data quality in an environment where many software components manipulate this data.

The interface has several types of functions to describe the nodes in the DMT. Some methods can be used to retrieve standard OMA DM metadata such as access type, cardinality, default, etc., others are for data extensions such as valid names and values. In some cases the standard behavior has been extended, for example it is possible to provide several valid MIME types, or to differentiate between normal and automatic dynamic nodes.

Most methods in this interface receive no input, just return information about some aspect of the node. However, there are two methods that behave differently, {@link #isValidName(String)} and {@link #isValidValue(DmtData)}. These validation methods are given a potential node name or value (respectively), and can decide whether it is valid for the given node. Passing the validation methods is a necessary condition for a name or value to be used, but it is not necessarily sufficient: the plugin may carry out more thorough (more expensive) checks when the node is actually created or set.

If a {@code MetaNode} is available for a node, the DmtAdmin must use the information provided by it to filter out invalid requests on that node. However, not all methods on this interface are actually used for this purpose, as many of them (e.g. {@link #getFormat()} or {@link #getValidNames()}) can be substituted with the validating methods. For example, {@link #isValidValue(DmtData)} can be expected to check the format, minimum, maximum, etc. of a given value, making it unnecessary for the DmtAdmin to call {@link #getFormat()}, {@link #getMin()}, {@link #getMax()} etc. separately. It is indicated in the description of each method if the DmtAdmin does not enforce the constraints defined by it - such methods are only for external use, for example in user interfaces.

Most of the methods of this class return {@code null} if a certain piece of meta information is not defined for the node or providing this information is not supported. Methods of this class do not throw exceptions.

117.14.11.1 public static final int AUTOMATIC = 2

Constant for representing an automatic node in the tree. This must be returned by {@link #getScope()}. {@code AUTOMATIC} nodes are part of the life cycle of their parent node, they usually describe attributes/properties of the parent.

117.14.11.2 public static final int CMD_ADD = 0

Constant for the ADD access type. If {@link #can(int)} returns {@code true} for this operation, this node can potentially be added to its parent. Nodes with {@link #PERMANENT} or {@link #AUTOMATIC} scope typically do not have this access type.

117.14.11.3 public static final int CMD_DELETE = 1

Constant for the DELETE access type. If {@link #can(int)} returns {@code true} for this operation, the node can potentially be deleted.

117.14.11.4 public static final int CMD_EXECUTE = 2

Constant for the EXECUTE access type. If {@link #can(int)} returns {@code true} for this operation, the node can potentially be executed.

117.14.11.5 public static final int CMD_GET = 4

Constant for the GET access type. If {@link #can(int)} returns {@code true} for this operation, the value, the list of child nodes (in case of interior nodes) and the properties of the node can potentially be retrieved.

- 117.14.11.6** **public static final int CMD_REPLACE = 3**
- Constant for the REPLACE access type. If {@link #can(int)} returns {@code true} for this operation, the value and other properties of the node can potentially be modified.
- 117.14.11.7** **public static final int DYNAMIC = 1**
- Constant for representing a dynamic node in the tree. This must be returned by {@link #getScope()}. Dynamic nodes can be added and deleted.
- 117.14.11.8** **public static final int PERMANENT = 0**
- Constant for representing a {@code PERMANENT} node in the tree. This must be returned by {@link #getScope()} if the node cannot be added, deleted or modified in any way through tree operations. {@code PERMANENT} nodes in general map to the roots of Plugins.
- 117.14.11.9** **public abstract boolean can(int operation)**
- operation* One of the {@code MetaNode.CMD_...} constants.
- Check whether the given operation is valid for this node. If no meta-data is provided for a node, all operations are valid.
- Returns* {@code false} if the operation is not valid for this node or the operation code is not one of the allowed constants
- 117.14.11.10** **public abstract DmtData getDefault()**
- Get the default value of this node if any.
- Returns* The default value or {@code null} if not defined
- 117.14.11.11** **public abstract String getDescription()**
- Get the explanation string associated with this node. Can be {@code null} if no description is provided for this node.
- Returns* node description string or {@code null} for no description
- 117.14.11.12** **public abstract Object getExtensionProperty(String key)**
- key* the key for the extension property
- Returns the value for the specified extension property key. This method only works if the provider of this {@code MetaNode} provides proprietary extensions to node meta data.
- Returns* the value of the requested property, cannot be {@code null}
- Throws* `IllegalArgumentException` – if the specified key is not supported by this {@code MetaNode}
- 117.14.11.13** **public abstract String[][] getExtensionPropertyKeys()**
- Returns the list of extension property keys, if the provider of this {@code MetaNode} provides proprietary extensions to node meta data. The method returns {@code null} if the node doesn't provide such extensions.
- Returns* the array of supported extension property keys
- 117.14.11.14** **public abstract int getFormat()**
- Get the node's format, expressed in terms of type constants defined in {@link DmtData}. If there are multiple formats allowed for the node then the format constants are OR-ed. Interior nodes must have {@link DmtData#FORMAT_NODE} format, and this code must not be returned for leaf nodes. If no meta-data is provided for a node, all applicable formats are considered valid (with the above constraints regarding interior and leaf nodes).

Note that the 'format' term is a legacy from OMA DM, it is more customary to think of this as 'type'.

The formats returned by this method are not checked by DmtAdmin, they are only for external use, for example in user interfaces. DmtAdmin only calls {@link #isValidValue(DmtData)} for checking the value, its behavior should be consistent with this method.

Returns the allowed format(s) of the node

117.14.11.15 public abstract double getMax()

- Get the maximum allowed value associated with a node of numeric format. If no meta-data is provided for a node, there is no upper limit to its value. This method is only meaningful if the node has one of the numeric formats: integer, float, or long format. The returned limit has {@code double} type, as this can be used to denote all numeric limits with full precision. The actual maximum should be the largest integer, float or long number that does not exceed the returned value.

The information returned by this method is not checked by DmtAdmin, it is only for external use, for example in user interfaces. DmtAdmin only calls {@link #isValidValue(DmtData)} for checking the value, its behavior should be consistent with this method.

Returns the allowed maximum, or {@code Double.MAX_VALUE} if there is no upper limit defined or the node's format is not one of the numeric formats integer, float, or long

117.14.11.16 public abstract int getMaxOccurrence()

- Get the number of maximum occurrences of this type of nodes on the same level in the DMT. Returns {@code Integer.MAX_VALUE} if there is no upper limit. Note that if the occurrence is greater than 1 then this node can not have siblings with different metadata. In other words, if different types of nodes coexist on the same level, their occurrence can not be greater than 1. If no meta-data is provided for a node, there is no upper limit on the number of occurrences.

Returns The maximum allowed occurrence of this node type

117.14.11.17 public abstract String[][] getMimeType()

- Get the list of MIME types this node can hold. The first element of the returned list must be the default MIME type.

All MIME types are considered valid if no meta-data is provided for a node or if {@code null} is returned by this method. In this case the default MIME type cannot be retrieved from the meta-data, but the node may still have a default. This hidden default (if it exists) can be utilized by passing {@code null} as the type parameter of {@link DmtSession#setNodeType(String, String)} or {@link DmtSession#createLeafNode(String, DmtData, String)}.

Returns the list of allowed MIME types for this node, starting with the default MIME type, or {@code null} if all types are allowed

117.14.11.18 public abstract double getMin()

- Get the minimum allowed value associated with a node of numeric format. If no meta-data is provided for a node, there is no lower limit to its value. This method is only meaningful if the node has one of the numeric formats: integer, float, or long format. The returned limit has {@code double} type, as this can be used to denote both integer and float limits with full precision. The actual minimum should be the smallest integer, float or long value that is equal or larger than the returned value.

The information returned by this method is not checked by DmtAdmin, it is only for external use, for example in user interfaces. DmtAdmin only calls {@link #isValidValue(DmtData)} for checking the value, its behavior should be consistent with this method.

Returns the allowed minimum, or {@code Double.MIN_VALUE} if there is no lower limit defined or the node's format is not one of the numeric formats integer, float, or long

117.14.11.19 public abstract String[][] getRawFormatNames()

- Get the format names for any raw formats supported by the node. This method is only meaningful if the list of supported formats returned by {@link #getFormat()} contains {@link DmtData#FORMAT_RAW_STRING} or {@link DmtData#FORMAT_RAW_BINARY}; it specifies precisely which raw format(s) are actually supported. If the node cannot contain data in one of the raw types, this method must return {@code null}.

The format names returned by this method are not checked by DmtAdmin, they are only for external use, for example in user interfaces. DmtAdmin only calls {@link #isValidValue(DmtData)} for checking the value, its behavior should be consistent with this method.

Returns the allowed format name(s) of raw data stored by the node, or {@code null} if raw formats are not supported

117.14.11.20 public abstract int getScope()

- Return the scope of the node. Valid values are {@link #PERMANENT MetaNode.PERMANENT}, {@link #DYNAMIC MetaNode.DYNAMIC} and {@link #AUTOMATIC MetaNode.AUTOMATIC}. Note that a permanent node is not the same as a node where the DELETE operation is not allowed. Permanent nodes never can be deleted, whereas a non-deletable node can disappear in a recursive DELETE operation issued on one of its parents. If no meta-data is provided for a node, it can be assumed to be a dynamic node.

Returns {@link #PERMANENT} for permanent nodes, {@link #AUTOMATIC} for nodes that are automatically created, and {@link #DYNAMIC} otherwise

117.14.11.21 public abstract String[][] getValidNames()

- Return an array of Strings if valid names are defined for the node, or {@code null} if no valid name list is defined or if this piece of meta info is not supported. If no meta-data is provided for a node, all names are considered valid.

The information returned by this method is not checked by DmtAdmin, it is only for external use, for example in user interfaces. DmtAdmin only calls {@link #isValidName(String)} for checking the name, its behavior should be consistent with this method.

Returns the valid values for this node name, or {@code null} if not defined

117.14.11.22 public abstract DmtData[][] getValidValues()

- Return an array of DmtData objects if valid values are defined for the node, or {@code null} otherwise. If no meta-data is provided for a node, all values are considered valid.

The information returned by this method is not checked by DmtAdmin, it is only for external use, for example in user interfaces. DmtAdmin only calls {@link #isValidValue(DmtData)} for checking the value, its behavior should be consistent with this method.

Returns the valid values for this node, or {@code null} if not defined

117.14.11.23 public abstract boolean isLeaf()

- Check whether the node is a leaf node or an internal one.

Returns {@code true} if the node is a leaf node

117.14.11.24 public abstract boolean isValidName(String name)

name the node name to check for validity

- Checks whether the given name is a valid name for this node. This method can be used for example to ensure that the node name is always one of a predefined set of valid names, or that it matches a specific pattern. This method should be consistent with the values returned by {@link #getValidNames()} (if any), the DmtAdmin only calls this method for name validation.

This method may return {@code true} even if not all aspects of the name have been checked, expensive operations (for example those that require external resources) need not be performed here. The actual node creation may still indicate that the node name is invalid.

Returns {@code false} if the specified name is found to be invalid for the node described by this meta-node, {@code true} otherwise

117.14.11.25 **public abstract boolean isValidValue(DmtData value)**

value the value to check for validity

- Checks whether the given value is valid for this node. This method can be used to ensure that the value has the correct format and range, that it is well formed, etc. This method should be consistent with the constraints defined by the {@link #getFormat()}, {@link #getValidValues()}, {@link #getMin()} and {@link #getMax()} methods (if applicable), as the Dmt Admin only calls this method for value validation.

This method may return {@code true} even if not all aspects of the value have been checked, expensive operations (for example those that require external resources) need not be performed here. The actual value setting method may still indicate that the value is invalid.

Returns {@code false} if the specified value is found to be invalid for the node described by this meta-node, {@code true} otherwise

117.14.11.26 **public abstract boolean isZeroOccurrenceAllowed()**

- Check whether zero occurrence of this node is valid. If no meta-data is returned for a node, zero occurrences are allowed.

Returns {@code true} if zero occurrence of this node is valid

117.14.12 **public final class Uri**

This class contains static utility methods to manipulate DMT URIs.

Syntax of valid DMT URIs:

- A slash ({@code '/'} \u002F) is the separator of the node names. Slashes used in node name must therefore be escaped using a backslash slash ({@code "\u002F"}). The backslash must be escaped with a double backslash sequence. A backslash found must be ignored when it is not followed by a slash or backslash.
- The node name can be constructed using full Unicode character set (except the Supplementary code, not being supported by CLDC/CDC). However, using the full Unicode character set for node names is discouraged because the encoding in the underlying storage as well as the encoding needed in communications can create significant performance and memory usage overhead. Names that are restricted to the URI set {@code [-a-zA-Z0-9_!~*()]} are most efficient.
- URIs used in the DMT must be treated and interpreted as case sensitive.
- No End Slash: URI must not end with the delimiter slash ({@code '/'} \u002F). This implies that the root node must be denoted as {@code "."} and not {@code "."/}.
- No parent denotation: URI must not be constructed using the character sequence {@code "../"} to traverse the tree upwards.
- Single Root: The character sequence {@code "../"} must not be used anywhere else but in the beginning of a URI.

117.14.12.1 **public static final String PATH_SEPARATOR = "/"**

This constant stands for a string identifying the path separator in the DmTree ("/").

Since @since 2.0

117.14.12.2 public static final char PATH_SEPARATOR_CHAR = '/'

This constant stands for a char identifying the path separator in the DmTree ('/').

Since @since 2.0

117.14.12.3 public static final String ROOT_NODE = ""

This constant stands for a string identifying the root of the DmTree ("").

Since @since 2.0

117.14.12.4 public static final char ROOT_NODE_CHAR = '.'

This constant stands for a char identifying the root of the DmTree ('.').

Since @since 2.0

117.14.12.5 public static String decode(String nodeName)

nodeName the node name to be decoded

- Decode the node name so that back slash and forward slash are unescaped from a back slash.

Returns the decoded node name

Since @since 2.0

117.14.12.6 public static String encode(String nodeName)

nodeName the node name to be encoded

- Encode the node name so that back slash and forward slash are escaped with a back slash. This method is the reverse of {@link #decode(String)}.

Returns the encoded node name

Since @since 2.0

117.14.12.7 public static boolean isAbsoluteUri(String uri)

uri the URI to be checked, must not be {@code null} and must contain a valid URI

- Checks whether the specified URI is an absolute URI. An absolute URI contains the complete path to a node in the DMT starting from the DMT root ("").

Returns whether the specified URI is absolute

Throws NullPointerException – if the specified URI is {@code null}
 IllegalArgumentException – if the specified URI is malformed

117.14.12.8 public static boolean isValidUri(String uri)

uri the URI to be validated

- Checks whether the specified URI is valid. A URI is considered valid if it meets the following constraints:

- the URI is not {@code null};
- the URI follows the syntax defined for valid DMT URIs;

The exact definition of the length of a URI and its segments is given in the descriptions of the {@code getMaxUriLength()} and {@code getMaxSegmentNameLength()} methods.

Returns whether the specified URI is valid

117.14.12.9 public static String mangle(String nodeName)

nodeName the node name to be mangled (if necessary), must not be {@code null} or empty

- Returns a node name that is valid for the tree operation methods, based on the given node name. This transformation is not idempotent, so it must not be called with a parameter that is the result of a previous {@code mangle} method call.

Node name mangling is needed in the following cases:

- if the name contains '/' or '\' characters

A node name that does not suffer from either of these problems is guaranteed to remain unchanged by this method. Therefore the client may skip the mangling if the node name is known to be valid (though it is always safe to call this method).

The method returns the normalized {@code nodeName} as described below. Invalid node names are normalized in different ways, depending on the cause. If the name contains '/' or '\' characters, then these are simply escaped by inserting an additional '\' before each occurrence. If the length of the name does exceed the limit, the following mechanism is used to normalize it:

- the SHA-1 digest of the name is calculated
- the digest is encoded with the base 64 algorithm
- all '/' characters in the encoded digest are replaced with '_'
- trailing '=' signs are removed

Returns the normalized node name that is valid for tree operations

Throws `NullPointerException` – if {@code nodeName} is `{@code null}`

`IllegalArgumentException` – if {@code nodeName} is empty

117.14.12.10 **public static String[][] toPath(String uri)**

uri the URI to be split, must not be `{@code null}`

- Split the specified URI along the path separator '/' characters and return an array of URI segments. Special characters in the returned segments are escaped. The returned array may be empty if the specified URI was empty.

Returns an array of URI segments created by splitting the specified URI

Throws `NullPointerException` – if the specified URI is `{@code null}`

`IllegalArgumentException` – if the specified URI is malformed

117.14.12.11 **public static String toUri(String[] path)**

path a possibly empty array of URI segments, must not be `{@code null}`

- Construct a URI from the specified URI segments. The segments must already be mangled. If the specified path is an empty array then an empty URI (`{@code ""}`) is returned.

Returns the URI created from the specified segments

Throws `NullPointerException` – if the specified path or any of its segments are `{@code null}`

`IllegalArgumentException` – if the specified path contains too many or malformed segments or the resulting URI is too long

117.15 **org.osgi.service.dmt.spi**

Device Management Tree SPI Package Version 2.0.

This package contains the interface classes that compose the Device Management SPI (Service Provider Interface). These interfaces are implemented by DMT plugins; users of the `DmtAdmin` interface do not interact directly with these.

Bundles wishing to use this package must list the package in the Import-Package header of the bundle's manifest. This package has two types of users: the consumers that use the API in this package and the providers that implement the API in this package.

Example import for consumers using the API in this package:

```
{@code Import-Package: org.osgi.service.dmt.spi; version="[2.0,3.0]"}
```

Example import for providers implementing the API in this package:

```
{@code Import-Package: org.osgi.service.dmt.spi; version="[2.0,2.1]"}
```

117.15.1

Summary

- `DataPlugin` - An implementation of this interface takes the responsibility of handling data requests in a subtree of the DMT.
- `ExecPlugin` - An implementation of this interface takes the responsibility of handling node execute requests in a subtree of the DMT.
- `MountPlugin` - This interface can be optionally implemented by a `DataPlugin` or `ExecPlugin` in order to get information about its absolute mount points in the overall DMT.
- `MountPoint` - This interface can be implemented to represent a single mount point.
- `ReadableDataSession` - Provides read-only access to the part of the tree handled by the plugin that created this session.
- `ReadWriteDataSession` - Provides non-atomic read-write access to the part of the tree handled by the plugin that created this session.
- `TransactionalDataSession` - Provides atomic read-write access to the part of the tree handled by the plugin that created this session.

117.15.2

public abstract `DataPlugin`

An implementation of this interface takes the responsibility of handling data requests in a subtree of the DMT.

In an OSGi environment such implementations should be registered at the OSGi service registry specifying the list of root node URIs in a `String` array or in case of a single value as `String` in the `dataRootURIs` registration parameter.

When the first reference in a session is made to a node handled by this plugin, the `DmtAdmin` calls one of the `open...` methods to retrieve a plugin session object for processing the request. The called method depends on the lock type of the current session. In case of `openReadWriteSession(String[], DmtSession)` and `openAtomicSession(String[], DmtSession)`, the plugin may return `null` to indicate that the specified lock type is not supported. In this case the `DmtAdmin` may call `openReadOnlySession(String[], DmtSession)` to start a read-only plugin session, which can be used as long as there are no write operations on the nodes handled by this plugin.

The `sessionRoot` parameter of each method is a `String` array containing the segments of the URI pointing to the root of the session. This is an absolute path, so the first segment is always `."`. Special characters appear escaped in the segments.

117.15.2.1

public static final `String DATA_ROOT_URI = "dataRootURIs"`

The string to be used as key for the `dataRootURIs` property when an `DataPlugin` is registered.

Since `@since 2.0`

117.15.2.2 public static final String MOUNT_POINTS = "mountPoints"

The string to be used as key for the mount points property when a DataPlugin is registered with mount points.

117.15.2.3 public abstract TransactionalDataSession openAtomicSession(String[][] sessionRoot, DmtSession session) throws DmtException

sessionRoot the path to the subtree which is locked in the current session, must not be {@code null}

session the session from which this plugin instance is accessed, must not be {@code null}

- This method is called to signal the start of an atomic read-write session when the first reference is made within a {@code DmtSession} to a node which is handled by this plugin. Session information is given as it is needed for sending alerts back from the plugin.

The plugin can assume that there are no other sessions open on any subtree that has any overlap with the subtree of this session.

Returns a plugin session capable of executing read-write operations in an atomic block, or {@code null} if the plugin does not support atomic read-write sessions

Throws DmtException– with the following possible error codes:

- {@code NODE_NOT_FOUND} if {@code sessionRoot} points to a non-existing node
- {@code COMMAND_FAILED} if some unspecified error is encountered while attempting to complete the command

SecurityException– if some underlying operation failed because of lack of permissions

117.15.2.4 public abstract ReadableDataSession openReadOnlySession(String[][] sessionRoot, DmtSession session) throws DmtException

sessionRoot the path to the subtree which is accessed in the current session, must not be {@code null}

session the session from which this plugin instance is accessed, must not be {@code null}

- This method is called to signal the start of a read-only session when the first reference is made within a {@code DmtSession} to a node which is handled by this plugin. Session information is given as it is needed for sending alerts back from the plugin.

The plugin can assume that there are no writing sessions open on any subtree that has any overlap with the subtree of this session.

Returns a plugin session capable of executing read operations

Throws DmtException– with the following possible error codes:

- {@code NODE_NOT_FOUND} if {@code sessionRoot} points to a non-existing node
- {@code COMMAND_FAILED} if some unspecified error is encountered while attempting to complete the command

SecurityException– if some underlying operation failed because of lack of permissions

117.15.2.5 public abstract ReadWriteDataSession openReadWriteSession(String[][] sessionRoot, DmtSession session) throws DmtException

sessionRoot the path to the subtree which is locked in the current session, must not be {@code null}

session the session from which this plugin instance is accessed, must not be {@code null}

- This method is called to signal the start of a non-atomic read-write session when the first reference is made within a {@code DmtSession} to a node which is handled by this plugin. Session information is given as it is needed for sending alerts back from the plugin.

The plugin can assume that there are no other sessions open on any subtree that has any overlap with the subtree of this session.

Returns a plugin session capable of executing read-write operations, or `{@code null}` if the plugin does not support non-atomic read-write sessions

Throws `DmtException`– with the following possible error codes:

- `{@code NODE_NOT_FOUND}` if `{@code sessionRoot}` points to a non-existing node
- `{@code COMMAND_FAILED}` if some unspecified error is encountered while attempting to complete the command

`SecurityException`– if some underlying operation failed because of lack of permissions

117.15.3 **public abstract ExecPlugin**

An implementation of this interface takes the responsibility of handling node execute requests in a subtree of the DMT.

In an OSGi environment such implementations should be registered at the OSGi service registry specifying the list of root node URIs in a `{@code String}` array or in case of a single value as `{@code String}` in the `{@code execRootURIs}` registration parameter.

117.15.3.1 **public static final String EXEC_ROOT_URI = "execRootURIs"**

The string to be used as key for the "execRootURIs" property when an ExecPlugin is registered.

Since `@since 2.0`

117.15.3.2 **public static final String MOUNT_POINTS = "mountPoints"**

The string to be used as key for the mount points property when an Exec Plugin is registered with mount points.

117.15.3.3 **public abstract void execute(DmtSession session, String[][] nodePath, String correlator, String data) throws DmtException**

session a reference to the session in which the operation was issued, must not be `{@code null}`

nodePath the absolute path of the node to be executed, must not be `{@code null}`

correlator an identifier to associate this operation with any alerts sent in response to it, can be `{@code null}`

data the parameter of the execute operation, can be `{@code null}`

- Execute the given node with the given data. This operation corresponds to the EXEC command in OMA DM.

The semantics of an execute operation and the data parameter it takes depends on the definition of the managed object on which the command is issued. Session information is given as it is needed for sending alerts back from the plugin. If a correlation ID is specified, it should be used as the `{@code correlator}` parameter for alerts sent in response to this execute operation.

The `{@code nodePath}` parameter contains an array of path segments identifying the node to be executed in the subtree of this plugin. This is an absolute path, so the first segment is always `"."`. Special characters appear escaped in the segments.

Throws `DmtException`– with the following possible error codes:

- `{@code NODE_NOT_FOUND}` if the node does not exist
- `{@code METADATA_MISMATCH}` if the command failed because of meta-data restrictions
- `{@code DATA_STORE_FAILURE}` if an error occurred while accessing the data store
- `{@code COMMAND_FAILED}` if some unspecified error is encountered while attempting to complete the command

See Also `DmtSession.execute(String, String)`, `DmtSession.execute(String, String, String)`

117.15.4 **public abstract MountPlugin**

This interface can be optionally implemented by a {@code DataPlugin} or {@code ExecPlugin} in order to get information about its absolute mount points in the overall DMT.

This is especially interesting, if the plugin is mapped to the tree as part of a list. In such a case the id for this particular data plugin is determined by the `DmtAdmin` after the registration of the plugin and therefore unknown to the plugin in advance.

This is not a service interface, the Data or Exec Plugin does not also have to register this interface as a service, the Dmt Admin should use an {@code instanceof} to detect that a Plugin is also a Mount Plugin.

Since @since 2.0

117.15.4.1 **public abstract void mountPointAdded(MountPoint mountPoint)**

mountPoint the newly mapped mount point

- Provides the {@code MountPoint} describing the path where the plugin is mapped in the overall DMT. The given `mountPoint` is withdrawn with the {@link #mountPointRemoved(MountPoint)} method. Corresponding mount points must compare equal and have an appropriate hash code.

117.15.4.2 **public abstract void mountPointRemoved(MountPoint mountPoint)**

mountPoint The unmapped mount point array of {@code MountPoint} objects that have been removed from the mapping

- Informs the plugin that the provided {@code MountPoint} objects have been removed from the mapping. The given `mountPoint` is withdrawn method. Mount points must compare equal and have an appropriate hash code with the given Mount Point in {@link #mountPointAdded(MountPoint)}.

NOTE: attempts to invoke the {@code postEvent} method on the provided {@code MountPoint} must be ignored.

117.15.5 **public abstract MountPoint**

This interface can be implemented to represent a single mount point.

It provides function to get the absolute mounted uri and a shortcut method to post events via the `DmtAdmin`.

Since @since 2.0

117.15.5.1 **public abstract boolean equals(Object other)**

- This object must provide a suitable hash function such that a Mount Point given in {@link MountPlugin#mountPointAdded(MountPoint)} is equal to the corresponding Mount Point in {@link MountPlugin#mountPointRemoved(MountPoint)}. {@link Object#equals(Object)}

117.15.5.2 **public abstract String[] getMountPath()**

- Provides the absolute mount path of this {@code MountPoint}

Returns the absolute mount path of this {@code MountPoint}

117.15.5.3 **public abstract int hashCode()**

- This object must provide a suitable hash function such that a Mount Point given in {@link MountPlugin#mountPointAdded(MountPoint)} has the same `hashCode` as the corresponding Mount Point in {@link MountPlugin#mountPointRemoved(MountPoint)}. {@link Object#hashCode()}

117-15.5.4 public abstract void postEvent(String topic, String[][] relativeURIs, Dictionary<String, ?> properties)

topic the topic of the event to send. Valid values are:

- {@code org.osgi/service/dmt/DmtEvent/ADDED} if the change was caused by an add action
- {@code org.osgi/service/dmt/DmtEvent/DELETED} if the change was caused by a delete action
- {@code org.osgi/service/dmt/DmtEvent/REPLACED} if the change was caused by a replace action

Must not be {@code null}.

relativeURIs an array of affected node {@code URI}'s. All {@code URI}'s specified here are relative to the current {@code MountPoint}'s mountPath. The value of this parameter determines the value of the event property {@code EVENT_PROPERTY_NODES}. An empty array or {@code null} is permitted. In both cases the value of the events {@code EVENT_PROPERTY_NODES} property will be set to an empty array.

properties an optional parameter that can be provided to add properties to the Event that is going to be send by the DMTAdmin. If the properties contain a key {@code EVENT_PROPERTY_NODES}, then the value of this property is ignored and will be overwritten by {@code relativeURIs}.

- Posts an event via the DmtAdmin about changes in the current plugins subtree.

This method distributes Events asynchronously to the EventAdmin as well as to matching local DmtEventListeners.

Throws IllegalArgumentException – if the topic has not one of the defined values

117-15.5.5 public abstract void postEvent(String topic, String[][] relativeURIs, String[][] newRelativeURIs, Dictionary<String, ?> properties)

topic the topic of the event to send. Valid values are:

- {@code org.osgi/service/dmt/DmtEvent/RENAMED} if the change was caused by a rename action
- {@code org.osgi/service/dmt/DmtEvent/COPIED} if the change was caused by a copy action

Must not be {@code null}.

relativeURIs an array of affected node {@code URI}'s.

All {@code URI}'s specified here are relative to the current {@code MountPoint}'s mountPath. The value of this parameter determines the value of the event property {@code EVENT_PROPERTY_NODES}. An empty array or {@code null} is permitted. In both cases the value of the events {@code EVENT_PROPERTY_NODES} property will be set to an empty array.

newRelativeURIs an array of affected node {@code URI}'s. The value of this parameter determines the value of the event property {@code EVENT_PROPERTY_NEW_NODES}. An empty array or {@code null} is permitted. In both cases the value of the events {@code EVENT_PROPERTY_NEW_NODES} property will be set to an empty array.

properties an optional parameter that can be provided to add properties to the Event that is going to be send by the DMTAdmin. If the properties contain the keys {@code EVENT_PROPERTY_NODES} or {@code EVENT_PROPERTY_NEW_NODES}, then the values of these properties are ignored and will be overwritten by {@code relativeURIs} and {@code newRelativeURIs}.

- Posts an event via the DmtAdmin about changes in the current plugins subtree.

This method distributes Events asynchronously to the EventAdmin as well as to matching local DmtEventListeners.

Throws IllegalArgumentException – if the topic has not one of the defined values

117.15.6 public abstract ReadableDataSession

Provides read-only access to the part of the tree handled by the plugin that created this session.

Since the {@link ReadWriteDataSession} and {@link TransactionalDataSession} interfaces inherit from this interface, some of the method descriptions do not apply for an instance that is only a {@code ReadableDataSession}. For example, the {@link #close()} method description also contains information about its behavior when invoked as part of a transactional session.

The {@code nodePath} parameters appearing in this interface always contain an array of path segments identifying a node in the subtree of this plugin. This parameter contains an absolute path, so the first segment is always ".". Special characters appear escaped in the segments.

Error handling

When a tree access command is called on the DmtAdmin service, it must perform an extensive set of checks on the parameters and the authority of the caller before delegating the call to a plugin. Therefore plugins can take certain circumstances for granted: that the path is valid and is within the subtree of the plugin and the session, the command can be applied to the given node (e.g. the target of {@code getChildNodeNames} is an interior node), etc. All errors described by the error codes {@link DmtException#INVALID_URI}, {@link DmtException#URI_TOO_LONG}, {@link DmtException#PERMISSION_DENIED}, {@link DmtException#COMMAND_NOT_ALLOWED} and {@link DmtException#TRANSACTION_ERROR} are fully filtered out before control reaches the plugin.

If the plugin provides meta-data for a node, the DmtAdmin service must also check the constraints specified by it, as described in {@link MetaNode}. If the plugin does not provide meta-data, it must perform the necessary checks for itself and use the {@link DmtException#METADATA_MISMATCH} error code to indicate such discrepancies.

The DmtAdmin does not check that the targeted node exists before calling the plugin. It is the responsibility of the plugin to perform this check and to throw a {@link DmtException#NODE_NOT_FOUND} if needed. In this case the DmtAdmin must pass through this exception to the caller of the corresponding DmtSession method.

The plugin can use the remaining error codes as needed. If an error does not fit into any other category, the {@link DmtException#COMMAND_FAILED} code should be used.

117.15.6.1 public abstract void close() throws DmtException

- Closes a session. This method is always called when the session ends for any reason: if the session is closed, if a fatal error occurs in any method, or if any error occurs during commit or rollback. In case the session was invalidated due to an exception during commit or rollback, it is guaranteed that no methods are called on the plugin until it is closed. In case the session was invalidated due to a fatal exception in one of the tree manipulation methods, only the rollback method is called before this (and only in atomic sessions).

This method should not perform any data manipulation, only cleanup operations. In non-atomic read-write sessions the data manipulation should be done instantly during each tree operation, while in atomic sessions the {@code DmtAdmin} always calls {@link TransactionalDataSession#commit()} automatically before the session is actually closed.

Throws DmtException— with the error code {@code COMMAND_FAILED} if the plugin failed to close for any reason

117.15.6.2 public abstract String[][] getChildNodeNames(String[][] nodePath) throws DmtException

nodePath the absolute path of the node

- Get the list of children names of a node. The returned array contains the names - not the URIs - of the immediate children nodes of the given node. The returned array may contain {@code null} entries, but these are removed by the DmtAdmin before returning it to the client.

Returns the list of child node names as a string array or an empty string array if the node has no children

Throws `DmtException`– with the following possible error codes:

- {@code NODE_NOT_FOUND} if {@code nodePath} points to a non-existing node
- {@code METADATA_MISMATCH} if the information could not be retrieved because of meta-data restrictions
- {@code DATA_STORE_FAILURE} if an error occurred while accessing the data store
- {@code COMMAND_FAILED} if some unspecified error is encountered while attempting to complete the command

`SecurityException`– if the caller does not have the necessary permissions to execute the underlying management operation

117.15.6.3 **public abstract MetaNode getMetaNode(String[][] nodePath) throws DmtException**

nodePath the absolute path of the node

- Get the meta data which describes a given node. Meta data can be only inspected, it can not be changed.

Meta data support by plugins is an optional feature. It can be used, for example, when a data plugin is implemented on top of a data store or another API that has their own metadata, such as a relational database, in order to avoid metadata duplication and inconsistency. The meta data specific to the plugin returned by this method is complemented by meta data from the `DmtAdmin` before returning it to the client. If there are differences in the meta data elements known by the plugin and the {@code DmtAdmin} then the plugin specific elements take precedence.

Note, that a node does not have to exist for having meta-data associated with it. This method may provide meta-data for any node that can possibly exist in the tree (any node defined by the Management Object provided by the plugin). For nodes that are not defined, a {@code DmtException} may be thrown with the {@code NODE_NOT_FOUND} error code. To allow easier implementation of plugins that do not provide meta-data, it is allowed to return {@code null} for any node, regardless of whether it is defined or not.

Returns a `MetaNode` which describes meta data information, can be {@code null} if there is no meta data available for the given node

Throws `DmtException`– with the following possible error codes:

- {@code NODE_NOT_FOUND} if {@code nodeUri} points to a node that is not defined in the tree (see above)
- {@code DATA_STORE_FAILURE} if an error occurred while accessing the data store
- {@code COMMAND_FAILED} if some unspecified error is encountered while attempting to complete the command

`SecurityException`– if the caller does not have the necessary permissions to execute the underlying management operation

117.15.6.4 **public abstract int getNodeSize(String[][] nodePath) throws DmtException**

nodePath the absolute path of the leaf node

- Get the size of the data in a leaf node. The value to return depends on the format of the data in the node, see the description of the {@link DmtData#getSize()} method for the definition of node size for each format.

Returns the size of the data in the node

Throws `DmtException`– with the following possible error codes:

- {@code NODE_NOT_FOUND} if {@code nodePath} points to a non-existing node

- {@code METADATA_MISMATCH} if the information could not be retrieved because of meta-data restrictions
- {@code FEATURE_NOT_SUPPORTED} if the Size property is not supported by the plugin
- {@code DATA_STORE_FAILURE} if an error occurred while accessing the data store
- {@code COMMAND_FAILED} if some unspecified error is encountered while attempting to complete the command

SecurityException– if the caller does not have the necessary permissions to execute the underlying management operation

See Also `DmtData.getSize()`

117.15.6.5 **public abstract Date getNodeTimestamp(String[][] nodePath) throws DmtException**

nodePath the absolute path of the node

- Get the timestamp when the node was last modified.

Returns the timestamp of the last modification

Throws DmtException– with the following possible error codes:

- {@code NODE_NOT_FOUND} if {@code nodePath} points to a non-existing node
- {@code METADATA_MISMATCH} if the information could not be retrieved because of meta-data restrictions
- {@code FEATURE_NOT_SUPPORTED} if the Timestamp property is not supported by the plugin
- {@code DATA_STORE_FAILURE} if an error occurred while accessing the data store
- {@code COMMAND_FAILED} if some unspecified error is encountered while attempting to complete the command

SecurityException– if the caller does not have the necessary permissions to execute the underlying management operation

117.15.6.6 **public abstract String getNodeTitle(String[][] nodePath) throws DmtException**

nodePath the absolute path of the node

- Get the title of a node. There might be no title property set for a node.

Returns the title of the node, or {@code null} if the node has no title

Throws DmtException– with the following possible error codes:

- {@code NODE_NOT_FOUND} if {@code nodePath} points to a non-existing node
- {@code METADATA_MISMATCH} if the information could not be retrieved because of meta-data restrictions
- {@code FEATURE_NOT_SUPPORTED} if the Title property is not supported by the plugin
- {@code DATA_STORE_FAILURE} if an error occurred while accessing the data store
- {@code COMMAND_FAILED} if some unspecified error is encountered while attempting to complete the command

SecurityException– if the caller does not have the necessary permissions to execute the underlying management operation

117.15.6.7 **public abstract String getNodeType(String[][] nodePath) throws DmtException**

nodePath the absolute path of the node

- Get the type of a node. The type of leaf node is the MIME type of the data it contains. The type of an interior node is a URI identifying a DDF document; a {@code null} type means that there is no DDF document overriding the tree structure defined by the ancestors.

Returns the type of the node, can be {@code null}

Throws `DmtException`– with the following possible error codes:

- {@code NODE_NOT_FOUND} if {@code nodePath} points to a non-existing node
- {@code METADATA_MISMATCH} if the information could not be retrieved because of meta-data restrictions
- {@code DATA_STORE_FAILURE} if an error occurred while accessing the data store
- {@code COMMAND_FAILED} if some unspecified error is encountered while attempting to complete the command

`SecurityException`– if the caller does not have the necessary permissions to execute the underlying management operation

117.15.6.8 **public abstract DmtData getNodeValue(String[][] nodePath) throws DmtException**

nodePath the absolute path of the node to retrieve

- Get the data contained in a leaf or interior node.

Returns the data of the leaf node, must not be {@code null}

Throws `DmtException`– with the following possible error codes:

- {@code NODE_NOT_FOUND} if {@code nodePath} points to a non-existing node
- {@code METADATA_MISMATCH} if the information could not be retrieved because of meta-data restrictions
- {@code FEATURE_NOT_SUPPORTED} if the specified node is an interior node and does not support Java object values
- {@code DATA_STORE_FAILURE} if an error occurred while accessing the data store
- {@code COMMAND_FAILED} if some unspecified error is encountered while attempting to complete the command

`SecurityException`– if the caller does not have the necessary permissions to execute the underlying management operation

117.15.6.9 **public abstract int getNodeVersion(String[][] nodePath) throws DmtException**

nodePath the absolute path of the node

- Get the version of a node. The version can not be set, it is calculated automatically by the device. It is incremented modulo 0x10000 at every modification of the value or any other property of the node, for both leaf and interior nodes. When a node is created the initial value is 0.

Returns the version of the node

Throws `DmtException`– with the following possible error codes:

- {@code NODE_NOT_FOUND} if {@code nodePath} points to a non-existing node
- {@code METADATA_MISMATCH} if the information could not be retrieved because of meta-data restrictions
- {@code FEATURE_NOT_SUPPORTED} if the Version property is not supported by the plugin
- {@code DATA_STORE_FAILURE} if an error occurred while accessing the data store
- {@code COMMAND_FAILED} if some unspecified error is encountered while attempting to complete the command

`SecurityException`– if the caller does not have the necessary permissions to execute the underlying management operation

117.15.6.10 public abstract boolean isLeafNode(String[][] nodePath) throws DmtException

nodePath the absolute path of the node

- Tells whether a node is a leaf or an interior node of the DMT.

Returns true if the given node is a leaf node

Throws DmtException– with the following possible error codes:

- {@code NODE_NOT_FOUND} if {@code nodePath} points to a non-existing node
- {@code METADATA_MISMATCH} if the information could not be retrieved because of meta-data restrictions
- {@code DATA_STORE_FAILURE} if an error occurred while accessing the data store
- {@code COMMAND_FAILED} if some unspecified error is encountered while attempting to complete the command

SecurityException– if the caller does not have the necessary permissions to execute the underlying management operation

117.15.6.11 public abstract boolean isNodeUri(String[][] nodePath)

nodePath the absolute path to check

- Check whether the specified path corresponds to a valid node in the DMT.

Returns true if the given node exists in the DMT

117.15.6.12 public abstract void nodeChanged(String[][] nodePath) throws DmtException

nodePath the absolute path of the node that has changed

- Notifies the plugin that the given node has changed outside the scope of the plugin, therefore the Version and Timestamp properties must be updated (if supported). This method is needed because the ACL property of a node is managed by the DmtAdmin instead of the plugin. The DmtAdmin must call this method whenever the ACL property of a node changes.

Throws DmtException– with the following possible error codes:

- {@code NODE_NOT_FOUND} if {@code nodePath} points to a non-existing node
- {@code DATA_STORE_FAILURE} if an error occurred while accessing the data store
- {@code COMMAND_FAILED} if some unspecified error is encountered while attempting to complete the command

117.15.7 public abstract ReadWriteDataSession extends ReadableDataSession

Provides non-atomic read-write access to the part of the tree handled by the plugin that created this session.

The {@code nodePath} parameters appearing in this interface always contain an array of path segments identifying a node in the subtree of this plugin. This parameter contains an absolute path, so the first segment is always ".". Special characters appear escaped in the segments.

Error handling

When a tree manipulation command is called on the DmtAdmin service, it must perform an extensive set of checks on the parameters and the authority of the caller before delegating the call to a plugin. Therefore plugins can take certain circumstances for granted: that the path is valid and is within the subtree of the plugin and the session, the command can be applied to the given node (e.g. the target of {@code setNodeValue} is a leaf node), etc. All errors described by the error codes {@code DmtException#INVALID_URI}, {@code DmtException#URI_TOO_LONG}, {@code DmtException#PERMISSION_DENIED}, {@code DmtException#COMMAND_NOT_ALLOWED} and

{@link DmtException#TRANSACTION_ERROR} are fully filtered out before control reaches the plugin.

If the plugin provides meta-data for a node, the DmtAdmin service must also check the constraints specified by it, as described in {@link MetaNode}. If the plugin does not provide meta-data, it must perform the necessary checks for itself and use the {@link DmtException#METADATA_MISMATCH} error code to indicate such discrepancies.

The DmtAdmin does not check that the targeted node exists (or that it does not exist, in case of a node creation) before calling the plugin. It is the responsibility of the plugin to perform this check and to throw a {@link DmtException#NODE_NOT_FOUND} or {@link DmtException#NODE_ALREADY_EXISTS} if needed. In this case the DmtAdmin must pass through this exception to the caller of the corresponding DmtSession method.

The plugin can use the remaining error codes as needed. If an error does not fit into any other category, the {@link DmtException#COMMAND_FAILED} code should be used.

117.15.7.1 **public abstract void copy(String[][] nodePath, String[][] newNodePath, boolean recursive) throws DmtException**

nodePath an absolute path specifying the node or the root of a subtree to be copied

newNodePath the absolute path of the new node or root of a subtree

recursive {@code false} if only a single node is copied, {@code true} if the whole subtree is copied

- Create a copy of a node or a whole subtree. Beside the structure and values of the nodes, most properties managed by the plugin must also be copied, with the exception of the Timestamp and Version properties.

Throws DmtException– with the following possible error codes:

- {@code NODE_NOT_FOUND} if {@code nodePath} points to a non-existing node, or if {@code newNodePath} points to a node that cannot exist in the tree
- {@code NODE_ALREADY_EXISTS} if {@code newNodePath} points to a node that already exists
- {@code METADATA_MISMATCH} if the node could not be copied because of meta-data restrictions
- {@code FEATURE_NOT_SUPPORTED} if the copy operation is not supported by the plugin
- {@code DATA_STORE_FAILURE} if an error occurred while accessing the data store
- {@code COMMAND_FAILED} if some unspecified error is encountered while attempting to complete the command

SecurityException– if the caller does not have the necessary permissions to execute the underlying management operation

See Also DmtSession.copy(String, String, boolean)

117.15.7.2 **public abstract void createInteriorNode(String[][] nodePath, String type) throws DmtException**

nodePath the absolute path of the node to create

type the type URI of the interior node, can be {@code null} if no node type is defined

- Create an interior node with a given type. The type of interior node, if specified, is a URI identifying a DDF document.

Throws DmtException– with the following possible error codes:

- {@code NODE_NOT_FOUND} if {@code nodePath} points to a node that cannot exist in the tree
- {@code NODE_ALREADY_EXISTS} if {@code nodeUri} points to a node that already exists
- {@code METADATA_MISMATCH} if the node could not be created because of meta-data restrictions

- {@code DATA_STORE_FAILURE} if an error occurred while accessing the data store
- {@code COMMAND_FAILED} if some unspecified error is encountered while attempting to complete the command

SecurityException– if the caller does not have the necessary permissions to execute the underlying management operation

See Also `DmtSession.createInteriorNode(String)`, `DmtSession.createInteriorNode(String, String)`

117.15.7.3 **public abstract void createLeafNode(String[][] nodePath, DmtData value, String mimeType) throws DmtException**

nodePath the absolute path of the node to create

value the value to be given to the new node, can be {@code null}

mimeType the MIME type to be given to the new node, can be {@code null}

- Create a leaf node with a given value and MIME type. If the specified value or MIME type is {@code null}, their default values must be taken.

Throws DmtException– with the following possible error codes:

- {@code NODE_NOT_FOUND} if {@code nodePath} points to a node that cannot exist in the tree
- {@code NODE_ALREADY_EXISTS} if {@code nodePath} points to a node that already exists
- {@code METADATA_MISMATCH} if the node could not be created because of meta-data restrictions
- {@code DATA_STORE_FAILURE} if an error occurred while accessing the data store
- {@code COMMAND_FAILED} if some unspecified error is encountered while attempting to complete the command

SecurityException– if the caller does not have the necessary permissions to execute the underlying management operation

See Also `DmtSession.createLeafNode(String)`, `DmtSession.createLeafNode(String, DmtData)`, `DmtSession.createLeafNode(String, DmtData, String)`

117.15.7.4 **public abstract void deleteNode(String[][] nodePath) throws DmtException**

nodePath the absolute path of the node to delete

- Delete the given node. Deleting interior nodes is recursive, the whole subtree under the given node is deleted.

Throws DmtException– with the following possible error codes:

- {@code NODE_NOT_FOUND} if {@code nodePath} points to a non-existing node
- {@code METADATA_MISMATCH} if the node could not be deleted because of meta-data restrictions
- {@code DATA_STORE_FAILURE} if an error occurred while accessing the data store
- {@code COMMAND_FAILED} if some unspecified error is encountered while attempting to complete the command

SecurityException– if the caller does not have the necessary permissions to execute the underlying management operation

See Also `DmtSession.deleteNode(String)`

117.15.7.5 **public abstract void renameNode(String[][] nodePath, String newName) throws DmtException**

nodePath the absolute path of the node to rename

newName the new name property of the node

- Rename a node. This operation only changes the name of the node (updating the timestamp and version properties if they are supported), the value and the other properties are not changed. The new name of the node must be provided, the new path is constructed from the base of the old path and the given name.

Throws DmtException– with the following possible error codes:

- {@code NODE_NOT_FOUND} if {@code nodePath} points to a non-existing node, or if the new node is not defined in the tree
- {@code NODE_ALREADY_EXISTS} if there already exists a sibling of {@code nodePath} with the name {@code newName}
- {@code METADATA_MISMATCH} if the node could not be renamed because of meta-data restrictions
- {@code DATA_STORE_FAILURE} if an error occurred while accessing the data store
- {@code COMMAND_FAILED} if some unspecified error is encountered while attempting to complete the command

SecurityException– if the caller does not have the necessary permissions to execute the underlying management operation

See Also DmtSession.renameNode(String, String)

117.15.7.6 **public abstract void setNodeTitle(String[][] nodePath, String title) throws DmtException**

nodePath the absolute path of the node

title the title text of the node, can be {@code null}

- Set the title property of a node. The length of the title is guaranteed not to exceed the limit of 255 bytes in UTF-8 encoding.

Throws DmtException– with the following possible error codes:

- {@code NODE_NOT_FOUND} if {@code nodePath} points to a non-existing node
- {@code METADATA_MISMATCH} if the title could not be set because of meta-data restrictions
- {@code FEATURE_NOT_SUPPORTED} if the Title property is not supported by the plugin
- {@code DATA_STORE_FAILURE} if an error occurred while accessing the data store
- {@code COMMAND_FAILED} if some unspecified error is encountered while attempting to complete the command

SecurityException– if the caller does not have the necessary permissions to execute the underlying management operation

See Also DmtSession.setNodeTitle(String, String)

117.15.7.7 **public abstract void setNodeType(String[][] nodePath, String type) throws DmtException**

nodePath the absolute path of the node

type the type of the node, can be {@code null}

- Set the type of a node. The type of leaf node is the MIME type of the data it contains. The type of an interior node is a URI identifying a DDF document.

For interior nodes, the {@code null} type should remove the reference (if any) to a DDF document overriding the tree structure defined by the ancestors. For leaf nodes, it requests that the default MIME type is used for the given node.

Throws DmtException– with the following possible error codes:

- {@code NODE_NOT_FOUND} if {@code nodePath} points to a non-existing node
- {@code METADATA_MISMATCH} if the type could not be set because of meta-data restrictions

- {@code DATA_STORE_FAILURE} if an error occurred while accessing the data store
- {@code COMMAND_FAILED} if some unspecified error is encountered while attempting to complete the command

SecurityException– if the caller does not have the necessary permissions to execute the underlying management operation

See Also `DmtSession.setNodeType(String, String)`

117.15.7.8 **public abstract void setNodeValue(String[][] nodePath, DmtData data) throws DmtException**

nodePath the absolute path of the node

data the data to be set, can be {@code null}

- Set the value of a leaf or interior node. The format of the node is contained in the {@code DmtData} object. For interior nodes, the format is {@code FORMAT_NODE}, while for leaf nodes this format is never used.

If the specified value is {@code null}, the default value must be taken; if there is no default value, a {@code DmtException} with error code {@code METADATA_MISMATCH} must be thrown.

Throws `DmtException`– with the following possible error codes:

- {@code NODE_NOT_FOUND} if {@code nodePath} points to a non-existing node
- {@code METADATA_MISMATCH} if the value could not be set because of meta-data restrictions
- {@code FEATURE_NOT_SUPPORTED} if the specified node is an interior node and does not support Java object values
- {@code DATA_STORE_FAILURE} if an error occurred while accessing the data store
- {@code COMMAND_FAILED} if some unspecified error is encountered while attempting to complete the command

SecurityException– if the caller does not have the necessary permissions to execute the underlying management operation

See Also `DmtSession.setNodeValue(String, DmtData)`

117.15.8 **public abstract TransactionalDataSession extends ReadWriteDataSession**

Provides atomic read-write access to the part of the tree handled by the plugin that created this session.

117.15.8.1 **public abstract void commit() throws DmtException**

- Commits a series of DMT operations issued in the current atomic session since the last transaction boundary. Transaction boundaries are the creation of this object that starts the session, and all subsequent {@code commit()} and {@code rollback()} calls.

This method can fail even if all operations were successful. This can happen due to some multi-node semantic constraints defined by a specific implementation. For example, node A can be required to always have children A/B, A/C and A/D. If this condition is broken when {@code commit()} is executed, the method will fail, and throw a {@code METADATA_MISMATCH} exception.

In many cases the tree is not the only way to manage a given part of the system. It may happen that while modifying some nodes in an atomic session, the underlying settings are modified in parallel outside the scope of the DMT. If this is detected during commit, an exception with the code {@code CONCURRENT_ACCESS} is thrown.

Throws `DmtException`– with the following possible error codes

- {@code METADATA_MISMATCH} if the operation failed because of meta-data restrictions

- {@code CONCURRENT_ACCESS} if it is detected that some modification has been made outside the scope of the DMT to the nodes affected in the session's operations
- {@code DATA_STORE_FAILURE} if an error occurred while accessing the data store
- {@code COMMAND_FAILED} if some unspecified error is encountered while attempting to complete the command

SecurityException – if the caller does not have the necessary permissions to execute the underlying management operation

117.15.8.2 **public abstract void rollback() throws DmtException**

- Rolls back a series of DMT operations issued in the current atomic session since the last transaction boundary. Transaction boundaries are the creation of this object that starts the session, and all subsequent {@link #commit} and {@link #rollback} calls.

Throws DmtException – with the error code {@code ROLLBACK_FAILED} in case the rollback did not succeed

SecurityException – if the caller does not have the necessary permissions to execute the underlying management operation

117.16 **org.osgi.service.dmt.notification**

Device Management Tree Notification Package Version 2.0.

This package contains the public API of the Notification service. This service enables the sending of asynchronous notifications to management servers. Permission classes are provided by the {@code org.osgi.service.dmt.security} package.

Bundles wishing to use this package must list the package in the Import-Package header of the bundle's manifest. This package has two types of users: the consumers that use the API in this package and the providers that implement the API in this package.

Example import for consumers using the API in this package:

```
{@code Import-Package: org.osgi.service.dmt.notification; version="[2.0,3.0)"}
```

Example import for providers implementing the API in this package:

```
{@code Import-Package: org.osgi.service.dmt.notification; version="[2.0,2.1)"}
```

117.16.1 **Summary**

- AlertItem - Immutable data structure carried in an alert (client initiated notification).
- NotificationService - NotificationService enables sending asynchronous notifications to a management server.

117.16.2 **public class AlertItem**

Immutable data structure carried in an alert (client initiated notification). The {@code AlertItem} describes details of various notifications that can be sent by the client, for example as alerts in the OMA DM protocol. The use cases include the client sending a session request to the server (alert 1201), the client notifying the server of completion of a software update operation (alert 1226) or sending back results in response to an asynchronous EXEC command.

The data syntax and semantics varies widely between various alerts, so does the optionality of particular parameters of an alert item. If an item, such as source or type, is not defined, the corresponding getter method returns {@code null}. For example, for alert 1201 (client-initiated session) all elements will be {@code null}.

The syntax used in {@code AlertItem} class corresponds to the OMA DM alert format. {@link NotificationService} implementations on other management protocols should map these constructs to the underlying protocol.

117.16.2.1 **public AlertItem(String source, String type, String mark, DmtData data)**

source the URI of the node which is the source of the alert item

type a MIME type or a URN that identifies the type of the data in the alert item

data a {@code DmtData} object that contains the format and value of the data in the alert item

mark the mark parameter of the alert item

- Create an instance of the alert item. The constructor takes all possible data entries as parameters. Any of these parameters can be {@code null}. The semantics of the parameters may be refined by the definition of a specific alert, identified by its alert code (see {@link NotificationService#sendNotification(String, int, String, AlertItem[])}). In case of Generic Alerts for example (code 1226), the {@code mark} parameter contains a severity string.

117.16.2.2 **public AlertItem(String[] source, String type, String mark, DmtData data)**

source the path of the node which is the source of the alert item

type a MIME type or a URN that identifies the type of the data in the alert item

data a {@code DmtData} object that contains the format and value of the data in the alert item

mark the mark parameter of the alert item

- Create an instance of the alert item, specifying the source node URI as an array of path segments. The constructor takes all possible data entries as parameters. Any of these parameters can be {@code null}. The semantics of the parameters may be refined by the definition of a specific alert, identified by its alert code (see {@link NotificationService#sendNotification(String, int, String, AlertItem[])}). In case of Generic Alerts for example (code 1226), the {@code mark} parameter contains a severity string.

117.16.2.3 **public DmtData getData()**

- Get the data associated with the alert item. The returned {@code DmtData} object contains the format and the value of the data in the alert item. There might be no data associated with the alert item.

Returns the data associated with the alert item, or {@code null} if there is no data

117.16.2.4 **public String getMark()**

- Get the mark parameter associated with the alert item. The interpretation of the mark parameter depends on the alert being sent, as identified by the alert code in {@link NotificationService#sendNotification(String, int, String, AlertItem[])}). There might be no mark associated with the alert item.

Returns the mark associated with the alert item, or {@code null} if there is no mark

117.16.2.5 **public String getSource()**

- Get the node which is the source of the alert. There might be no source associated with the alert item.

Returns the URI of the node which is the source of this alert, or {@code null} if there is no source

117.16.2.6 **public String getType()**

- Get the type associated with the alert item. The type string is a MIME type or a URN that identifies the type of the data in the alert item (returned by {@link #getData()}). There might be no type associated with the alert item.

Returns the type associated with the alert item, or {@code null} if there is no type

117.16.2.7 **public String toString()**

- Returns the string representation of this alert item. The returned string includes all parameters of the alert item, and has the following format:

```
AlertItem(<source>, <type>, <mark>, <data>)
```

The last parameter is the string representation of the data value. The format of the data is not explicitly included.

Returns the string representation of this alert item

117.16.3 **public abstract NotificationService**

NotificationService enables sending asynchronous notifications to a management server. The implementation of {@code NotificationService} should register itself in the OSGi service registry as a service.

117.16.3.1 **public abstract void sendNotification(String principal, int code, String correlator, AlertItem[] items) throws DmtException**

principal the principal name which is the recipient of this notification, can be {@code null}

code the alert code, can be 0 if not needed

correlator optional field that contains the correlation identifier of an associated exec command, can be {@code null} if not needed

items the data of the alert items carried in this alert, can be {@code null} or empty if not needed

- Sends a notification to a named principal. It is the responsibility of the {@code NotificationService} to route the notification to the given principal using the registered {@link org.osgi.service.dmt.notification.spi.RemoteAlertSender} services.

In remotely initiated sessions the principal name identifies the remote server that created the session, this can be obtained using the session's {@link DmtSession#getPrincipal() getPrincipal} call.

The principal name may be omitted if the client does not know the principal name. Even in this case the routing might be possible if the Notification Service finds an appropriate default destination (for example if it is only connected to one protocol adapter, which is only connected to one management server).

Since sending the notification and receiving acknowledgment for it is potentially a very time-consuming operation, notifications are sent asynchronously. This method should attempt to ensure that the notification can be sent successfully, and should throw an exception if it detects any problems. If the method returns without error, the notification is accepted for sending and the implementation must make a best-effort attempt to deliver it.

In case the notification is an asynchronous response to a previous {@link DmtSession#execute(String, String, String) execute} command, a correlation identifier can be specified to provide the association between the execute and the notification.

In order to send a notification using this method, the caller must have an {@code AlertPermission} with a target string matching the specified principal name. If the {@code principal} parameter is {@code null} (the principal name is not known), the target of the {@code AlertPermission} must be "*".

When this method is called with null correlator, null or empty AlertItem array, and a 0 code as values, it should send a protocol specific default notification to initiate a management session. For example, in case of OMA DM this is alert 1201 "Client Initiated Session". The {@code principal} parameter can be used to determine the recipient of the session initiation request.

Throws DmtException – with the following possible error codes:

- {@code UNAUTHORIZED} when the remote server rejected the request due to insufficient authorization
- {@code ALERT_NOT_ROUTED} when the alert can not be routed to the given principal
- {@code REMOTE_ERROR} in case of communication problems between the device and the destination
- {@code COMMAND_FAILED} for unspecified errors encountered while attempting to complete the command
- {@code FEATURE_NOT_SUPPORTED} if the underlying management protocol doesn't support asynchronous notifications

SecurityException – if the caller does not have the required {@code AlertPermission} with a target matching the {@code principal} parameter, as described above

117.17 org.osgi.service.dmt.notification.spi

Device Management Tree Notification SPI Package Version 2.0.

This package contains the SPI (Service Provider Interface) of the Notification service. These interfaces are implemented by Protocol Adapters capable of delivering notifications to management servers on a specific protocol. Users of the {@code NotificationService} interface do not interact directly with this package.

Bundles wishing to use this package must list the package in the Import-Package header of the bundle's manifest. This package has two types of users: the consumers that use the API in this package and the providers that implement the API in this package.

Example import for consumers using the API in this package:

```
{@code Import-Package: org.osgi.service.dmt.notification.spi; version="[2.0,3.0)"}
```

Example import for providers implementing the API in this package:

```
{@code Import-Package: org.osgi.service.dmt.notification.spi; version="[2.0,2.1)"}
```

117.17.1 Summary

- RemoteAlertSender - The RemoteAlertSender can be used to send notifications to (remote) entities identified by principal names.

117.17.2 public abstract RemoteAlertSender

The RemoteAlertSender can be used to send notifications to (remote) entities identified by principal names. This service is provided by Protocol Adapters, and is used by the {@link org.osgi.service.dmt.notification.NotificationService} when sending alerts. Implementations of this interface have to be able to connect and send alerts to one or more management servers in a protocol specific way.

The properties of the service registration should specify a list of destinations (principals) where the service is capable of sending alerts. This can be done by providing a {@code String} array of principal names in the {@code principals} registration property. If this property is not registered, the service will be treated as the default sender. The default alert sender is only used when a more specific alert sender cannot be found.

The {@code principals} registration property is used when the {@link org.osgi.service.dmt.notification.NotificationService#sendNotification(String, int, String, AlertItem[])} method is called, to find the proper {@code RemoteAlertSender} for the given destination. If the caller does not specify a

principal, the alert is only sent if the Notification Sender finds a default alert sender, or if the choice is unambiguous for some other reason (for example if only one alert sender is registered).

117.17.2.1 **public abstract void sendAlert(String principal, int code, String correlator, AlertItem[][] items) throws Exception**

principal the name identifying the server where the alert should be sent, can be {@code null}

code the alert code, can be 0 if not needed

correlator the correlation identifier of an associated EXEC command, or {@code null} if there is no associated EXEC

items the data of the alert items carried in this alert, can be empty or {@code null} if no alert items are needed

- Sends an alert to a server identified by its principal name. In case the alert is sent in response to a previous {@code link org.osgi.service.dmt.DmtSession#execute(String, String, String) execute} command, a correlation identifier can be specified to provide the association between the execute and the alert.

The {@code principal} parameter specifies which server the alert should be sent to. This parameter can be {@code null} if the client does not know the name of the destination. The alert should still be delivered if possible; for example if the alert sender is only connected to one destination.

Any exception thrown on this method will be propagated to the original sender of the event, wrapped in a {@code DmtException} with the code {@code REMOTE_ERROR}.

Since sending the alert and receiving acknowledgment for it is potentially a very time-consuming operation, alerts are sent asynchronously. This method should attempt to ensure that the alert can be sent successfully, and should throw an exception if it detects any problems. If the method returns without error, the alert is accepted for sending and the implementation must make a best-effort attempt to deliver it.

Throws Exception – if the alert can not be sent to the server

117.18 **org.osgi.service.dmt.security**

Device Management Tree Security Package Version 2.0.

This package contains the permission classes used by the Device Management API in environments that support the Java 2 security model.

Bundles wishing to use this package must list the package in the Import-Package header of the bundle's manifest. This package has two types of users: the consumers that use the API in this package and the providers that implement the API in this package.

Example import for consumers using the API in this package:

```
{@code Import-Package: org.osgi.service.dmt.security; version="[2.0,3.0)"}
```

Example import for providers implementing the API in this package:

```
{@code Import-Package: org.osgi.service.dmt.security; version="[2.0,2.1)"}
```

117.18.1 **Summary**

- **AlertPermission** - Indicates the callers authority to send alerts to management servers, identified by their principal names.
- **DmtPermission** - Controls access to management objects in the Device Management Tree (DMT).

- `DmtPrincipalPermission` - Indicates the callers authority to create DMT sessions on behalf of a remote management server.

117.18.2 **public class AlertPermission** **extends Permission**

Indicates the callers authority to send alerts to management servers, identified by their principal names.

{@code AlertPermission} has a target string which controls the principal names where alerts can be sent. A wildcard is allowed at the end of the target string, to allow sending alerts to any principal with a name matching the given prefix. The "*" target means that alerts can be sent to any destination.

117.18.2.1 **public AlertPermission(String target)**

target the name of a principal, can end with {@code *} to match any principal identifier with the given prefix

- Creates a new {@code AlertPermission} object with its name set to the target string. Name must be non-null and non-empty.

Throws `NullPointerException` – if {@code name} is {@code null}
`IllegalArgumentException` – if {@code name} is empty

117.18.2.2 **public AlertPermission(String target, String actions)**

target the name of the server, can end with {@code *} to match any server identifier with the given prefix

actions no actions defined, must be "*" for forward compatibility

- Creates a new {@code AlertPermission} object using the 'canonical' two argument constructor. In this version this class does not define any actions, the second argument of this constructor must be "*" so that this class can later be extended in a backward compatible way.

Throws `NullPointerException` – if {@code name} or {@code actions} is {@code null}
`IllegalArgumentException` – if {@code name} is empty or {@code actions} is not "*"

117.18.2.3 **public boolean equals(Object obj)**

obj the object to compare to this `AlertPermission` instance

- Checks whether the given object is equal to this `AlertPermission` instance. Two `AlertPermission` instances are equal if they have the same target string.

Returns {@code true} if the parameter represents the same permissions as this instance

117.18.2.4 **public String getActions()**

- Returns the action list (always {@code *} in the current version).

Returns the action string "*"

117.18.2.5 **public int hashCode()**

- Returns the hash code for this permission object. If two `AlertPermission` objects are equal according to the {@code equals(Object)} method, then calling this method on each of the two `AlertPermission` objects must produce the same integer result.

Returns hash code for this permission object

117.18.2.6 **public boolean implies(Permission p)**

p the permission to check for implication

- Checks if this AlertPermission object implies the specified permission. Another AlertPermission instance is implied by this permission either if the target strings are identical, or if this target can be made identical to the other target by replacing a trailing "*" with any string.

Returns true if this AlertPermission instance implies the specified permission

117.18.2.7 **public PermissionCollection newPermissionCollection()**

- Returns a new PermissionCollection object for storing AlertPermission objects.

Returns the new PermissionCollection

117.18.3 **public class DmtPermission extends Permission**

Controls access to management objects in the Device Management Tree (DMT). It is intended to control local access to the DMT. DmtPermission target string identifies the management object URI and the action field lists the OMA DM commands that are permitted on the management object. Example:

```
DmtPermission("./OSGi/bundles", "Add, Replace, Get");
```

This means that owner of this permission can execute Add, Replace and Get commands on the ./OSGi/bundles management object. It is possible to use wildcards in both the target and the actions field. Wildcard in the target field means that the owner of the permission can access children nodes of the target node. Example:

```
DmtPermission("./OSGi/bundles/*", "Get");
```

This means that owner of this permission has Get access on every child node of ./OSGi/bundles. The asterisk does not necessarily have to follow a '/' character. For example the {@code "./OSGi/a*"} target matches the {@code "./OSGi/applications"} subtree.

If wildcard is present in the actions field, all legal OMA DM commands are allowed on the designated nodes(s) by the owner of the permission. Action names are interpreted case-insensitively, but the canonical action string returned by {@code #getActions()} uses the forms defined by the action constants.

117.18.3.1 **public static final String ADD = "Add"**

Holders of DmtPermission with the Add action present can create new nodes in the DMT, that is they are authorized to execute the createInteriorNode() and createLeafNode() methods of the DmtSession. This action is also required for the copy() command, which needs to perform node creation operations (among others).

117.18.3.2 **public static final String DELETE = "Delete"**

Holders of DmtPermission with the Delete action present can delete nodes from the DMT, that is they are authorized to execute the deleteNode() method of the DmtSession.

117.18.3.3 **public static final String EXEC = "Exec"**

Holders of DmtPermission with the Exec action present can execute nodes in the DMT, that is they are authorized to call the execute() method of the DmtSession.

117.18.3.4 **public static final String GET = "Get"**

Holders of DmtPermission with the Get action present can query DMT node value or properties, that is they are authorized to execute the isLeafNode(), getNodeAcl(), getEffectiveNodeAcl(), getMetaNode(), getNodeValue(), getChildNodeNames(), getNodeTitle(), getNodeVersion(), getNodeTimeStamp(), getNodeSize() and getNodeType() methods of the DmtSession. This action is also required for the copy() command, which needs to perform node query operations (among others).

117.18.3.5 public static final String REPLACE = "Replace"

Holders of `DmtPermission` with the Replace action present can update DMT node value or properties, that is they are authorized to execute the `setNodeAcl()`, `setNodeTitle()`, `setNodeValue()`, `setNodeType()` and `renameNode()` methods of the `DmtSession`. This action is also be required for the `copy()` command if the original node had a title property (which must be set in the new node).

117.18.3.6 public DmtPermission(String dmtUri, String actions)

dmtUri URI of the management object (or subtree)

actions OMA DM actions allowed

- Creates a new `DmtPermission` object for the specified DMT URI with the specified actions. The given URI can be:
 - `{@code "*"}`, which matches all valid (see `{@link Uri#isValidUri(String)}`) absolute URIs;
 - the prefix of an absolute URI followed by the `{@code *}` character (for example `{@code "./OS-Gi/L*"}`), which matches all valid absolute URIs beginning with the given prefix;
 - a valid absolute URI, which matches itself.

Since the `{@code *}` character is itself a valid URI character, it can appear as the last character of a valid absolute URI. To distinguish this case from using `{@code *}` as a wildcard, the `{@code *}` character at the end of the URI must be escaped with the `{@code \}` character. For example the URI `{@code "./a*"} matches {@code "./a", {@code "./aa", {@code "./a/b" etc. while {@code "./a\*"} matches {@code "./a*"} only.`

The actions string must either be `"*"` to allow all actions, or it must contain a non-empty subset of the valid actions, defined as constants in this class.

Throws `NullPointerException`—if any of the parameters are `{@code null}`

`IllegalArgumentException`—if any of the parameters are invalid

117.18.3.7 public boolean equals(Object obj)

obj the object to compare to this `DmtPermission` instance

- Checks whether the given object is equal to this `DmtPermission` instance. Two `DmtPermission` instances are equal if they have the same target string and the same action mask. The `"*"` action mask is considered equal to a mask containing all actions.

Returns `{@code true}` if the parameter represents the same permissions as this instance

117.18.3.8 public String getActions()

- Returns the String representation of the action list. The allowed actions are listed in the following order: Add, Delete, Exec, Get, Replace. The wildcard character is not used in the returned string, even if the class was created using the `"*"` wildcard.

Returns canonical action list for this permission object

117.18.3.9 public int hashCode()

- Returns the hash code for this permission object. If two `DmtPermission` objects are equal according to the `{@link #equals(Object)}` method, then calling this method on each of the two `DmtPermission` objects must produce the same integer result.

Returns hash code for this permission object

117.18.3.10 public boolean implies(Permission p)

p the permission to check for implication

- Checks if this DmtPermission object "implies" the specified permission. This method returns `{@code false}` if and only if at least one of the following conditions are fulfilled for the specified permission:
 - it is not a DmtPermission
 - its set of actions contains an action not allowed by this permission
 - the set of nodes defined by its path contains a node not defined by the path of this permission

Returns true if this DmtPermission instance implies the specified permission

117.18.3.11 **public PermissionCollection newPermissionCollection()**

- Returns a new PermissionCollection object for storing DmtPermission objects.

Returns the new PermissionCollection

117.18.4 **public class DmtPrincipalPermission extends Permission**

Indicates the callers authority to create DMT sessions on behalf of a remote management server. Only protocol adapters communicating with management servers should be granted this permission.

`{@code DmtPrincipalPermission}` has a target string which controls the name of the principal on whose behalf the protocol adapter can act. A wildcard is allowed at the end of the target string, to allow using any principal name with the given prefix. The "*" target means the adapter can create a session in the name of any principal.

117.18.4.1 **public DmtPrincipalPermission(String target)**

target the name of the principal, can end with `{@code *}` to match any principal with the given prefix

- Creates a new `{@code DmtPrincipalPermission}` object with its name set to the target string. Name must be non-null and non-empty.

Throws NullPointerException – if `{@code name}` is `{@code null}`

IllegalArgumentException – if `{@code name}` is empty

117.18.4.2 **public DmtPrincipalPermission(String target, String actions)**

target the name of the principal, can end with `{@code *}` to match any principal with the given prefix

actions no actions defined, must be "*" for forward compatibility

- Creates a new `{@code DmtPrincipalPermission}` object using the 'canonical' two argument constructor. In this version this class does not define any actions, the second argument of this constructor must be "*" so that this class can later be extended in a backward compatible way.

Throws NullPointerException – if `{@code name}` or `{@code actions}` is `{@code null}`

IllegalArgumentException – if `{@code name}` is empty or `{@code actions}` is not "*"

117.18.4.3 **public boolean equals(Object obj)**

obj the object to compare to this DmtPrincipalPermission instance

- Checks whether the given object is equal to this DmtPrincipalPermission instance. Two DmtPrincipalPermission instances are equal if they have the same target string.

Returns `{@code true}` if the parameter represents the same permissions as this instance

117.18.4.4 **public String getActions()**

- Returns the action list (always `{@code *}` in the current version).

Returns the action string "*"

117.18.4.5 public int hashCode()

- Returns the hash code for this permission object. If two DmtPrincipalPermission objects are equal according to the {@link #equals(Object)} method, then calling this method on each of the two DmtPrincipalPermission objects must produce the same integer result.

Returns hash code for this permission object

117.18.4.6 public boolean implies(Permission p)

p the permission to check for implication

- Checks if this DmtPrincipalPermission object implies the specified permission. Another DmtPrincipalPermission instance is implied by this permission either if the target strings are identical, or if this target can be made identical to the other target by replacing a trailing "*" with any string.

Returns true if this DmtPrincipalPermission instance implies the specified permission

117.18.4.7 public PermissionCollection newPermissionCollection()

- Returns a new PermissionCollection object for storing DmtPrincipalPermission objects.

Returns the new PermissionCollection

117.19 References

- [1] *OMA DM-TND v1.2 draft*
https://www.openmobilealliance.org/release/DM/V1_2-20050607-C/OMA-TS-DM-TND-V1_2-20050607-C.pdf
- [2] *OMA DM-RepPro v1.2 draft*
https://www.openmobilealliance.org/release/DM/V1_2-20050607-C/OMA-TS-DM-Rep-Pro-V1_2-20050607-C.pdf
- [3] *IETF RFC2578, Structure of Management Information Version 2 (SMIv2)*
<https://www.ietf.org/rfc/rfc2578.txt>
- [4] *Java™ Management Extensions Instrumentation and Agent Specification v1.2, October 2002*,
<https://www.oracle.com/java/technologies/javase/javamanagement.html>
- [5] *JSR 9 - Federated Management Architecture (FMA) Specification*
Version 1.0, January 2000
<https://www.jcp.org/en/jsr/detail?id=9>
- [6] *WBEM Profile Template, DSP1000*
Status: Draft, Version 1.0 Preliminary, March 11, 2004
<https://www.dmtf.org/standards/wbem>
- [7] *SNMP*
http://www.wtcs.org/snmp4tpc/snmp_rfc.htm#rfc
- [8] *RFC 2396 Uniform Resource Identifiers (URI): Generic Syntax*
<https://www.ietf.org/rfc/rfc2396.txt>
- [9] *MIME Media Types*
<https://www.iana.org/assignments/media-types/>
- [10] *RFC 3548 The Base16, Base32, and Base64 Data Encodings*
<https://www.ietf.org/rfc/rfc3548.txt>
- [11] *Secure Hash Algorithm 1*
<https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.180-4.pdf>

- [12] *TR-069 CPE WAN Management Protocol (CWMP)*
Customer Premises Equipment Wide Area Network Management Protocol (CWMP)
<https://en.wikipedia.org/wiki/TR-069>
- [13] *XML Schema Part 2: Datatypes Second Edition*
<https://www.w3.org/TR/xmlschema-2/>

Licensed under the Eclipse Foundation Specification License – v1.0. Copyright © Contributors to the Eclipse Foundation

DRAFT

End Of Document

Licensed under the Eclipse Foundation Specification License – v1.0. Copyright © Contributors to the Eclipse Foundation

DRAFT